

Credit Card Fraud Detection

GROUP 007

MILESTONE PRESENTATION

5TH JUNE 2025

Namitha Navaneetha Krishnan

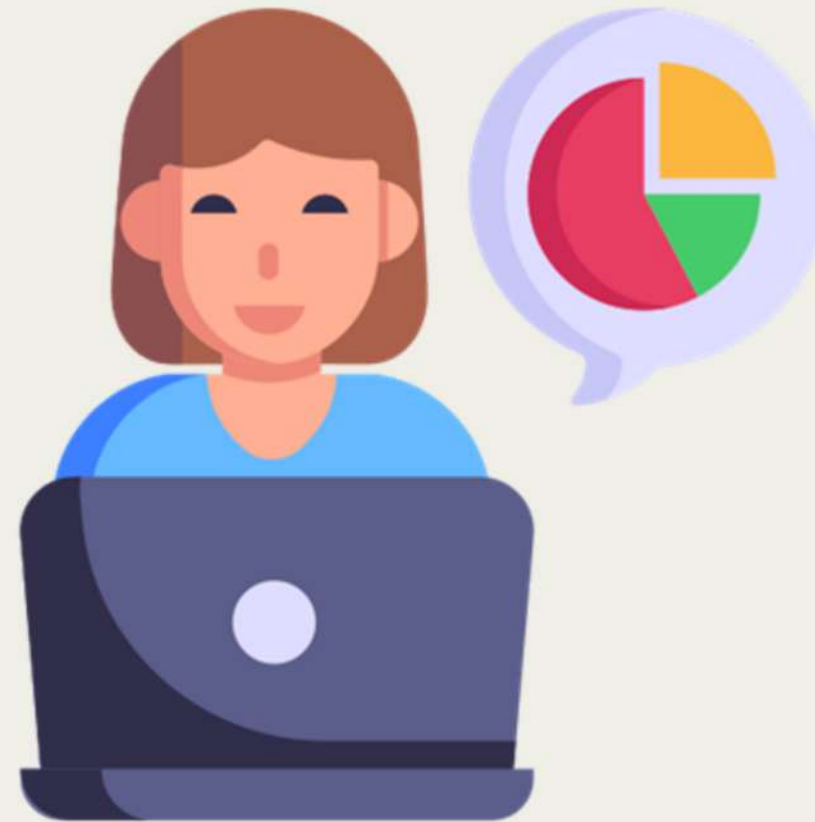
Prajnya Kalmane

Ashwin Srambikal

ROLES



Prajnya Kalmane



Namitha Navaneetha Krishnan



Ashwin Srambikal

CREDIT CARD FRAUD DETECTION

Each year, financial institutions lose more than \$30 billion to credit card fraud. How can we turn that around using AI?”

- Credit card fraud is not just a financial threat — it erodes customer trust and brand reputation.
- Artificial Intelligence offers a scalable, proactive approach to detecting fraud in real time.
- To solve this effectively, we must first understand the business and data mining goals.



BUSINESS UNDERSTANDING

Determine Business Objectives

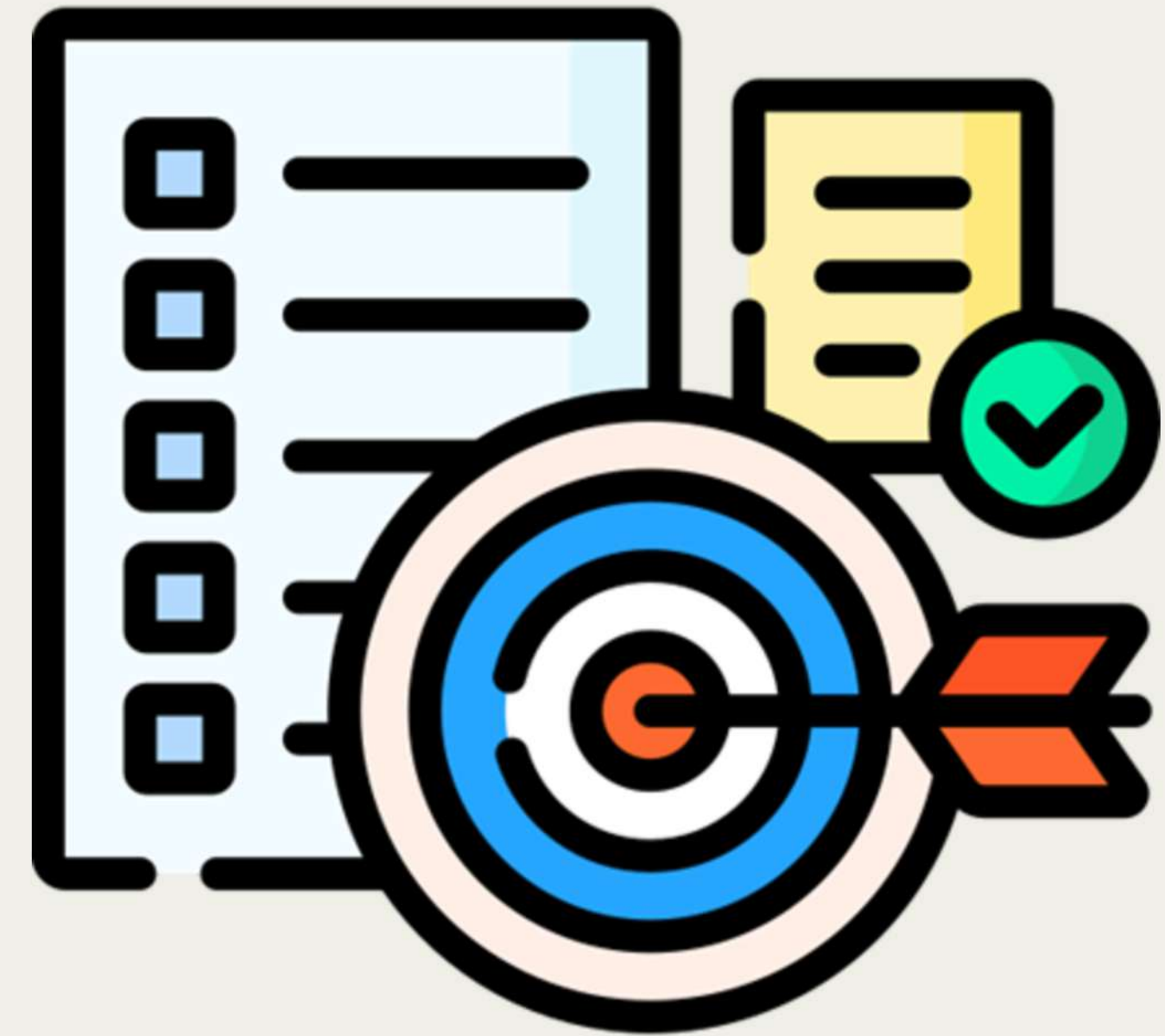
To minimize financial losses caused by credit card fraud while maintaining a positive customer experience.

- Fraudulent transactions result in billions of dollars lost annually.
- Financial institutions aim to detect fraud early and accurately to:

Protect customers from theft.

Reduce reimbursement costs.

Maintain trust and brand reputation.



BUSINESS UNDERSTANDING

Situation Assessment

- **High Financial Exposure:**

Credit card fraud results in multi-billion annual losses globally, directly impacting financial institutions' revenue and operational costs.

- **Customer Trust & Regulatory Compliance:**

Institutions must maintain consumer confidence while complying with strict financial regulations such as PCI-DSS and PSD2, demanding robust fraud monitoring systems.

- **Operational Trade-offs:**

There is a critical need to balance fraud detection accuracy with minimizing false positives, to avoid disrupting legitimate customer transactions and degrading user experience.

- **Scalability and Adaptability:**

Fraud patterns constantly evolve, necessitating adaptive machine learning models that can scale with transaction volume and respond to emerging threats.

DETERMINE DATA MINING GOALS

Objective for ML:

Build a predictive model that can

- Detect fraudulent transactions with high recall (catch as many frauds as possible).
- Maintain a low false positive rate to avoid inconveniencing real users.
- Be efficient and scalable for potential real-time integration.

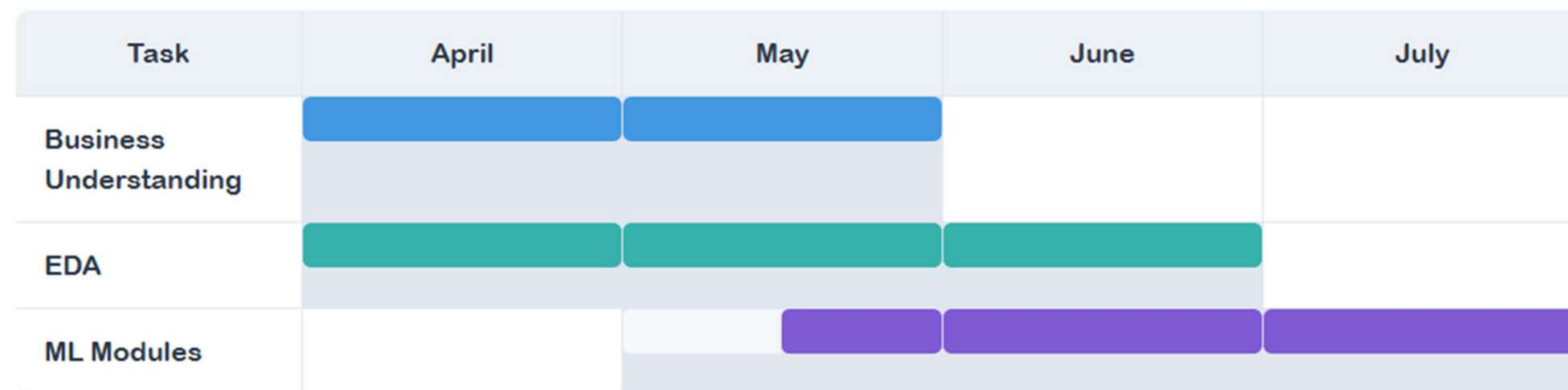
Explored Models:

- Logistic Regression
- Random Forest
- XGBoost



PROJECT TIMELINE

Project Timeline Gantt Chart



Business understanding

Defining Business Use-Case: To highlight how a data-driven solution will reduce losses, improve customer trust, and streamline operations, ultimately delivering a strong return on investment for financial institutions.

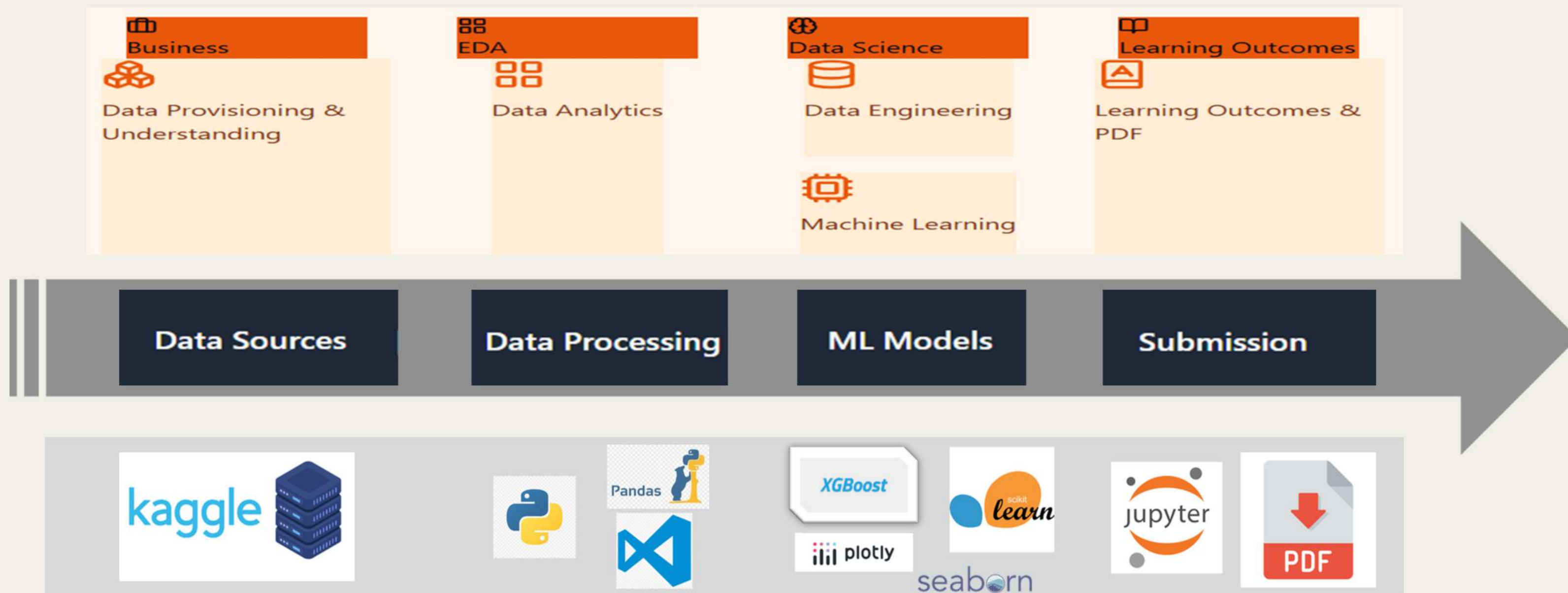
EDA

Loading of raw data in notebook, loading different libraries
Understanding, cleaning, standardizing, filtering data
Data Preparation Explore Data Data Quality Data Cleaning

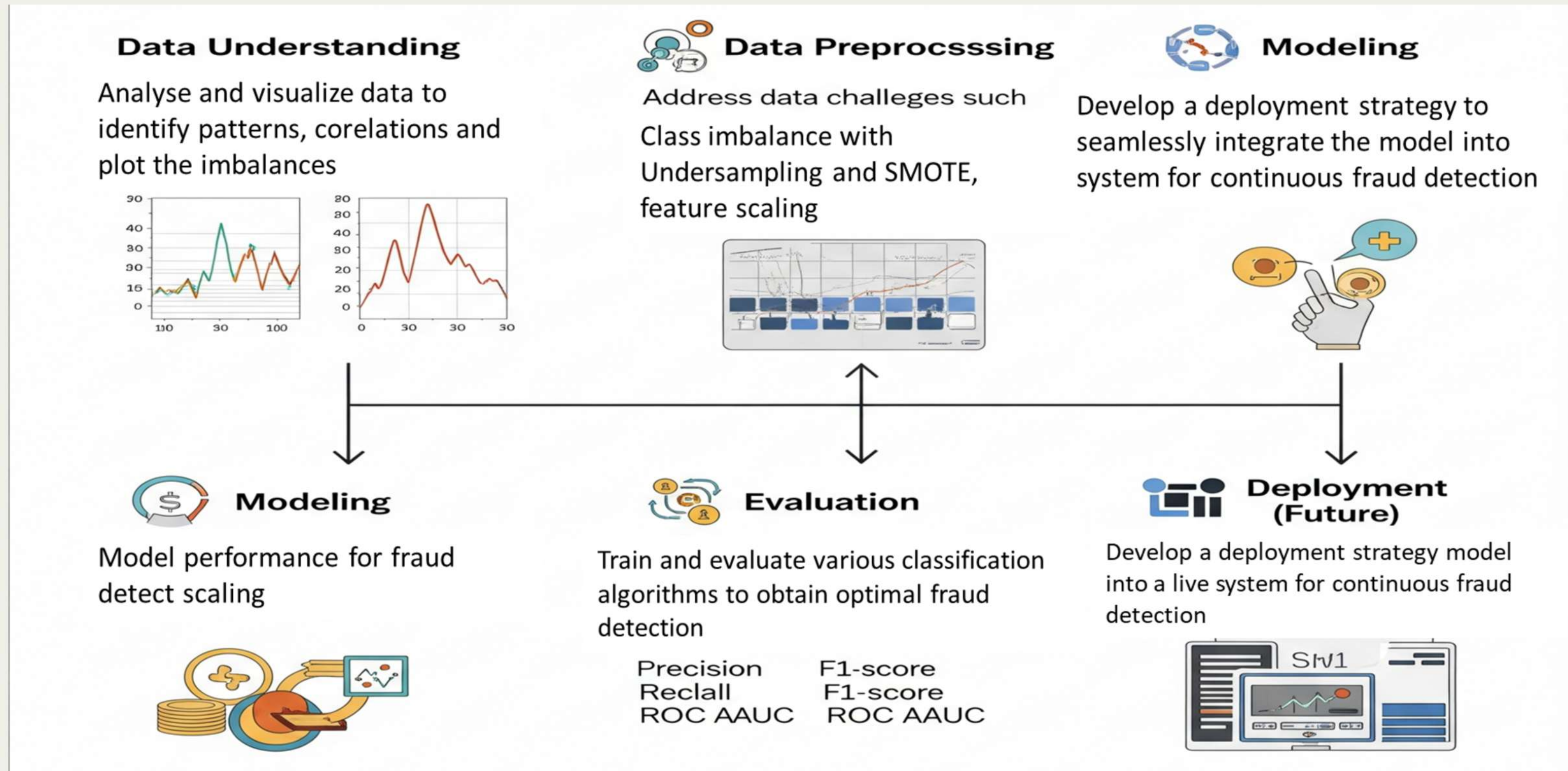
ML Modules

Justification & Development of 3 different ML modules.
ML Improvements: Feature engineering for different ML Models.
Analyze and select the best Improved ML models

TECHNOLOGICAL FOUNDATION

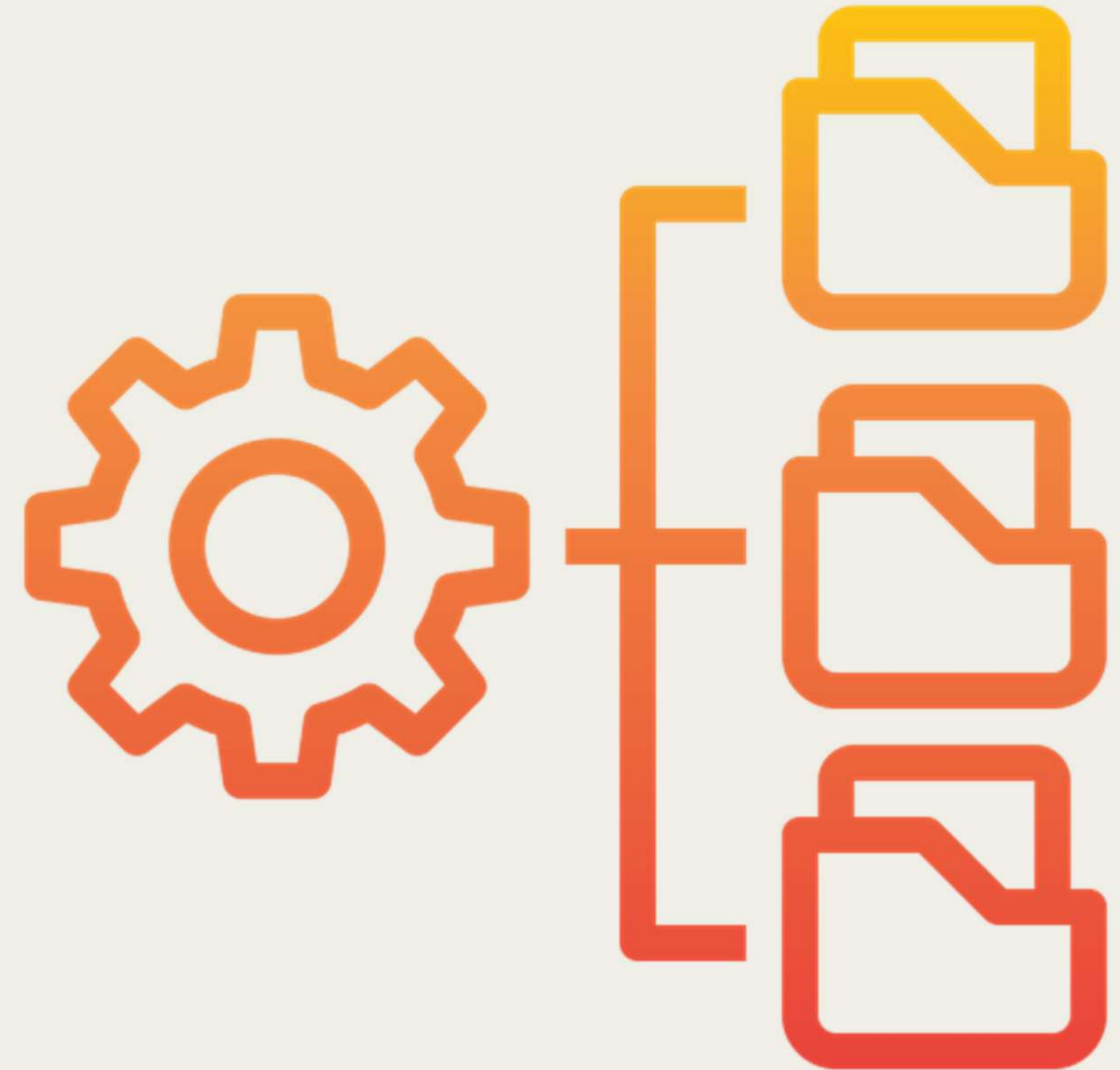


PRODUCE PROJECT PLAN



DATA PREPARATION & EXPLORATION

- **Collect Initial Data**
- **Describe Data**
- **Explore Data**
- **Verify Data Quality**



COLLECT INITIAL DATA

Objective: Gather relevant data for identifying fraudulent credit card transactions.

Source:

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
(Dataset from Kaggle – Credit Card Fraud Detection)

Overview:

- Total Rows: 284,807
- Total Columns: 31

Features include:

Time, Amount: Raw numerical features
V1–V28: PCA-transformed anonymized features
Class: Target variable (0 = Non-Fraud, 1 = Fraud)

File Used: creditcard.csv

Initial inspection shows a well-structured dataset suitable for binary classification.



DESCRIBE DATA

Goal: Understand each feature's type, distribution, and basic statistics.

Feature Types:

- All columns are numerical (including the target).
- No categorical or text data.

Descriptive Statistics:

Time: Elapsed time in seconds from the first transaction.

Amount: Varies from 0 to ~25,691, needs scaling.

V1–V28: Result of PCA transformation, anonymized features.

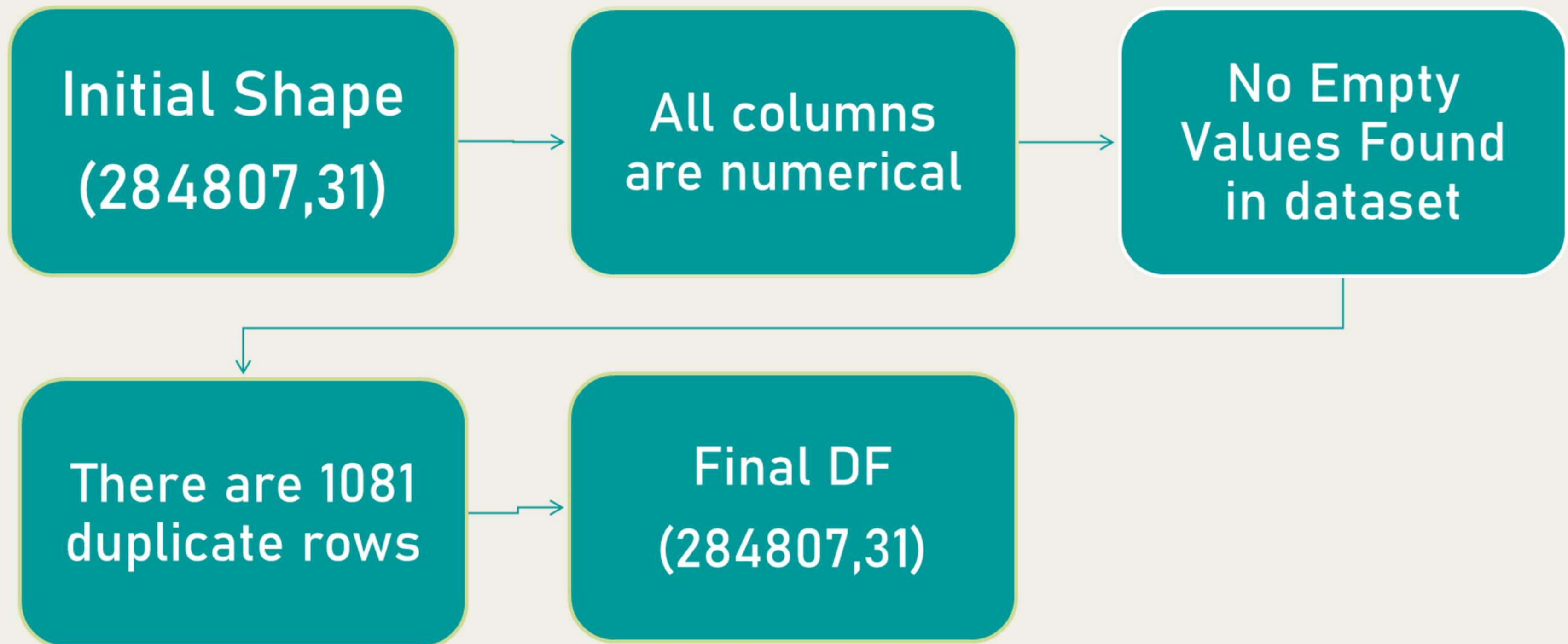
Class: Highly imbalanced binary target.

Summary Statistics:

- Columns like Amount and Time have large ranges.
- PCA features (V1–V28) show varying means and standard deviations.
- No extreme outliers at first glance.

Column Name	Non-Null Count	Unique Values	Data Type
Time	284807	124592	float64
V1	284807	275663	float64
V2	284807	275663	float64
V3	284807	275663	float64
V4	284807	275663	float64
V5	284807	275663	float64
V6	284807	275663	float64
V7	284807	275663	float64
V8	284807	275663	float64
V9	284807	275663	float64
V10	284807	275663	float64
V11	284807	275663	float64
V12	284807	275663	float64
V13	284807	275663	float64
V14	284807	275663	float64
V15	284807	275663	float64
V16	284807	275663	float64
V17	284807	275663	float64
V18	284807	275663	float64
V19	284807	275663	float64
V20	284807	275663	float64
V21	284807	275663	float64
V22	284807	275663	float64
V23	284807	275663	float64
V24	284807	275663	float64
V25	284807	275663	float64
V26	284807	275663	float64
V27	284807	275663	float64
V28	284807	275663	float64
Amount	284807	32767	float64
Class	284807	2	int64

DATA QUALITY



SUMMARIZE DATA QUALITY

Objective: Ensure data integrity before modeling.

Checks Performed:

•Missing Values:

None found in any of the 31 columns.

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	V27	V28	Amount	Class
Total	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Percent	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

•Duplicates:

1081 identified.

```
data.duplicated().sum()
np.int64(1081)
```

•Consistency:

All columns have consistent data types and valid ranges.

Result:

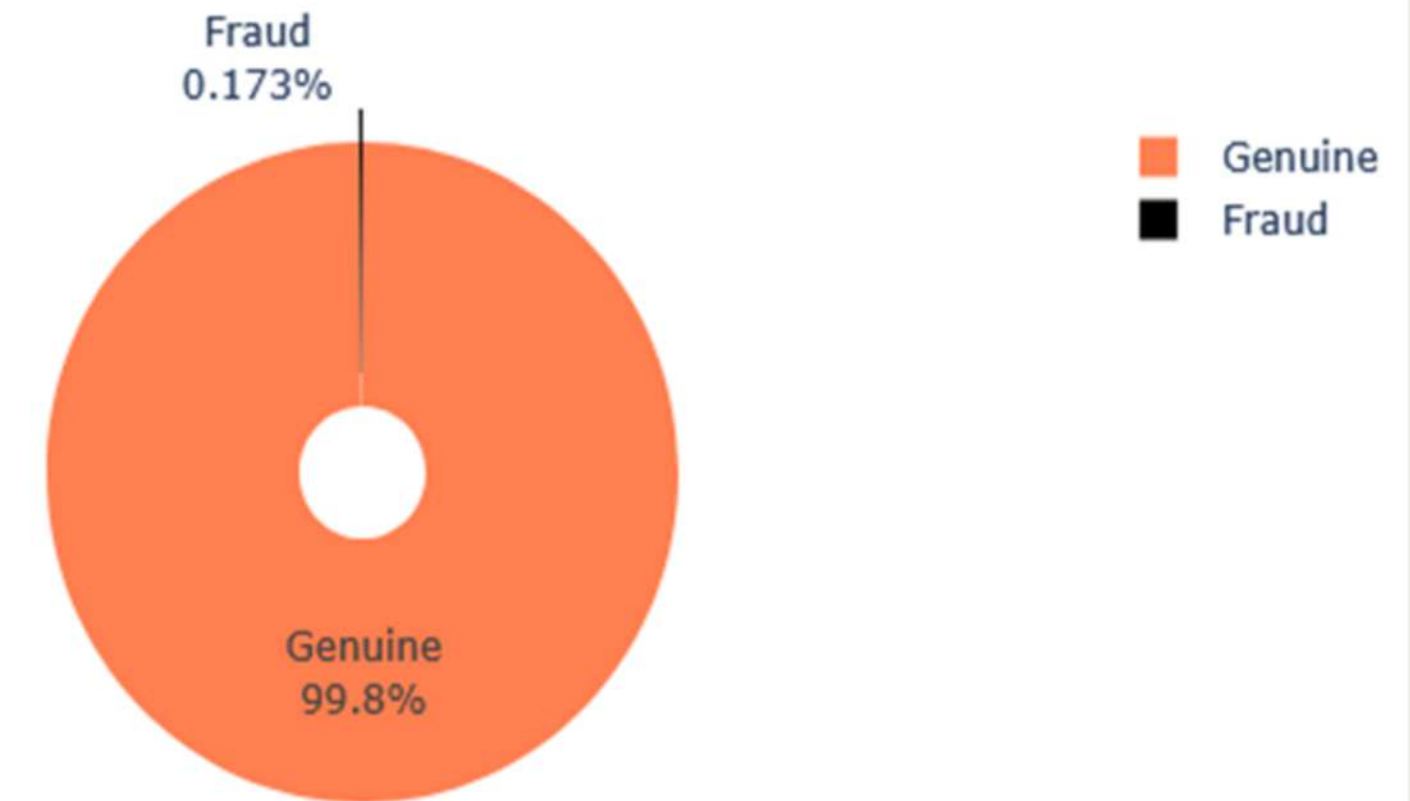
“Data is clean, but highly imbalanced”.

This imbalance must be addressed before model training using techniques like SMOTE, under sampling, or class weight adjustment..”

EXPLORE DATA

- **Objective:** Identify patterns and potential modelling challenges.
- **Target Variable (Class) Distribution:**
- **Class 0 (Non-Fraud): 99.80%.i.e. 284,315 transactions**
- **Class 1 (Fraud): 0.173% .i.e. 492 transactions**
- **Imbalance Ratio: ~0.17% fraud**

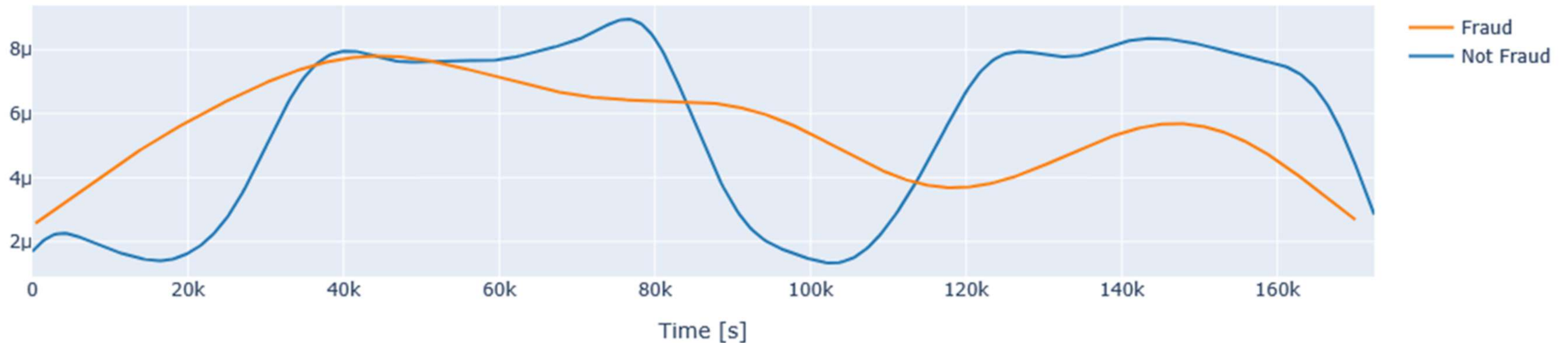
Fraud vs Genuine Transactions (3D Style Pie Chart)



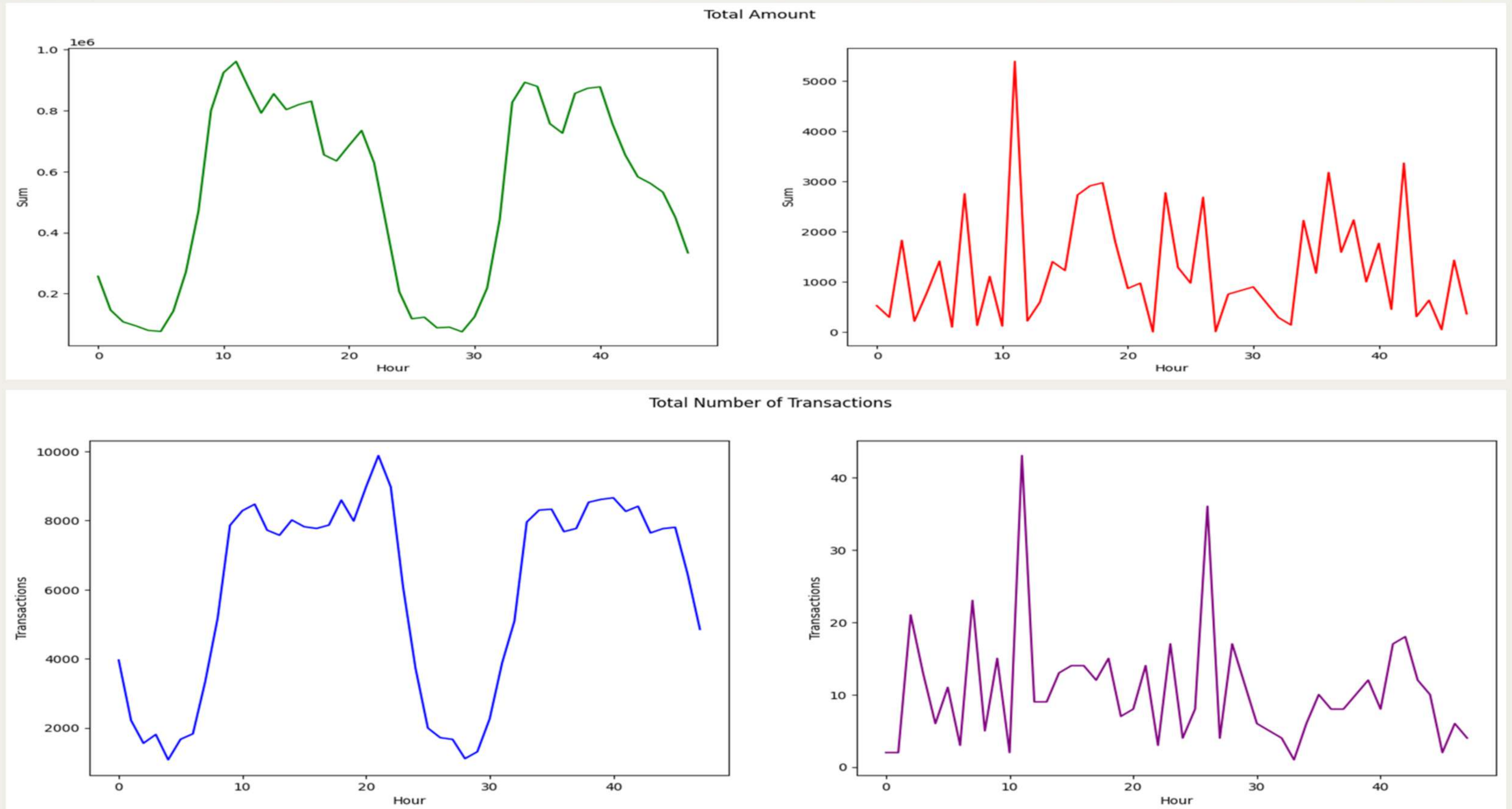
TRANSACTIONS IN TIME

- Fraudulent transactions have a distribution more even than valid transactions - are equally distributed in time.

Credit Card Transactions Time Density Plot

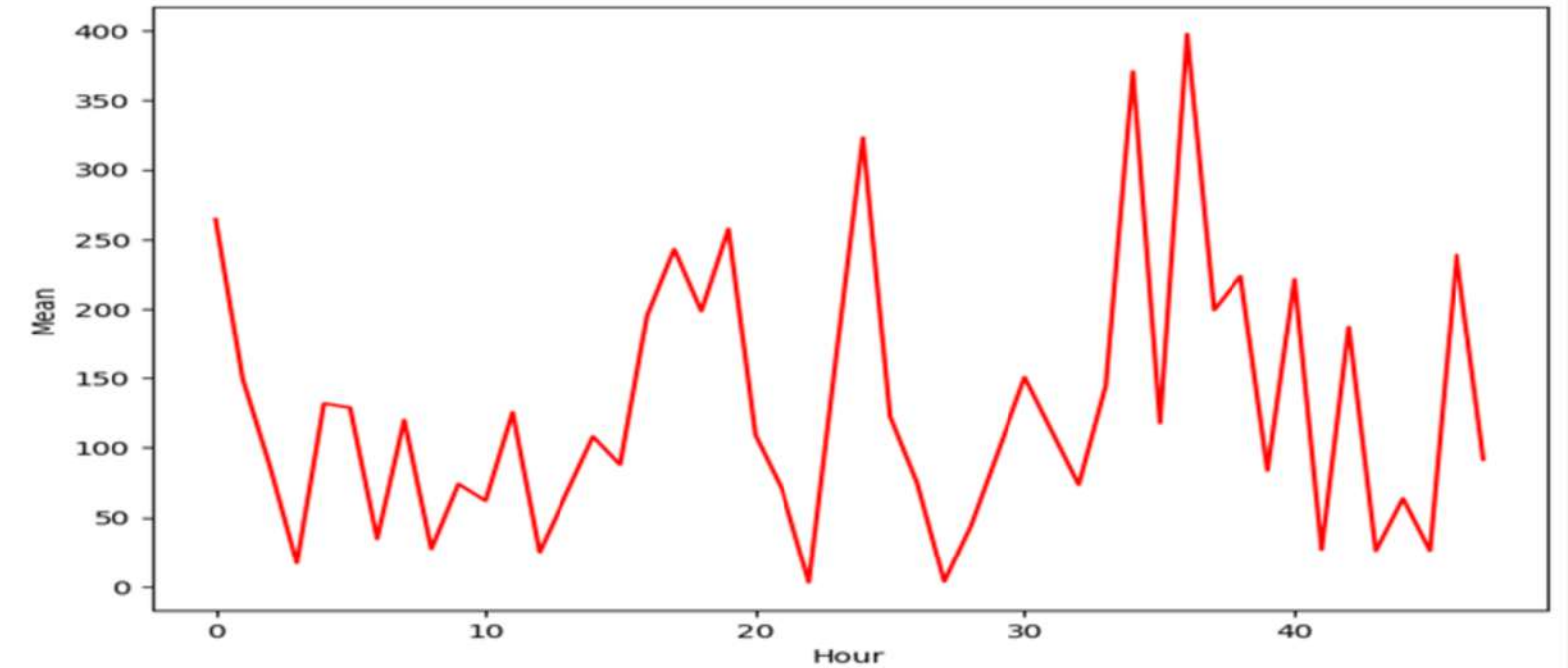
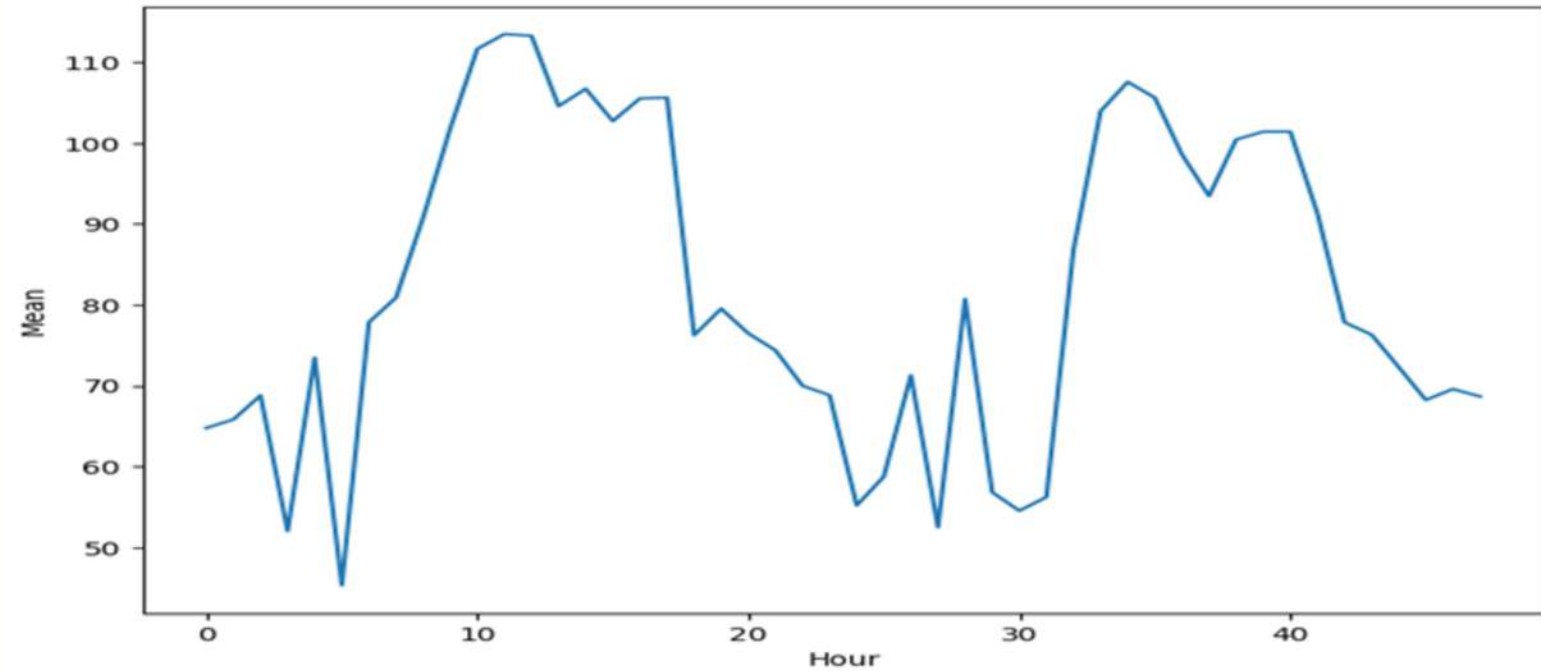


TRANSACTIONS IN TIME

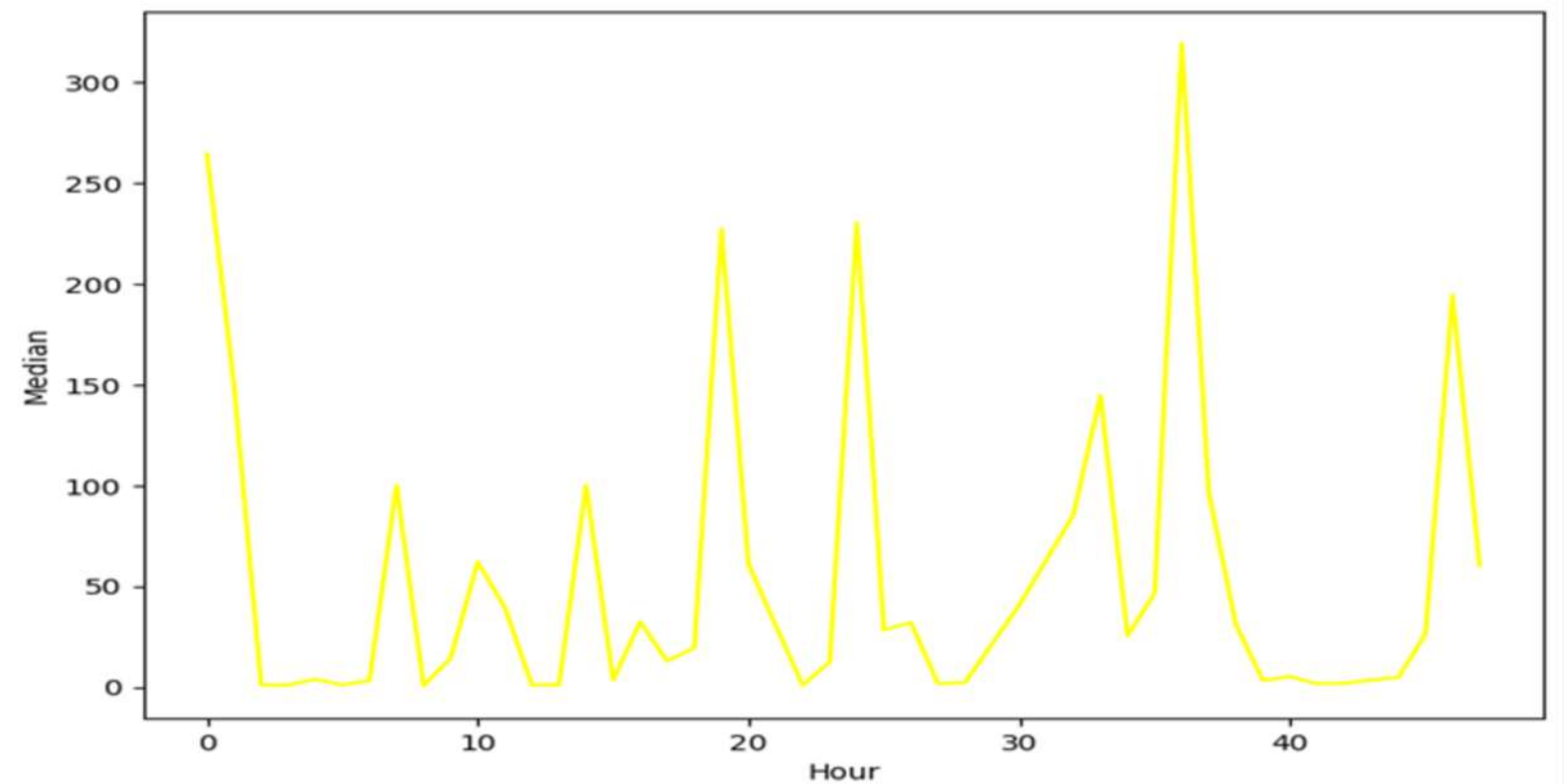
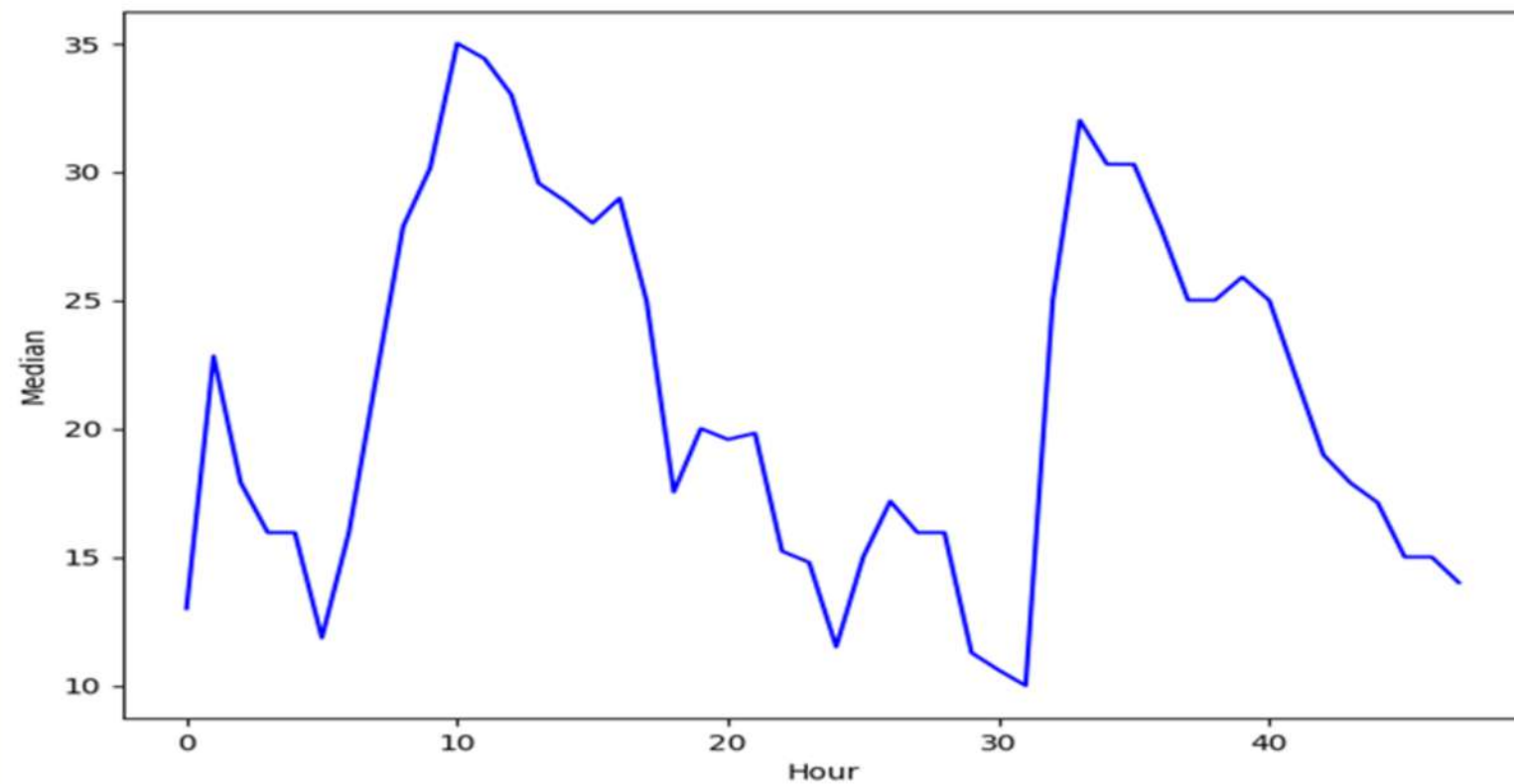


TRANSACTIONS IN TIME

Average Amount of Transactions

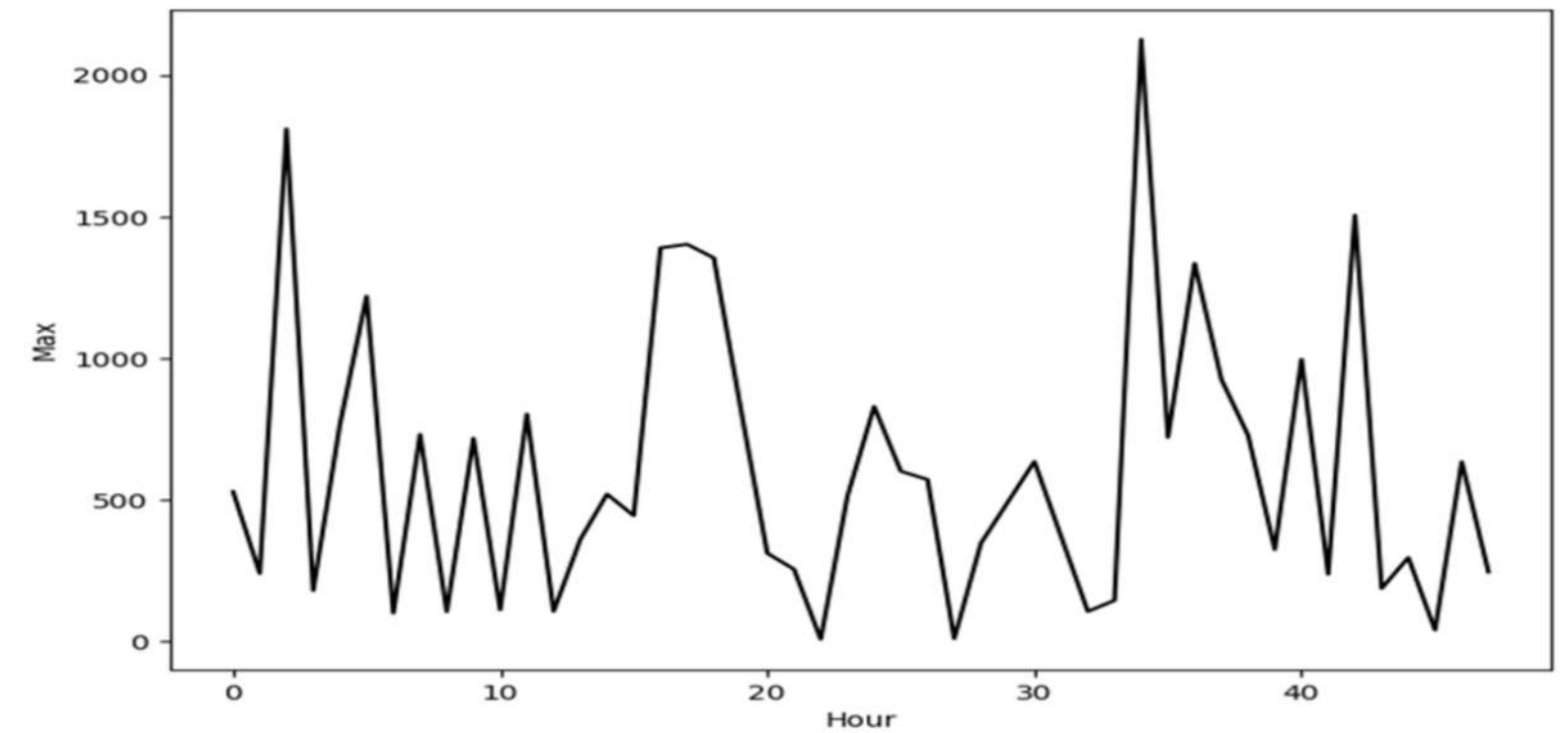
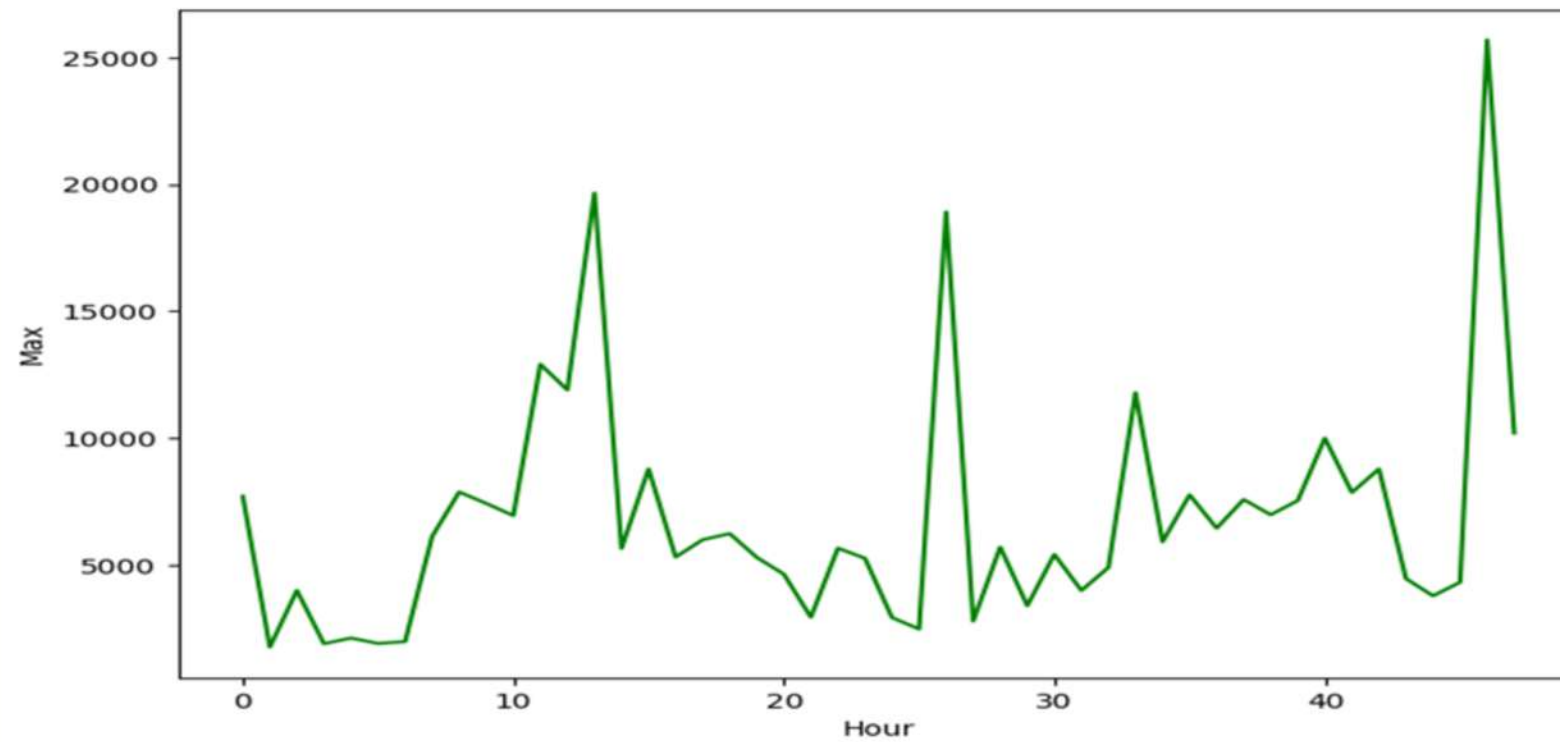


Median Amount of Transactions

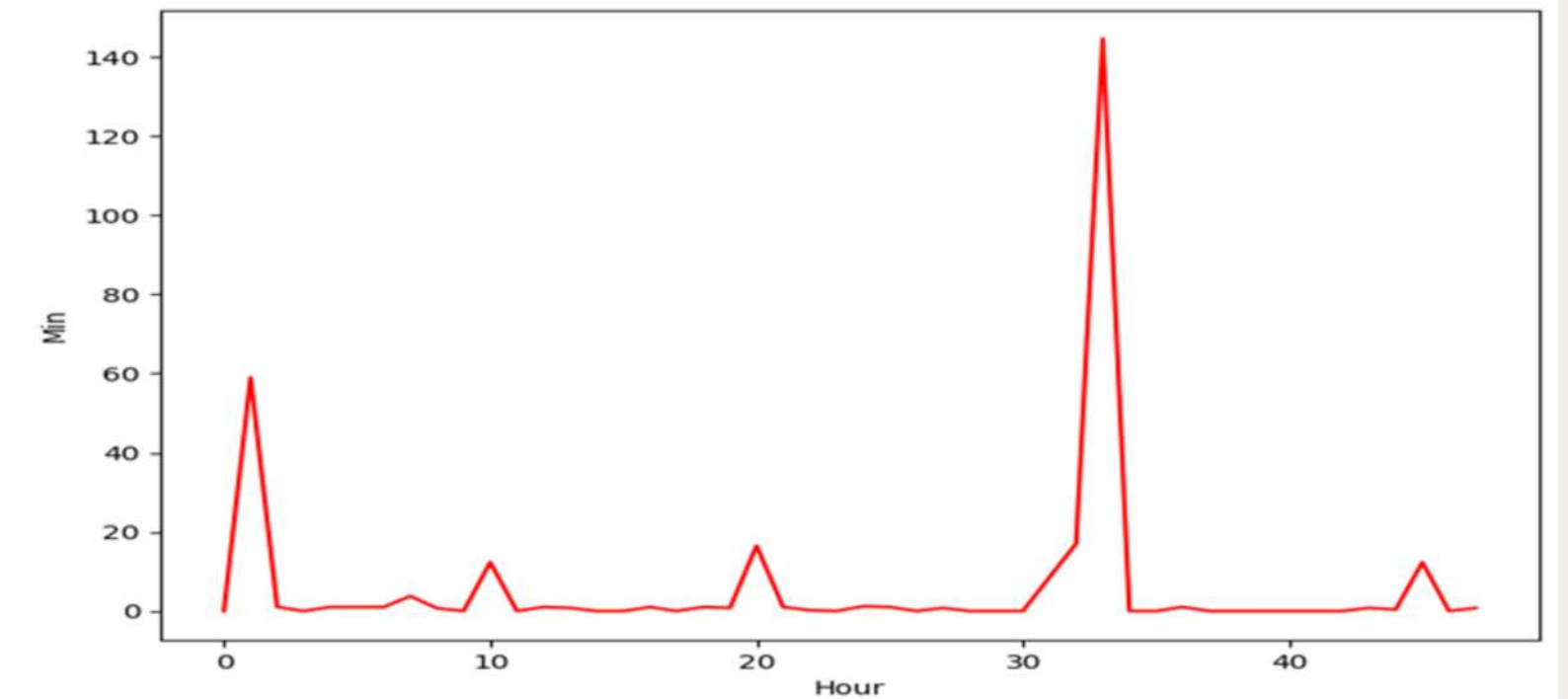
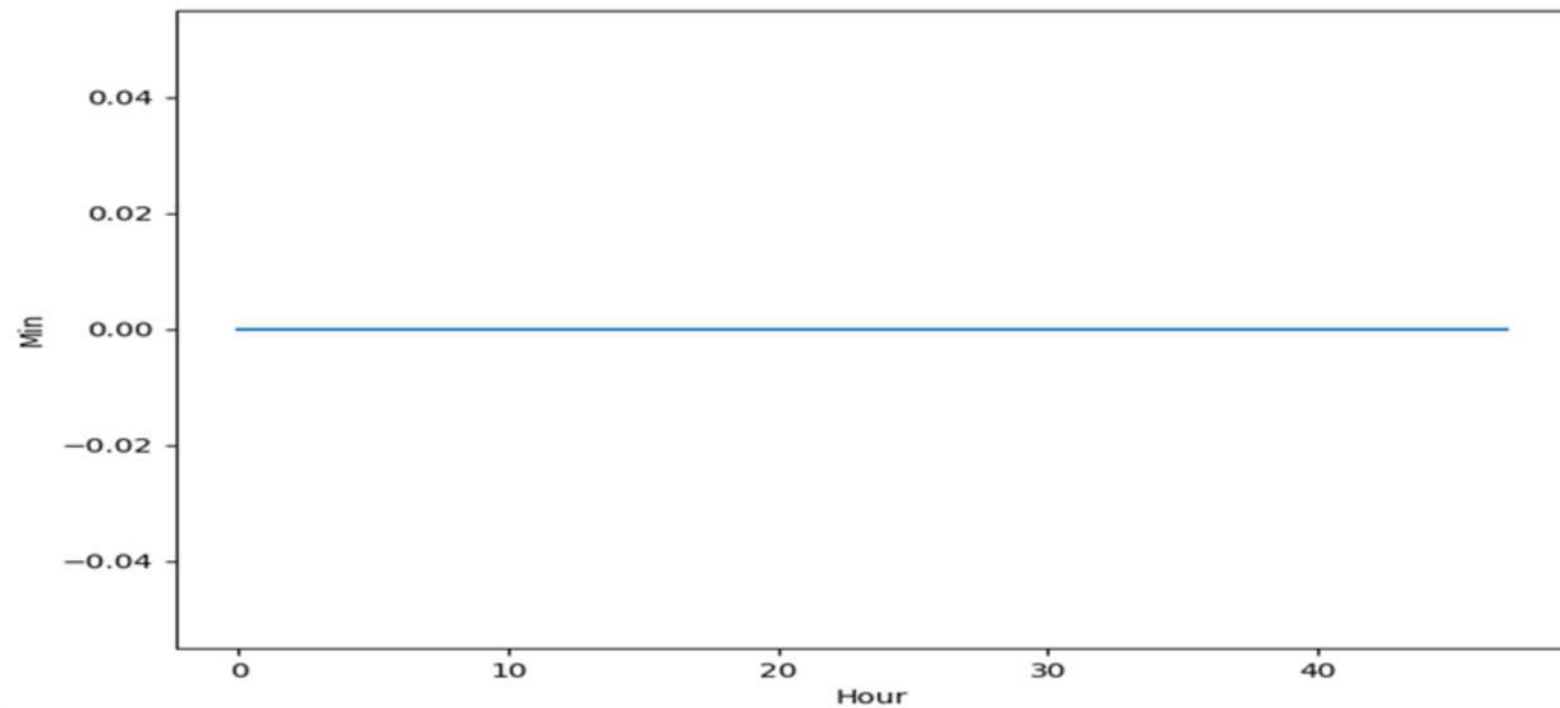


TRANSACTIONS IN TIME

Maximum Amount of Transactions

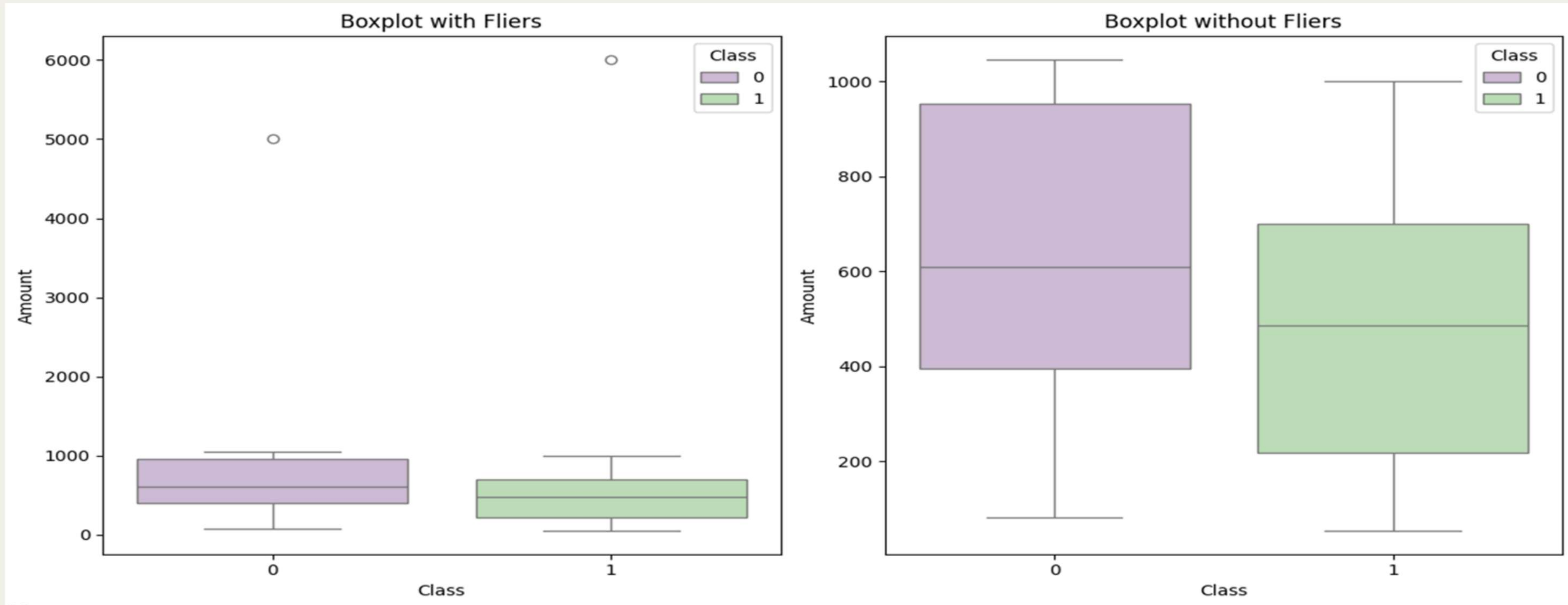


Minimum Amount of Transactions

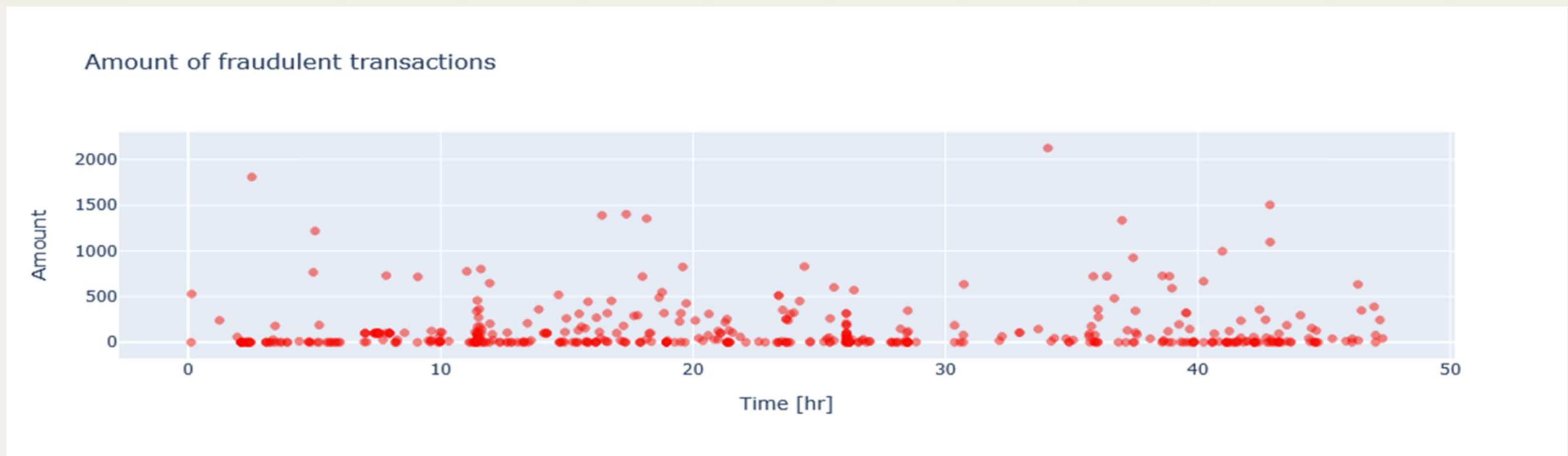


TRANSACTIONS IN AMOUNT

- Visualizing the distribution of 'Amount' for two different 'Class' categories (0 and 1).



TRANSACTIONS IN AMOUNT



```
class_0.describe()
```

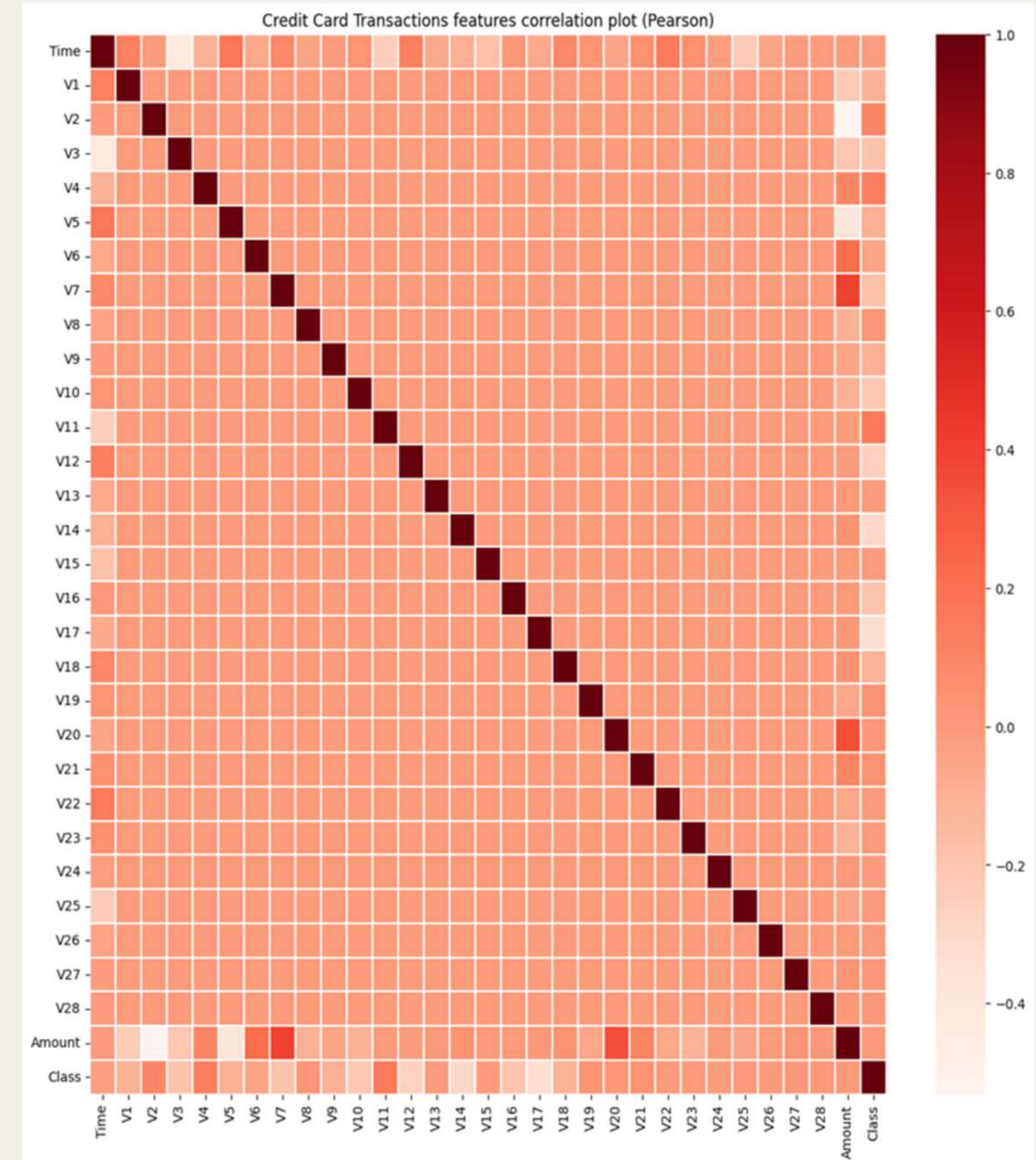
```
count      51.000000
mean       730.852548
std        686.390835
min         81.770176
25%        396.121975
50%        610.363993
75%        953.657020
max       5000.000000
Name: Amount, dtype: float64
```

```
class_1.describe()
```

```
count      49.000000
mean       603.304045
std        838.714685
min         54.398560
25%        218.001690
50%        486.004090
75%        700.701682
max       6000.000000
Name: Amount, dtype: float64
```

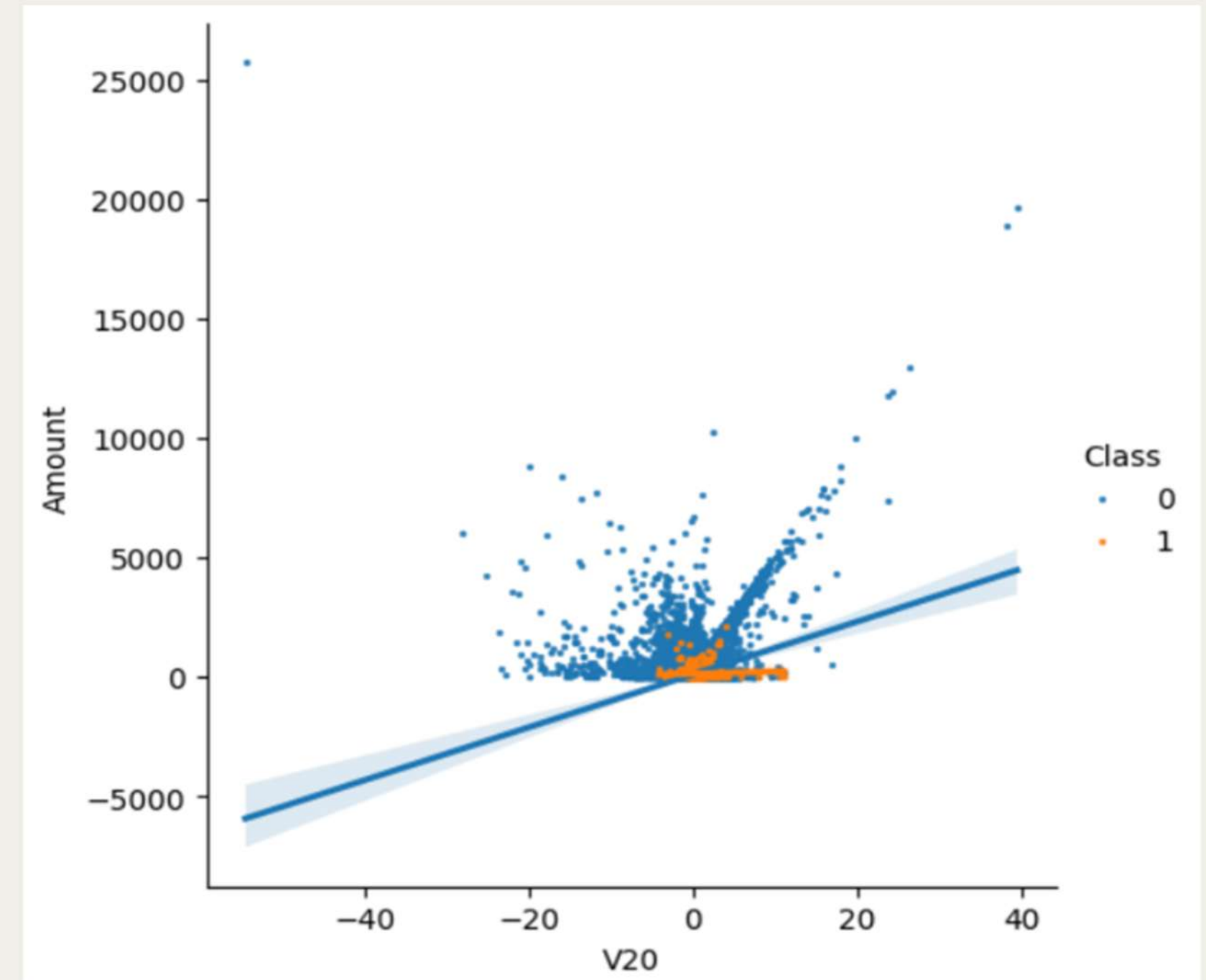
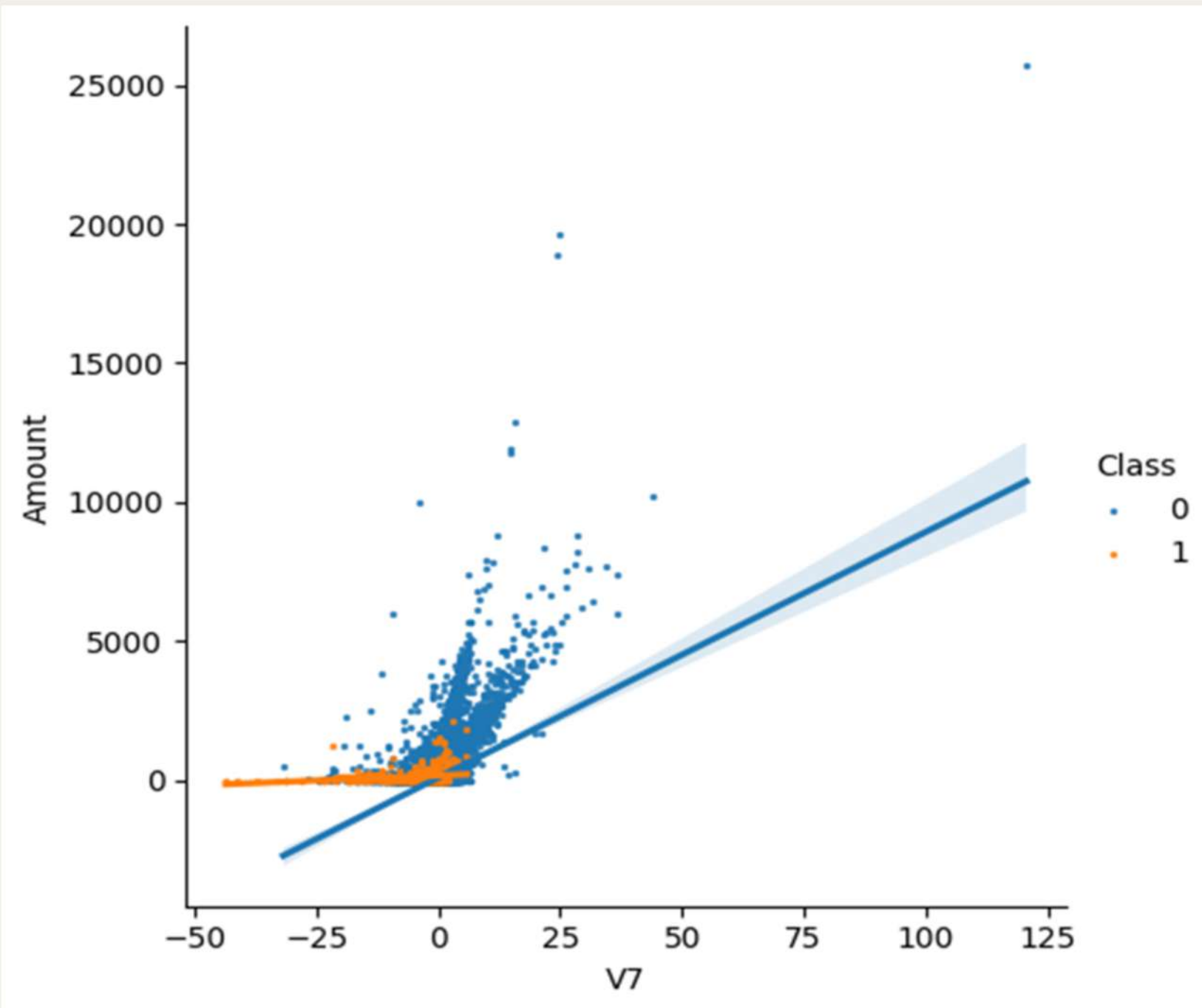

FEATURES IN CORRELATION

- There is no notable correlation between features V1-V28.
- There are certain correlations between some of these features and Time (inverse correlation with V3)
- There are certain correlations between some of these features and Amount (direct correlation with V7 and V20, inverse correlation with V1 and V5).



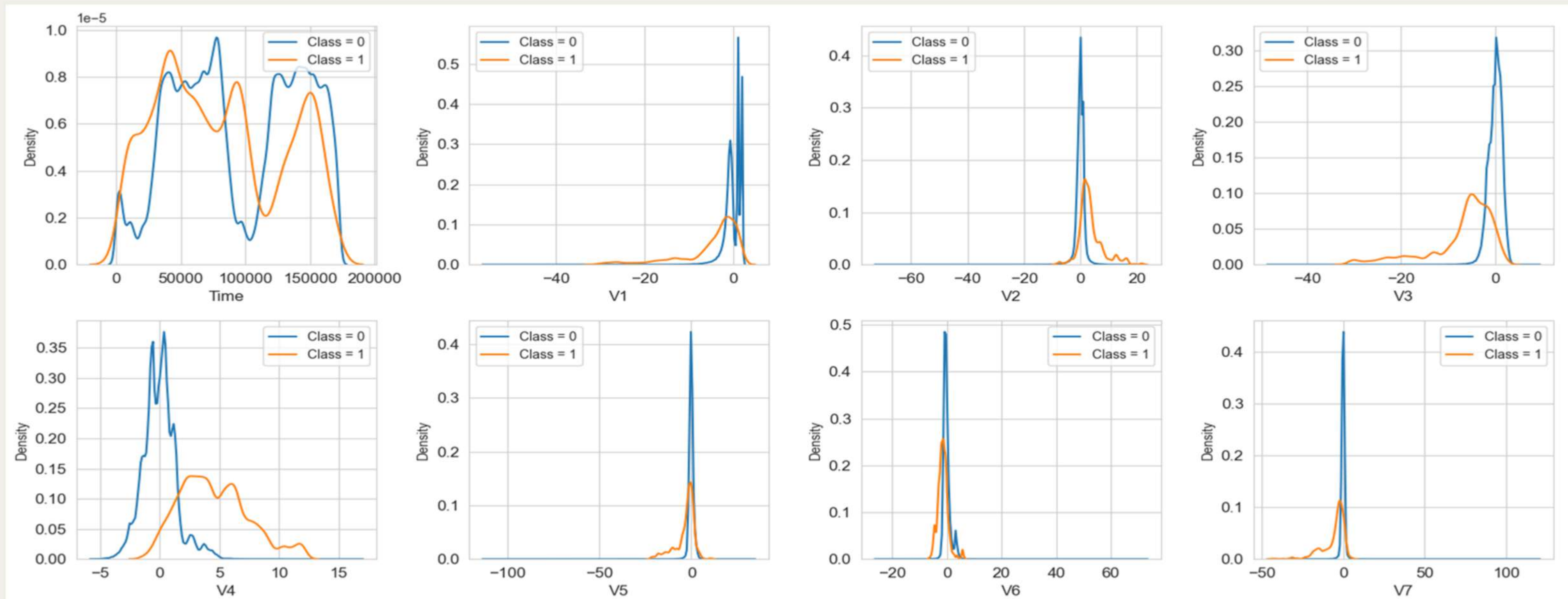
SCATTER PLOT WITH A REGRESSION LINE

- The two couples of features are correlated (the regression lines for Class = 0 have a positive slope, whilst the regression line for Class = 1 have a smaller positive slope).

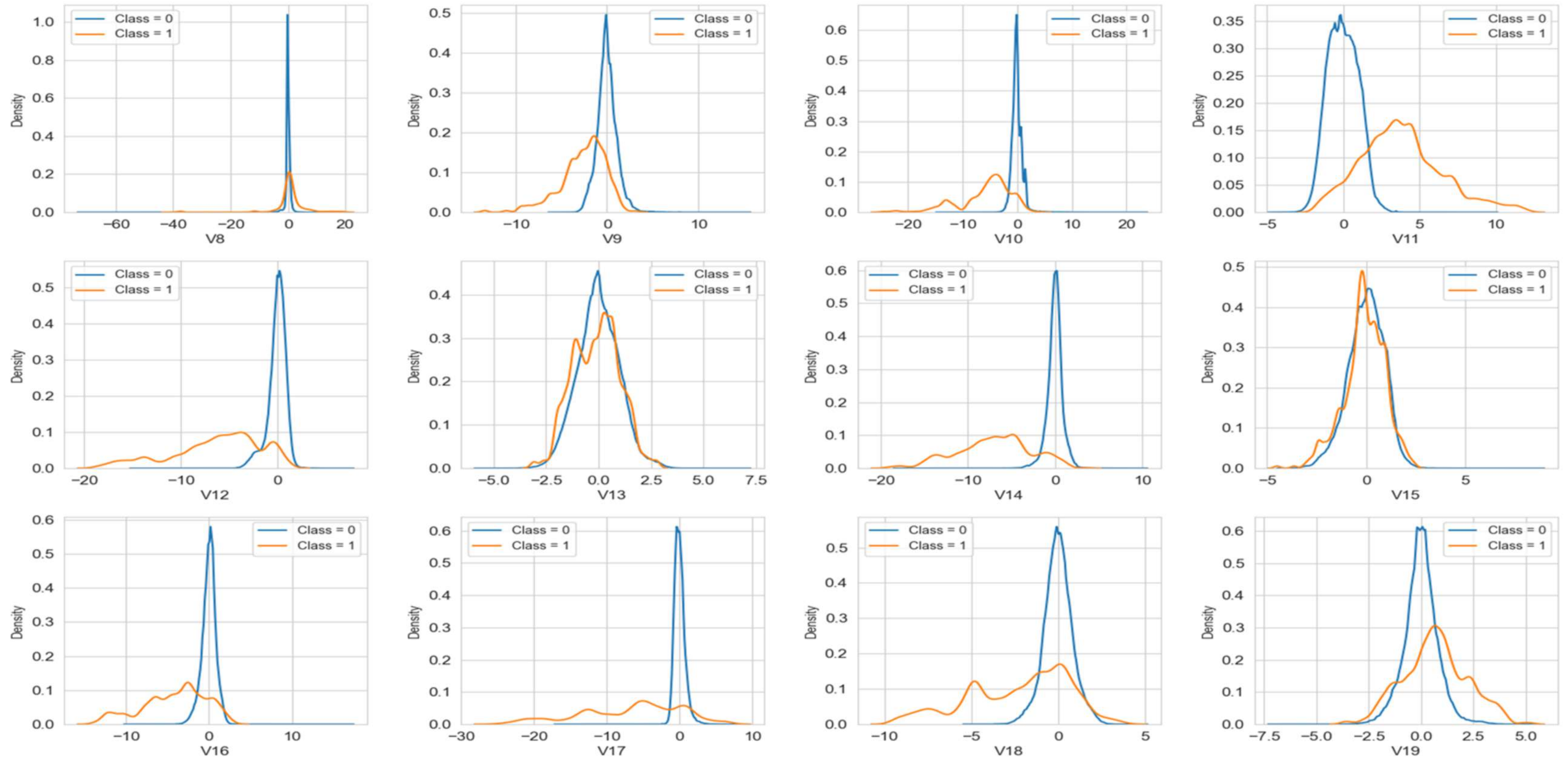


FEATURES DENSITY PLOT

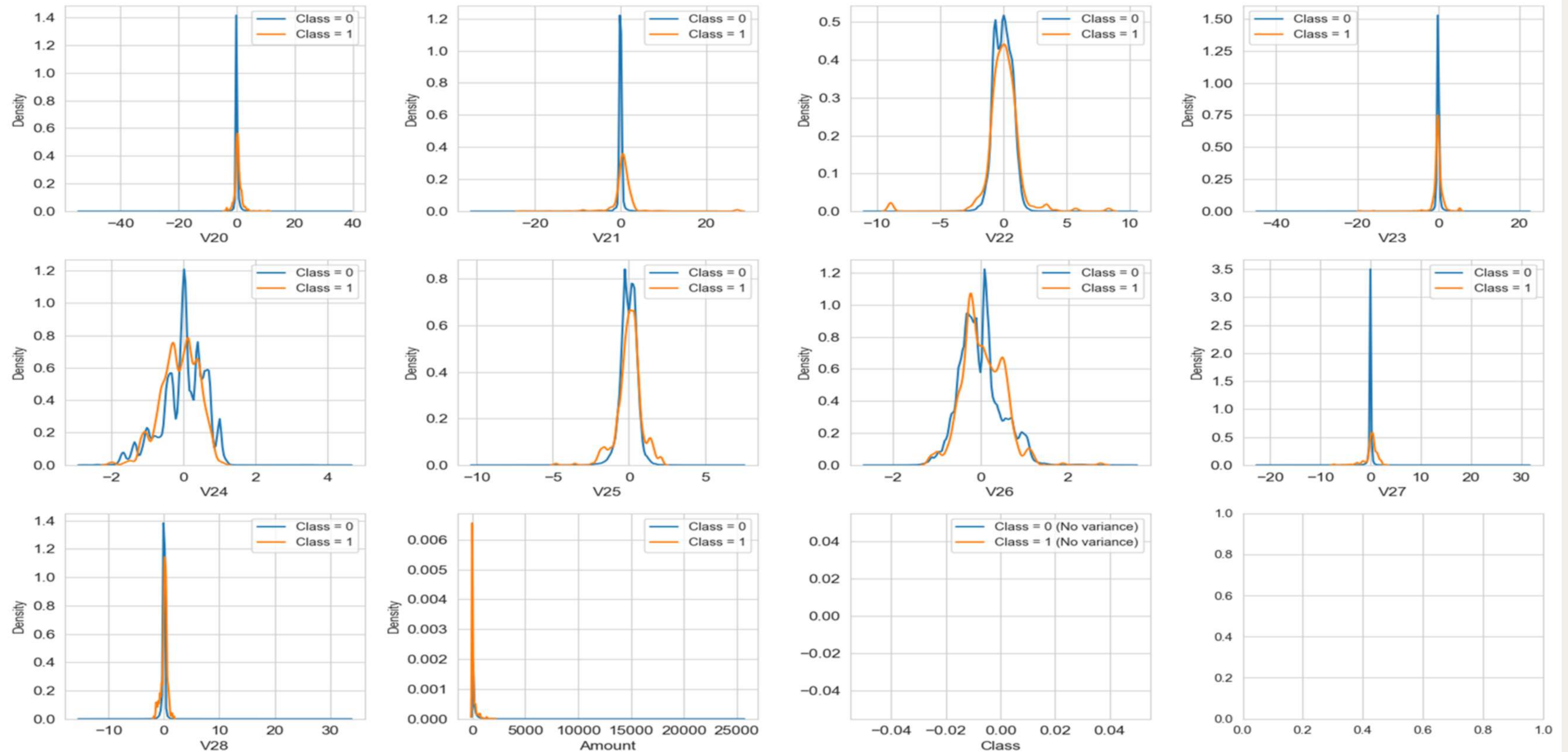
• Good selectivity in terms of distribution for the two values of Class: V4, V11 have clearly separated distributions for Class values 0 and 1, V12, V14, V18 are partially separated, V1, V2, V3, V10 have a quite distinct profile, whilst V25, V26, V28 have similar profiles for the two values of Class.



FEATURES DENSITY PLOT

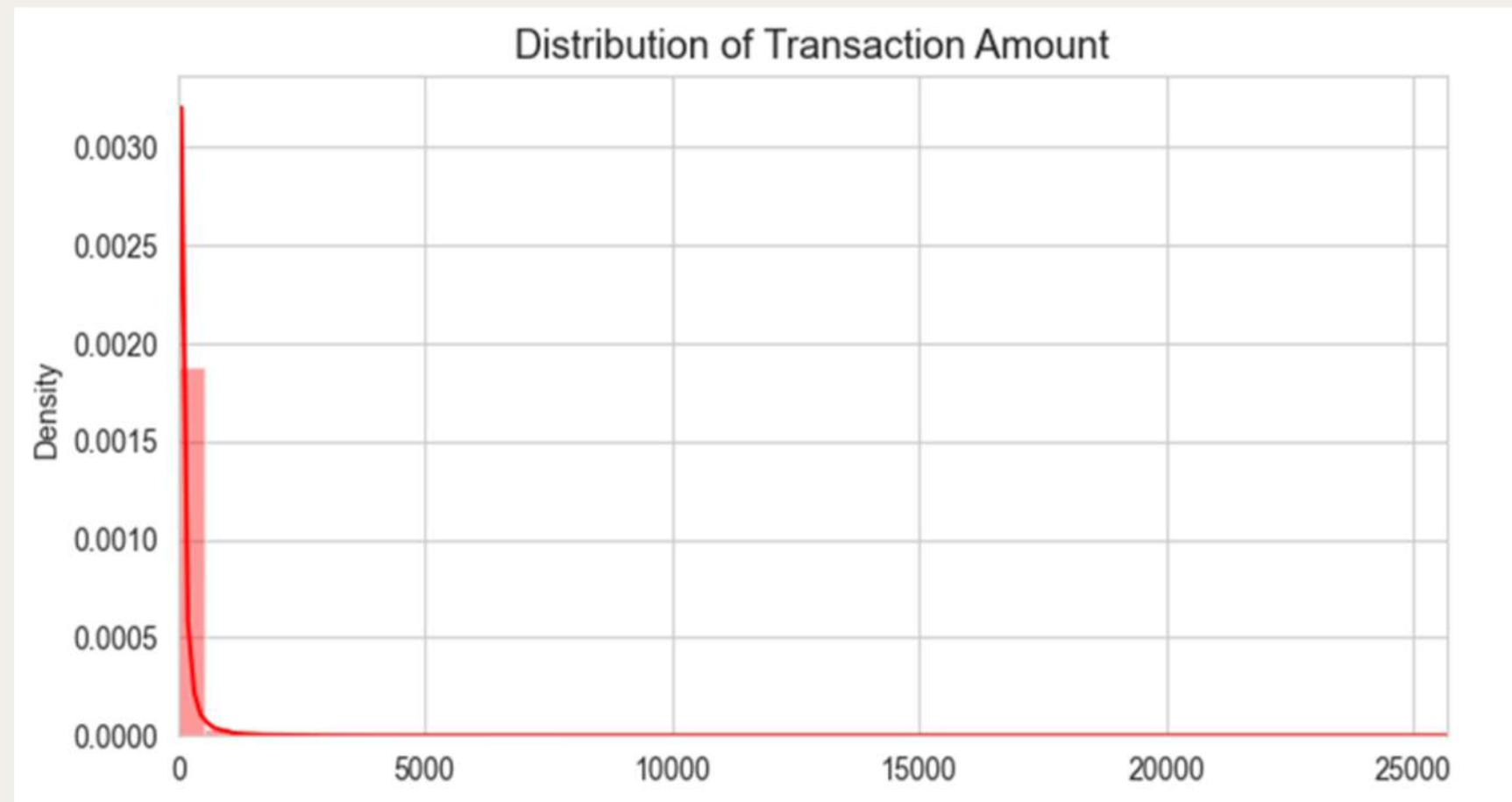


FEATURES DENSITY PLOT

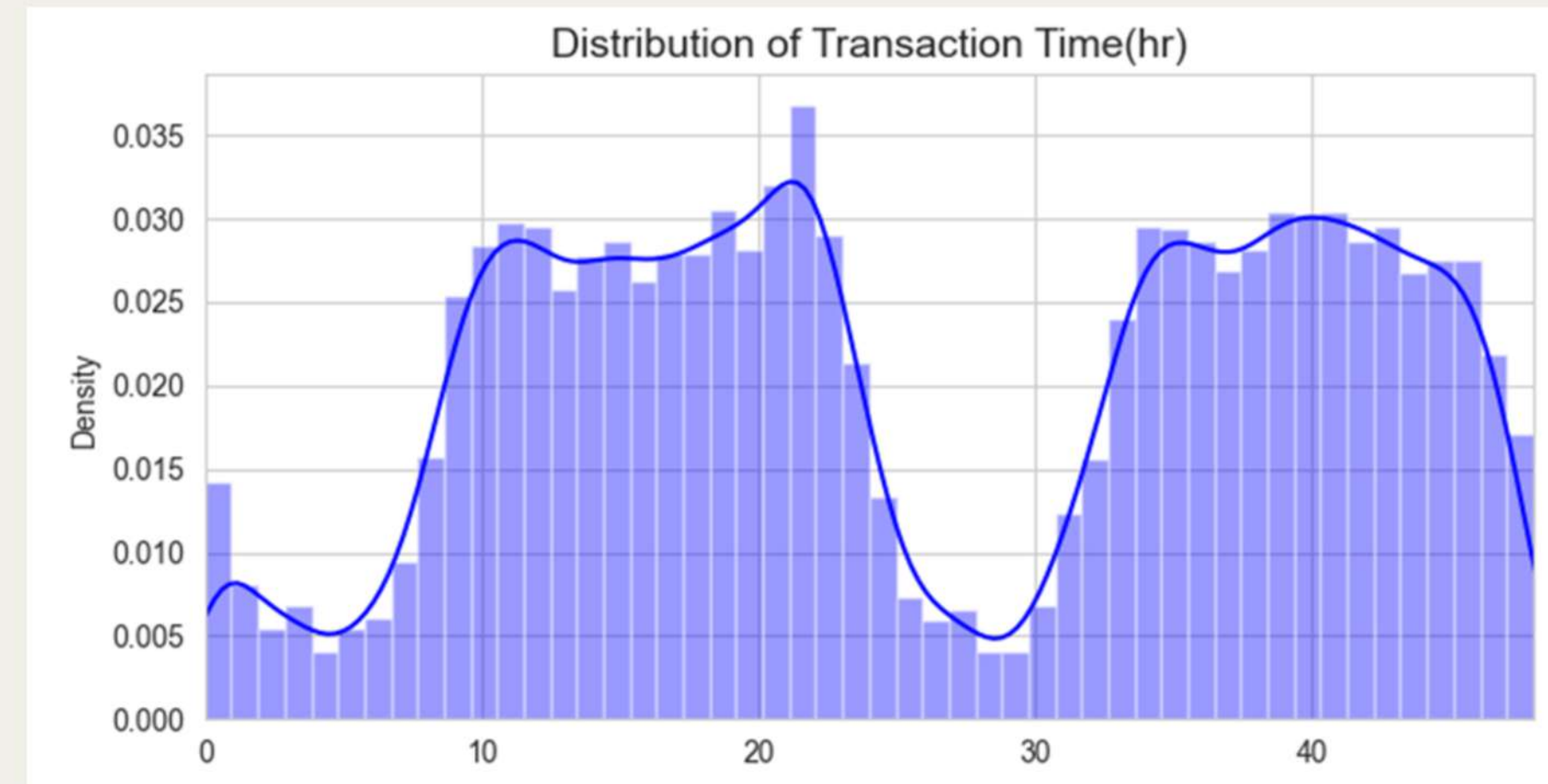


DISTRIBUTION OF TRANSACTIONS OF AMOUNT AND TIME

This graph explains that credit card transaction amounts are heavily skewed towards smaller values, with large transactions being very infrequent.



This graph explains that credit card transaction activity follows a strong diurnal pattern, with peak volumes occurring during the day and evening and lowest volumes in the early morning hours, consistently over multiple days.



JUSTIFICATION OF ML MODULE SELECTION

Metric / ML Model	XGBoost Classifier	Random Forest	Logistic Regression
F2-Score (Prioritizes Recall)	High (Excellent balance for fraud detection)	High (Very good balance)	Moderate (Can sacrifice precision/recall)
Number of False Negatives (FN)	Low (Minimizes missed fraud incidents)	Low (Good at minimizing missed fraud)	Moderate to High (Can miss more fraud)
Customer Friction (False Declines)	Low to Moderate (Tunable to reduce)	Low to Moderate (Tunable to reduce)	Moderate to High (Higher risk of legitimate declines)
Real-time Prediction Latency	Very Low (Fast inference for single predictions)	Low (Fast inference for single predictions)	Extremely Low (Near instant)
Ability to Learn Complex Patterns	Excellent (Captures non-linearities and interactions)	Excellent (Captures non-linearities and interactions)	Limited (Primarily linear relationships)
Interpretability/Explainability	Moderate (Feature Importance, SHAP/LIME for specific cases)	Moderate (Feature Importance, SHAP/LIME)	High (Direct coefficient and probability interpretation)
Ease of Model Update/Retraining	Moderate (Requires re-training on new data)	Moderate (Requires re-training on new data)	High (Relatively fast to retrain)
Robustness to Outliers	Good (Less sensitive than linear models)	Excellent (Ensemble averaging reduces outlier impact)	Moderate (Sensitive to extreme values)
Deployment Complexity	Moderate (More dependencies, larger model size)	Moderate (Can have large model files)	Low (Simple, compact model)

FURTHER STEPS

Work Stream	Deliverables	Status
Business Understanding	Should research on Business Use-Case	Done
	Should: Identifying Stakeholders	Done
	Should: Generate Project Plan	Done
	Should: Process Workflow and resources allocation	Done
Exploratory Data Analysis	Shall: Selection of Data, Loading of Dataset, Loading libraries	Done
	Shall: Understanding dataset by Exploring, creating multiple plots	Done
	Should: Cleaning, Standardizing of Column values, Filtering and dropping unwanted columns based on data understanding	Done
ML Model	Should: Justification of ML model selection	Done
	Shall: Create three ML model on the processed and cleaned data.	Ongoing
	Should: Performance Analysis of initial ML Model	Ongoing
	Should: improving ML model accuracy Further improvements(Feature engineering, etc.)	Ongoing
Documentation	Should: Prepare PDF Document with all the slidesand prepare well documented Notebook for Final Submission	Ongoing

Credit Card Fraud Detection

GROUP 007

FINAL PRESENTATION

26TH JUNE 2025

Namitha Navaneetha Krishnan

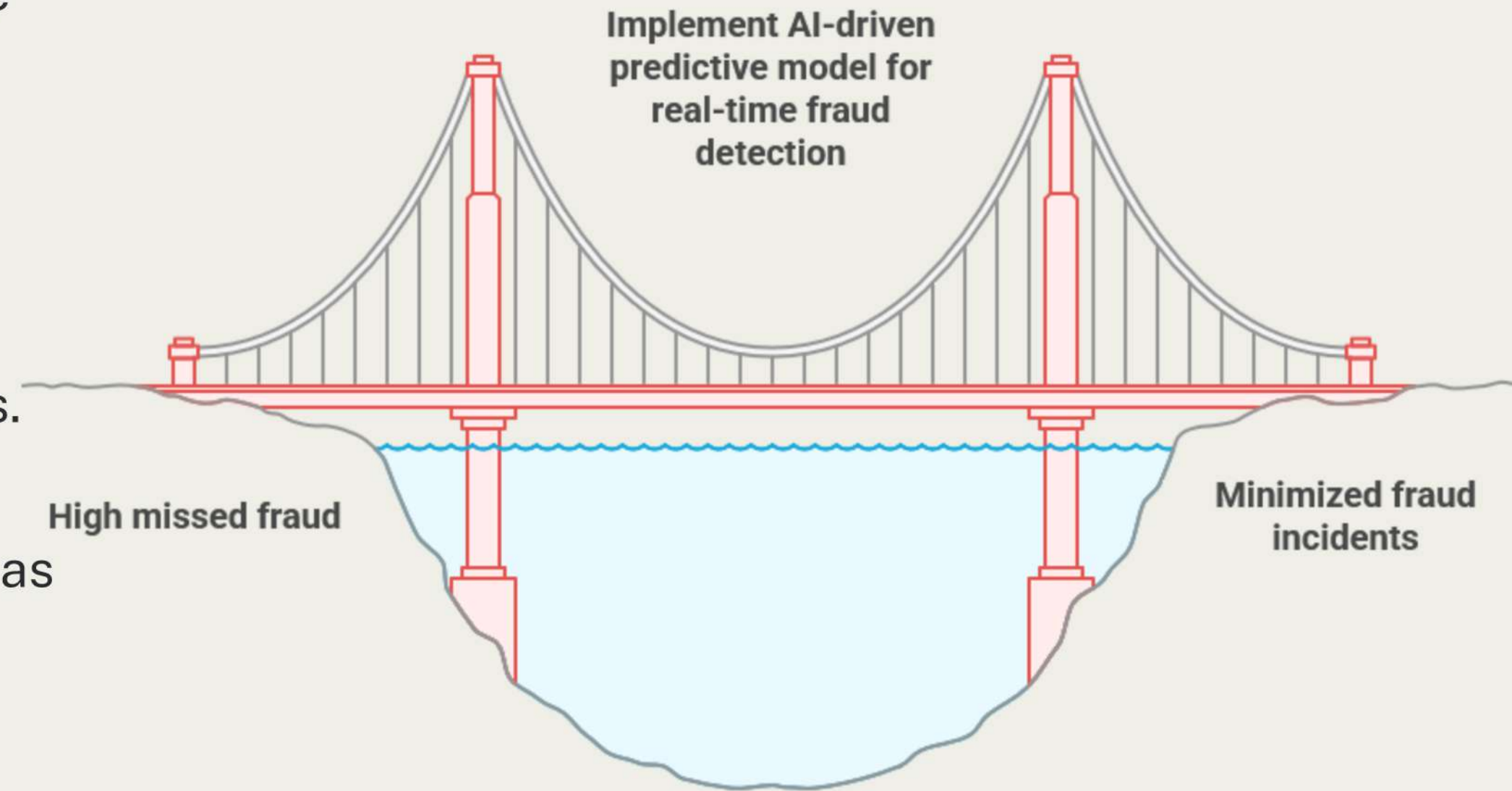
Prajnya Kalmane

Ashwin Srambikal

BUSINESS UNDERSTANDING

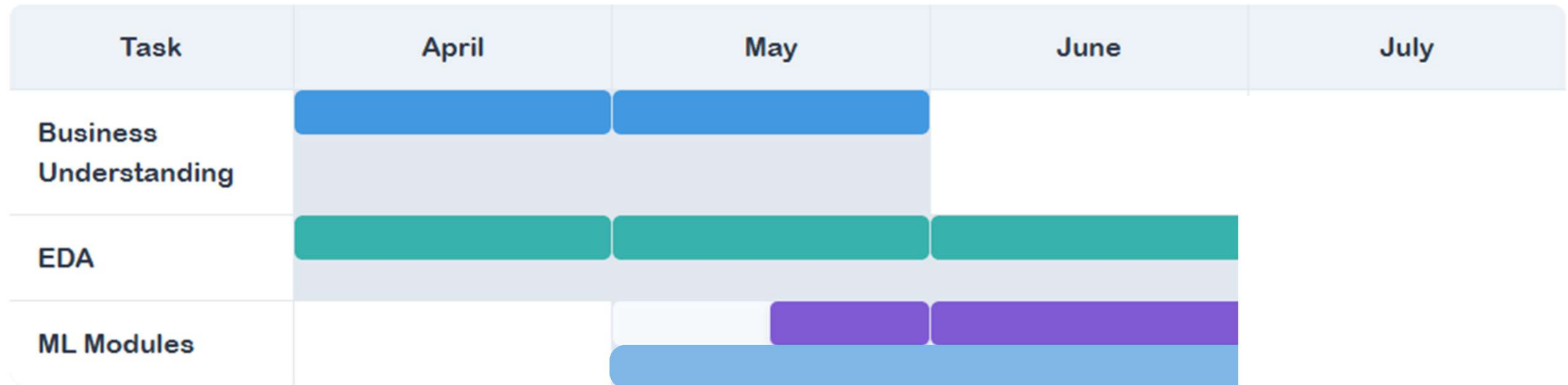
The project's main goal is to minimize these financial losses and maintain a positive customer experience by implementing an Artificial Intelligence-driven solution. This involves building a predictive model capable of:

- **High Recall:** Effectively detecting fraudulent transactions to minimize missed fraud incidents.
- **Low False Positives:** Reducing instances where legitimate transactions are incorrectly flagged as fraud, thereby avoiding inconvenience to customers.
- **Real-time Efficiency:** Ensuring the model can perform rapid predictions for seamless integration into real-time systems.



PROJECT TIMELINE

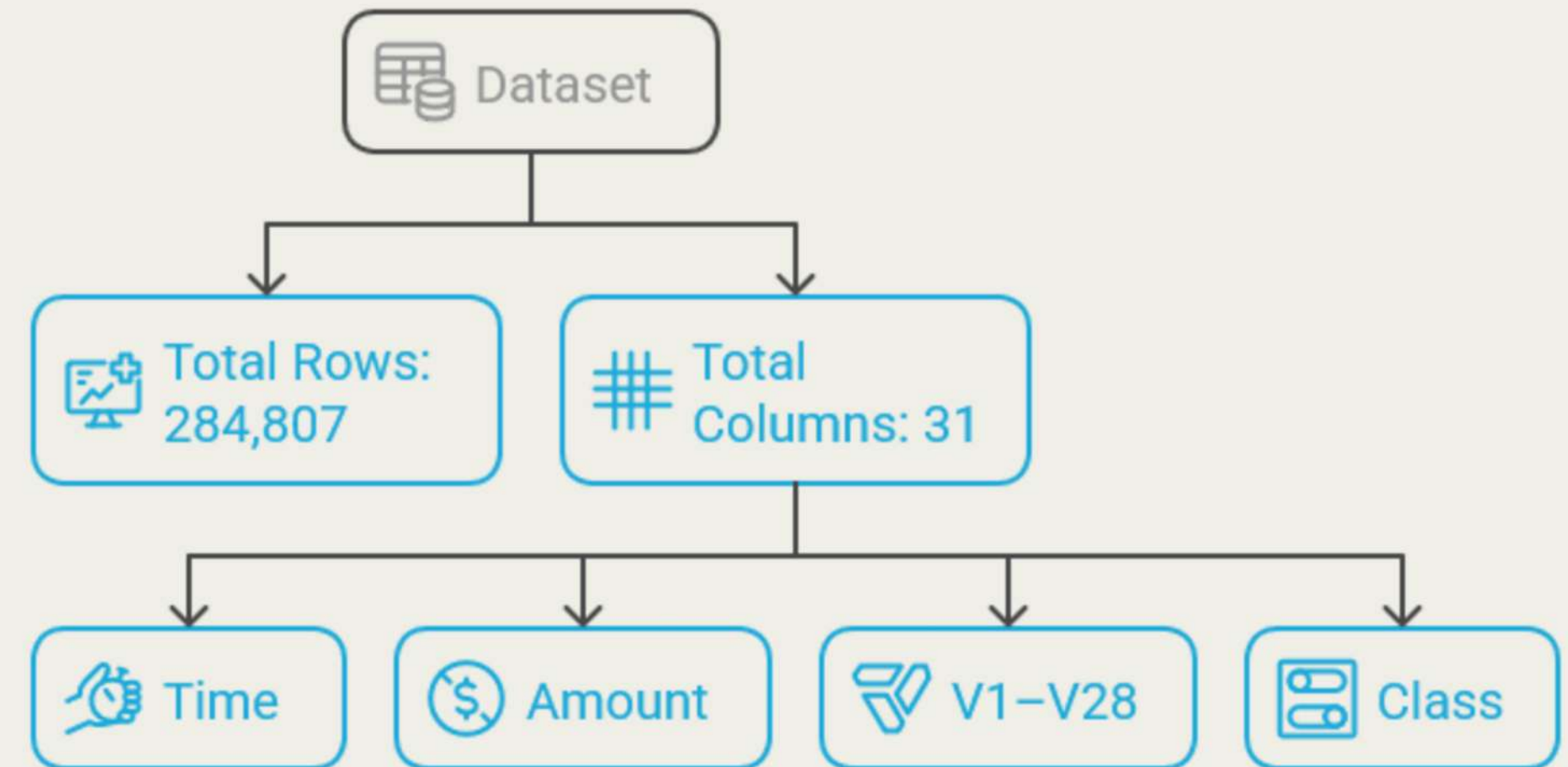
Project Timeline Gantt Chart



DATA UNDERSTANDING

Overview:

- Total Rows: 284,807
- Total Columns: 31
- Features include:
- Time, Amount: Raw numerical features
- V1–V28: PCA-transformed anonymized features
- Class: Target variable (0 = Non-Fraud, 1 = Fraud)



Source:

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

33 (Dataset from Kaggle – Credit Card Fraud Detection)

DATA UNDERSTANDING

Feature Types:

- All columns are numerical (including the target).
- No categorical or text data.

Descriptive Statistics:

Time: Elapsed time in seconds from the first transaction.

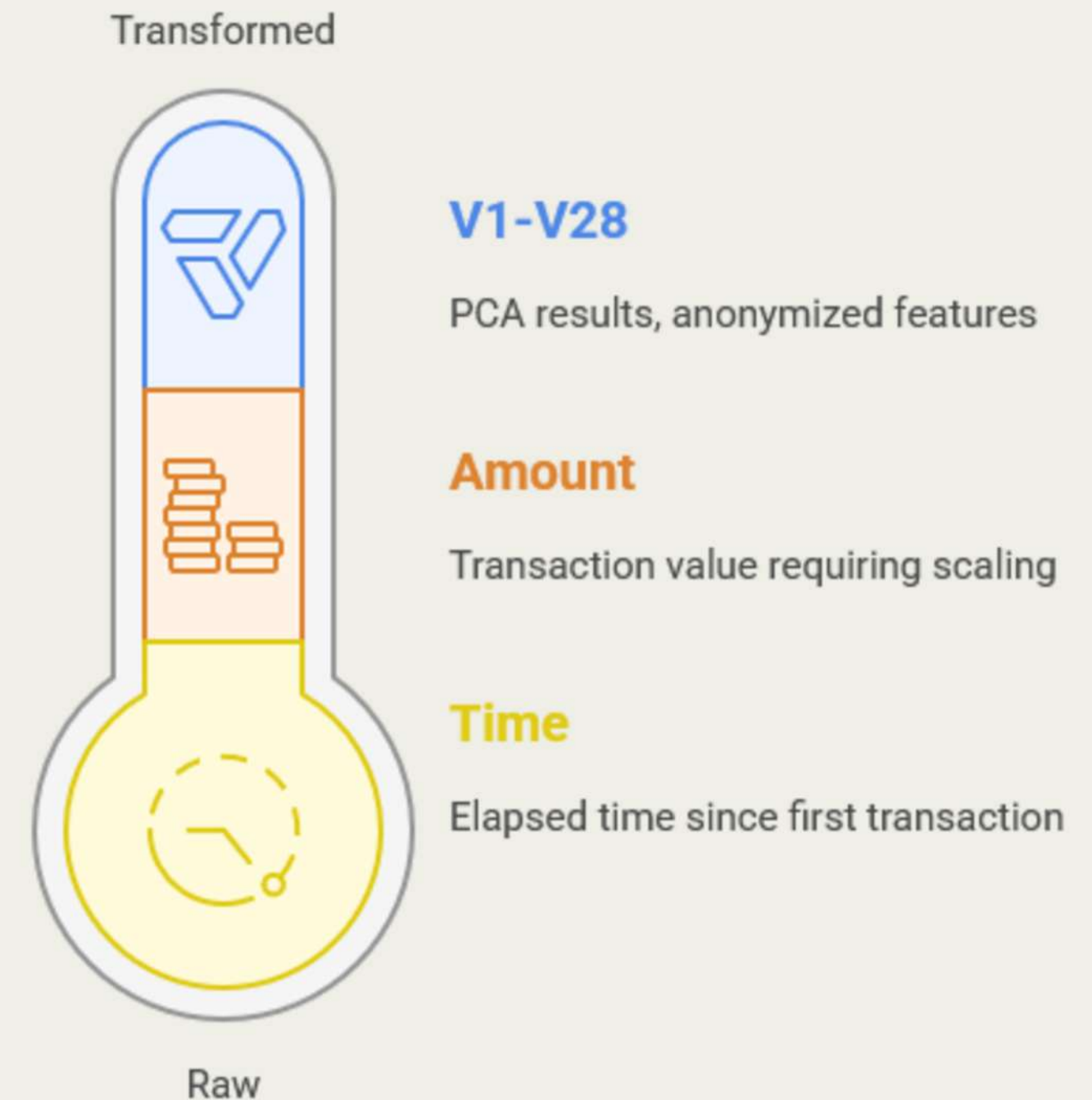
Amount: Varies from 0 to ~25,691, needs scaling.

V1-V28: Result of PCA transformation, anonymized features.

Class: Highly imbalanced binary target.

Summary Statistics:

- Columns like Amount and Time have large ranges.
- PCA features (V1-V28) show varying means and standard deviations.
- No extreme outliers at first glance.



ML MODULES

- **XGBoost Classifier**

XGBoost Classifier excels in fraud detection with high F2-Score and low false negatives, effectively minimizing missed fraud and learning complex, non-linear patterns. While offering very fast real-time predictions, its interpretability and deployment can be moderately complex.

- **Logistic Regression**

Logistic Regression is a simple, highly interpretable model, fast for real-time predictions and updates, but limited in learning complex patterns and sensitive to outliers in fraud detection.

- **Random Forest**

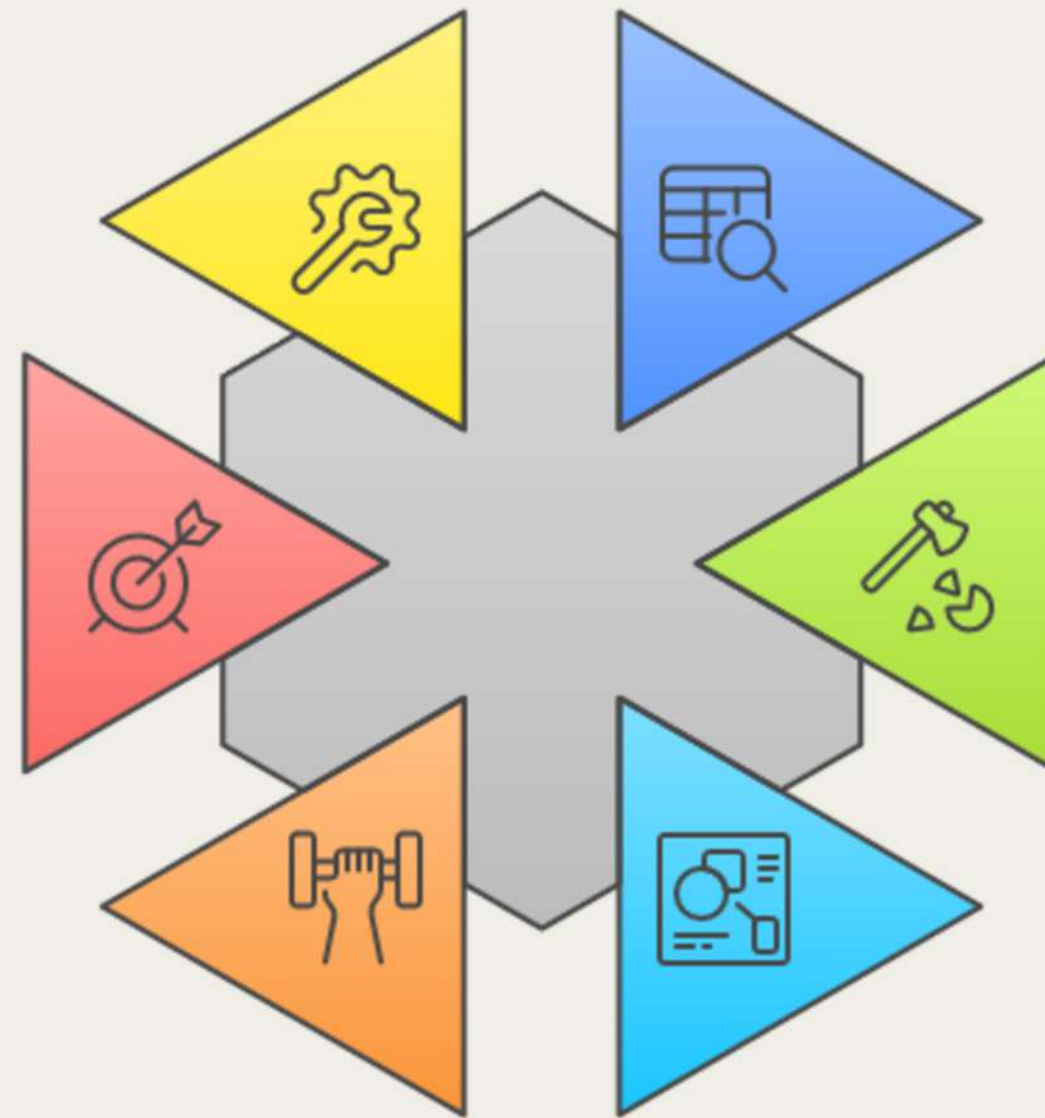
Random Forest is a highly effective model for fraud detection, offering excellent F2-scores, minimal false negatives, strong outlier robustness, and the ability to learn complex patterns, with fast real-time predictions.

Which model to use for fraud detection?



ML MODULES: XGBOOST CLASSIFIER

XGBoost (eXtreme Gradient Boosting) is an optimized distributed gradient boosting library designed to be highly efficient, flexible, and portable. It implements machine learning algorithms under the Gradient Boosting framework. When used for classification tasks (like fraud detection), it's specifically referred to as an XGBoost Classifier



- ▶ **Data Preparation**
Loading and preprocessing data for analysis
- ▶ **Data Splitting**
Dividing data into training and testing sets
- ▶ **Model Definition**
Initializing the XGBoost model object
- ▶ **Model Training**
Fitting the model to the training data
- ▶ **Prediction & Evaluation**
Making predictions and evaluating performance
- ▶ **Hyperparameter Tuning**
Optimizing model performance through parameter adjustment

ML MODULES: XGBOOST CLASSIFIER

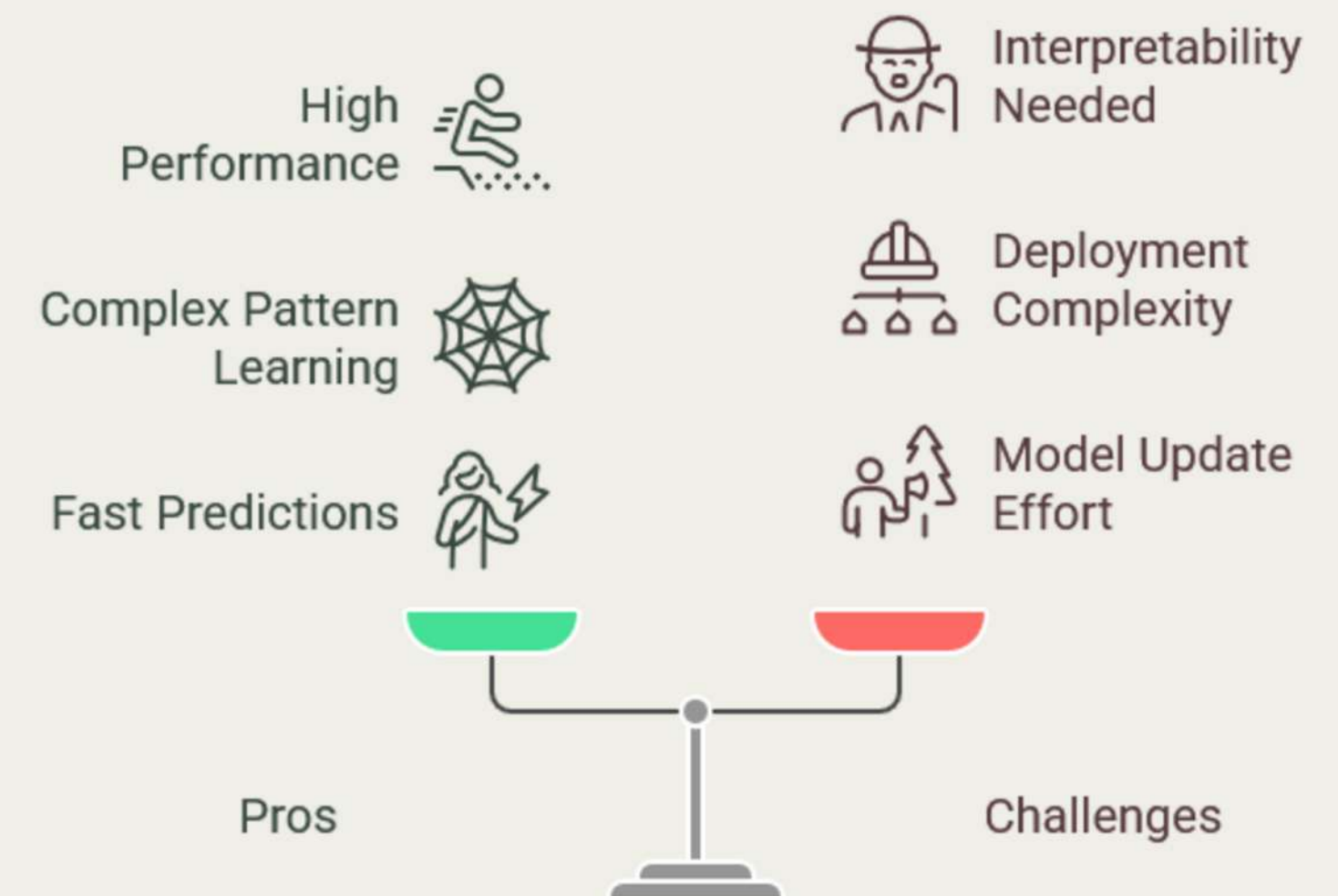
Pros for the Credit Card Fraud Detection Project:

- **High Performance:** Offers excellent F2-Scores and very low false negatives, crucial for catching fraud while balancing precision.
- **Complex Pattern Learning:** Highly effective at identifying intricate, non-linear patterns in transactional data, which is common in sophisticated fraud.
- **Fast Predictions:** Provides very low real-time prediction latency, making it suitable for immediate fraud flagging.
- **Robustness:** Good at handling outliers often present in real-world financial data.

Challenges for the Project:

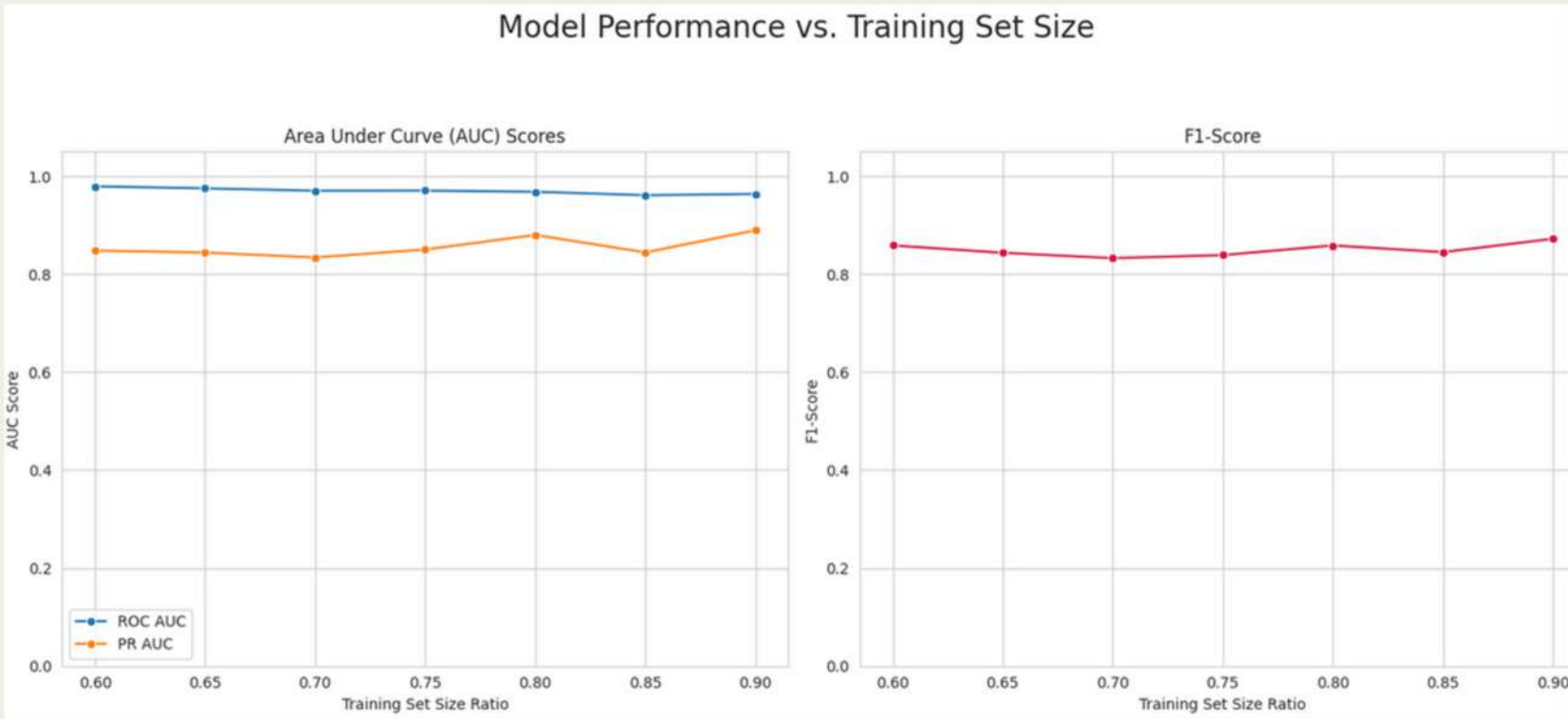
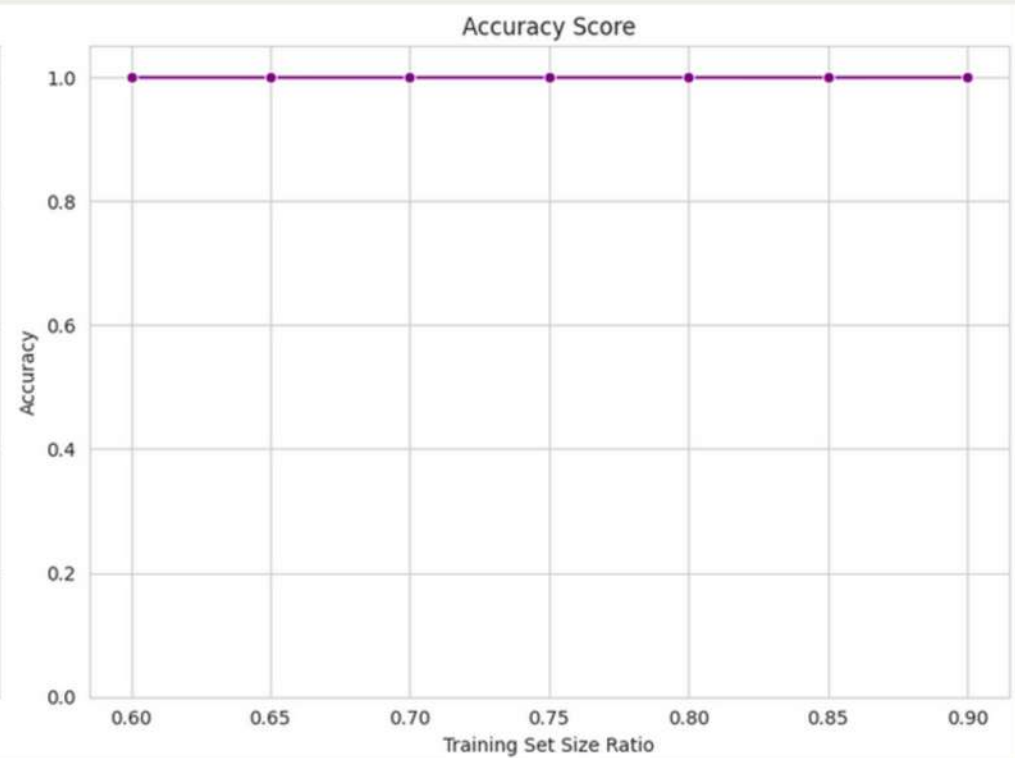
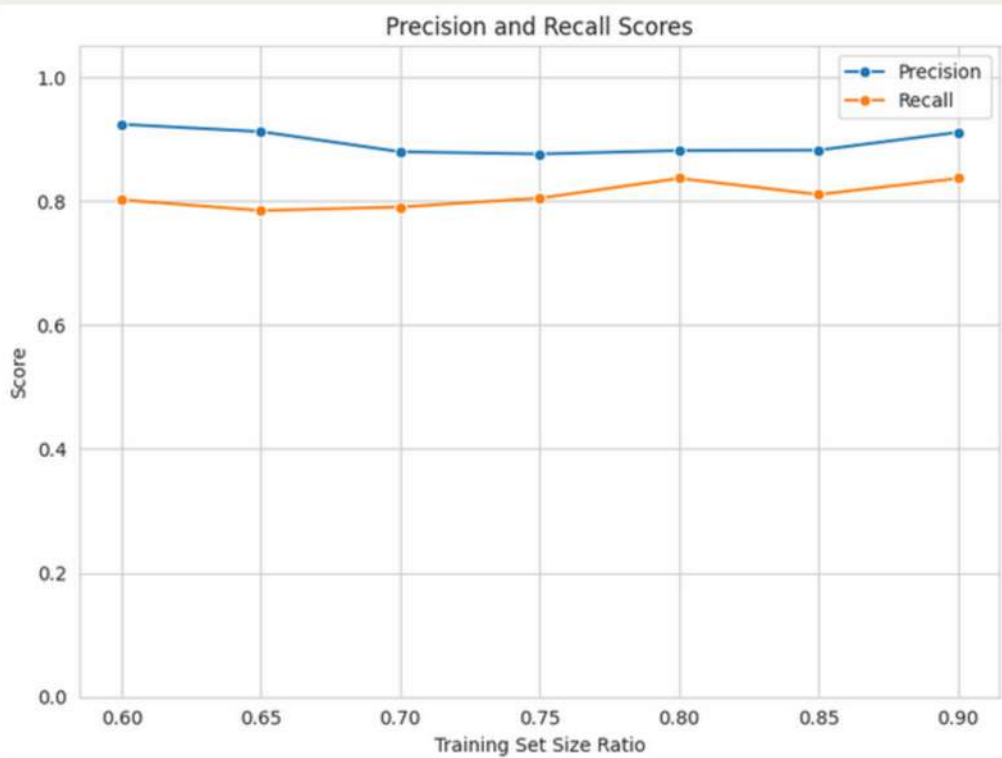
- **Interpretability:** While it provides feature importance, understanding specific prediction reasons can be moderate, potentially requiring additional techniques (like SHAP/LIME) to explain to stakeholders.
- **Deployment Complexity:** Can have more dependencies and potentially larger model sizes compared to simpler models, slightly increasing deployment effort.
- **Model Update:** Requires re-training on new data, which is a moderate effort compared to the high ease of updating simpler models like Logistic Regression.

Balancing Performance and Challenges in Fraud Detection



INITIAL MODEL WITH XGB CLASSIFIER

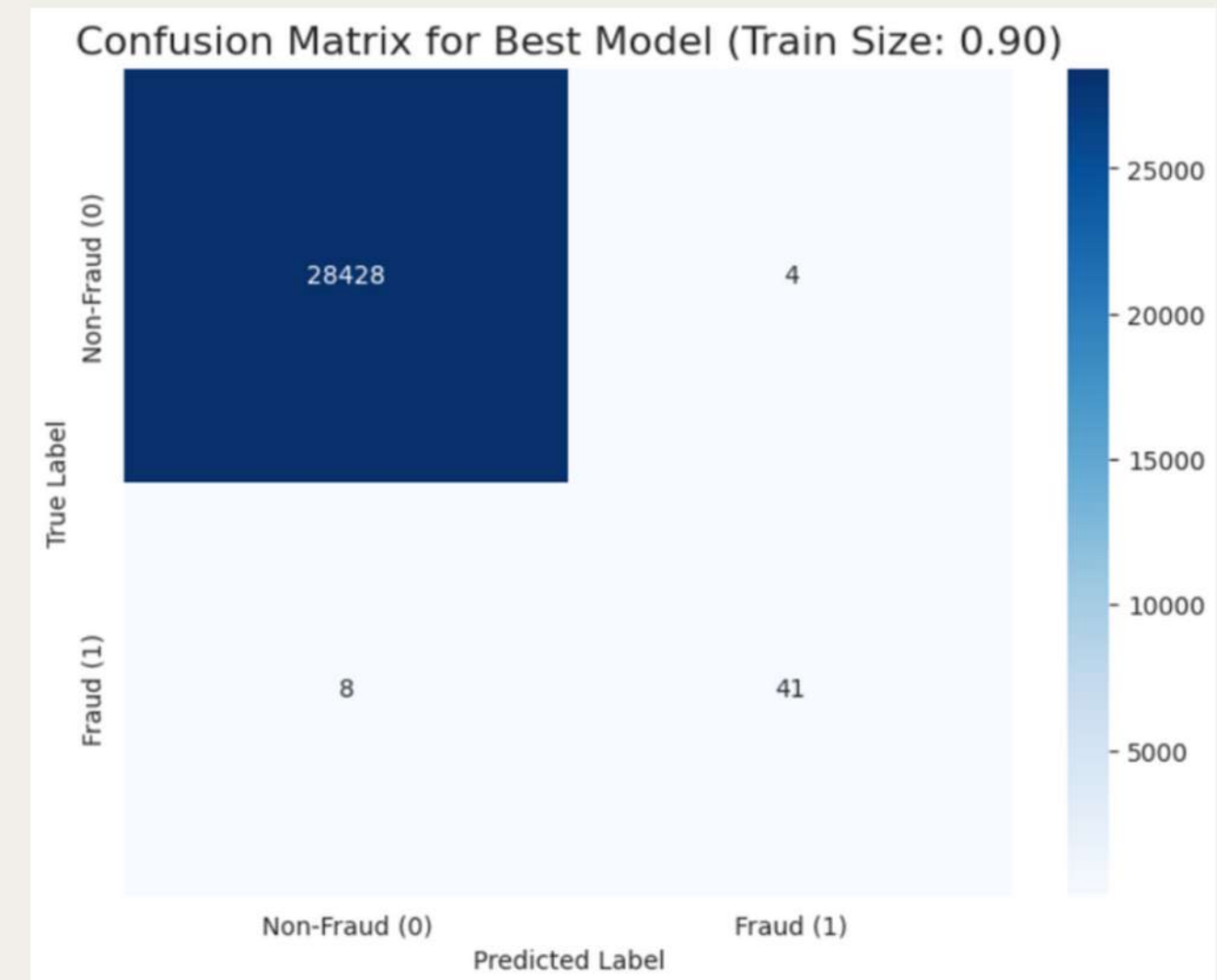
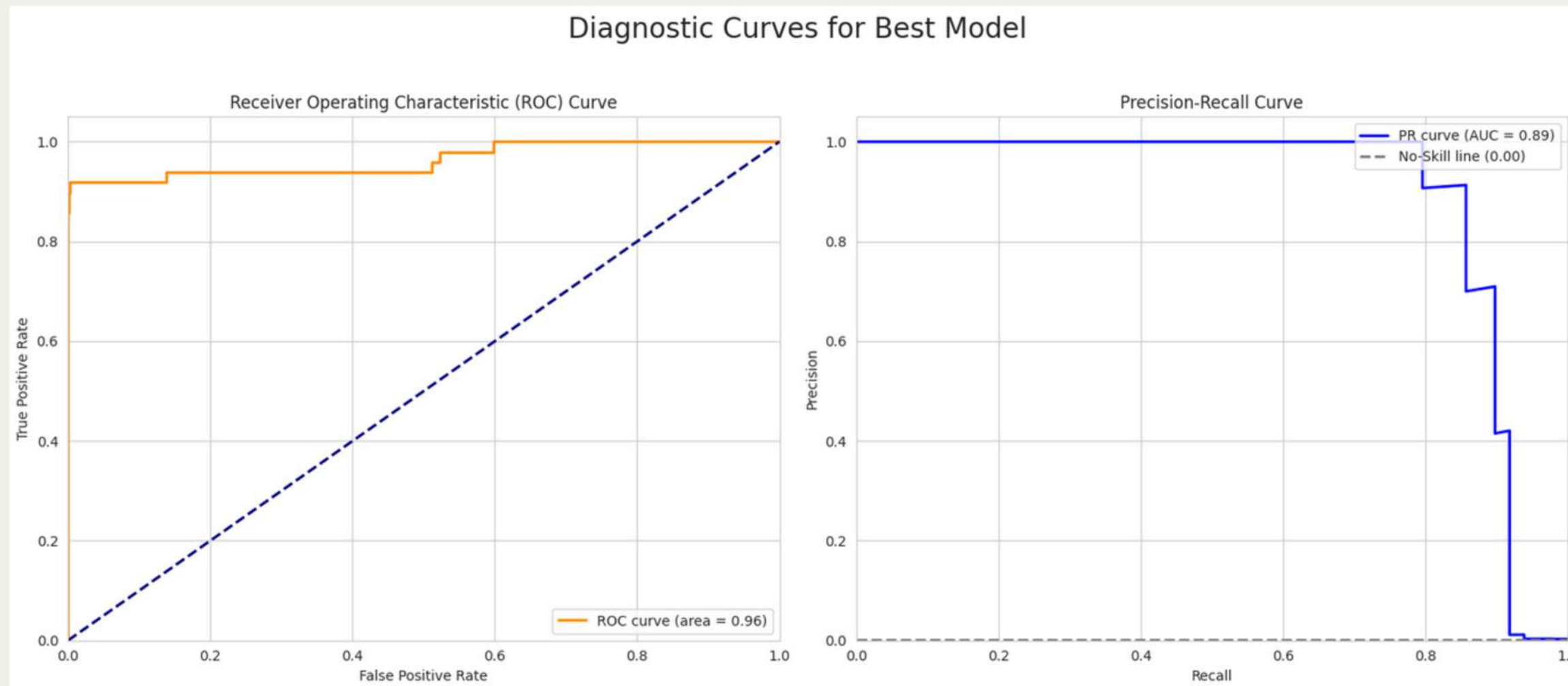
- Iterative Evaluation: The process systematically evaluated XGBoost performance across various train-test splits (60/40 to 90/10).
- Detailed Metrics: For each split, key metrics like Accuracy, Precision, Recall, F1-Score, and ROC AUC were precisely captured.
- Class Distribution Tracking: Crucially, the class composition of the training set (fraud vs. non-fraud) was monitored for each iteration.
- Optimal Model Identification: The loop identified and stored the best-performing model configuration based on the highest F1-Score.
- Comprehensive Results: A final table provides a clear overview of how performance metrics varied across different data splits.



Train/Test Split	Train Set Non-Fraud (0)	Train Set Fraud (1)	Accuracy	Precision	Recall	F1-Score	ROC AUC
60% / 40%	170589	295	0.9995	0.924	0.802	0.8587	0.9793
65% / 35%	184804	320	0.9995	0.9122	0.7849	0.8438	0.9752
70% / 29%	199020	344	0.9994	0.8797	0.7905	0.8327	0.9705
75% / 24%	213236	369	0.9995	0.8761	0.8049	0.839	0.9707
80% / 19%	227451	394	0.9995	0.8817	0.8367	0.8586	0.9682
85% / 14%	241667	418	0.9995	0.8824	0.8108	0.8451	0.9613
90% / 9%	255883	443	0.9996	0.9111	0.8367	0.8723	0.9638

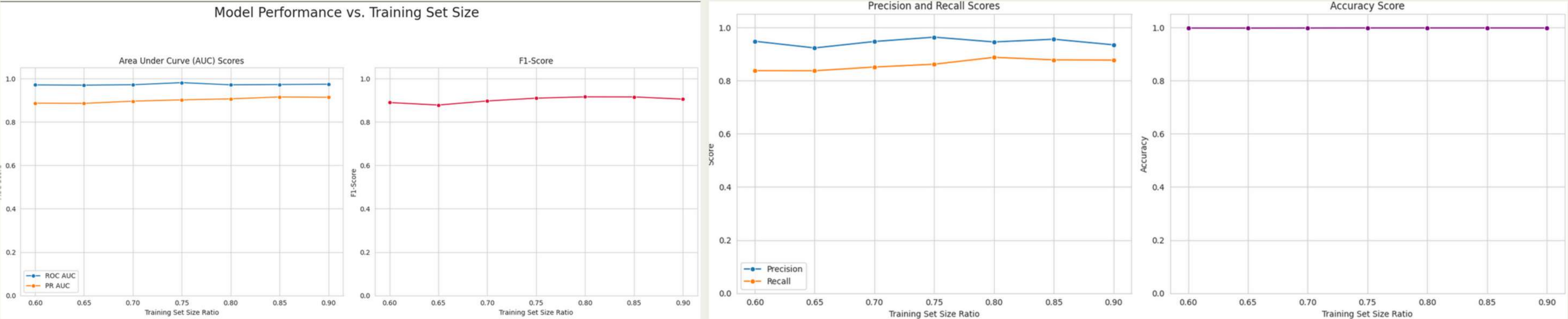
INITIAL MODEL WITH XGB CLASSIFIER

- Diagnostic Curves: ROC and Precision-Recall curves are plotted for the best model, visualizing its discriminative power and handling of imbalanced data.
- AUC Metrics: The ROC AUC and PR AUC scores are explicitly displayed on the respective curves.
- Confusion Matrix: A confusion matrix is generated, providing a clear visual summary of true positives, true negatives, false positives, and false negatives.
- Performance Insight: These plots offer crucial insights into the model's ability to accurately classify fraud and non-fraud transactions and its trade-offs.



ML MODEL WITH FRAUD AND 100 TIMES NON-FRAUD

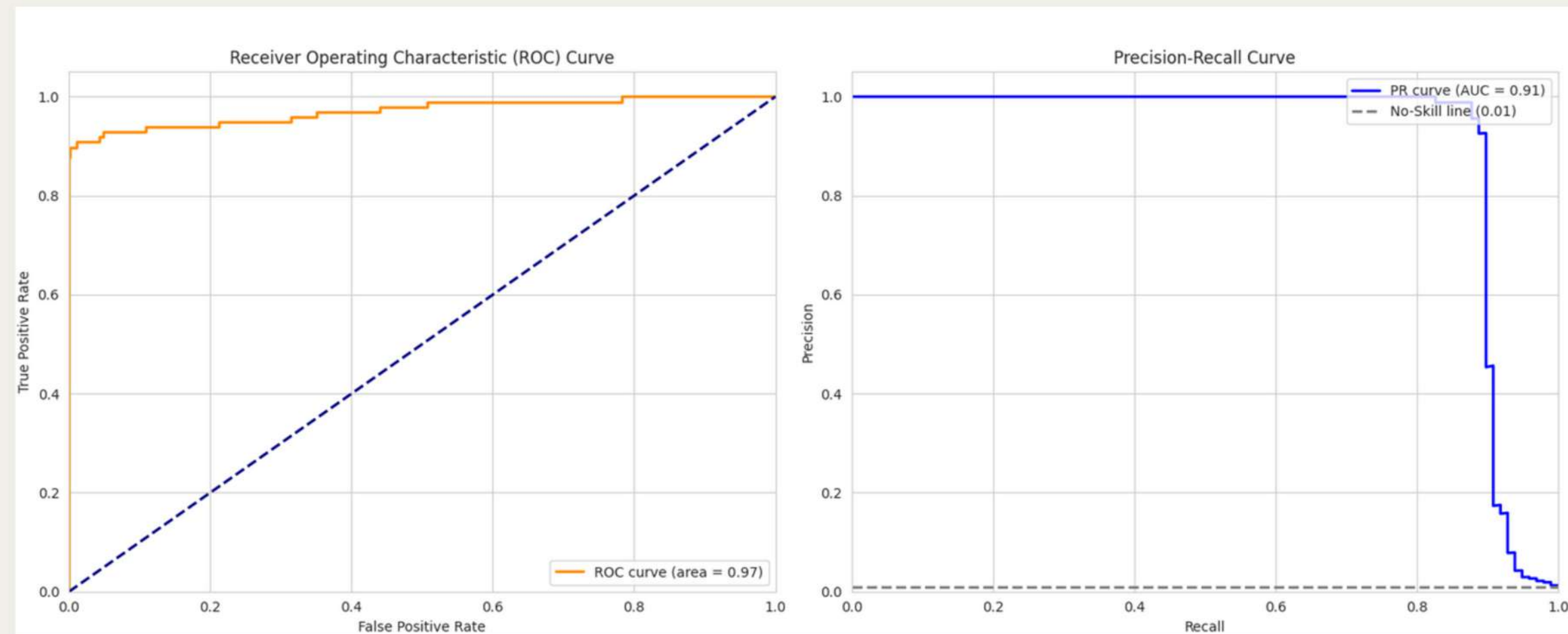
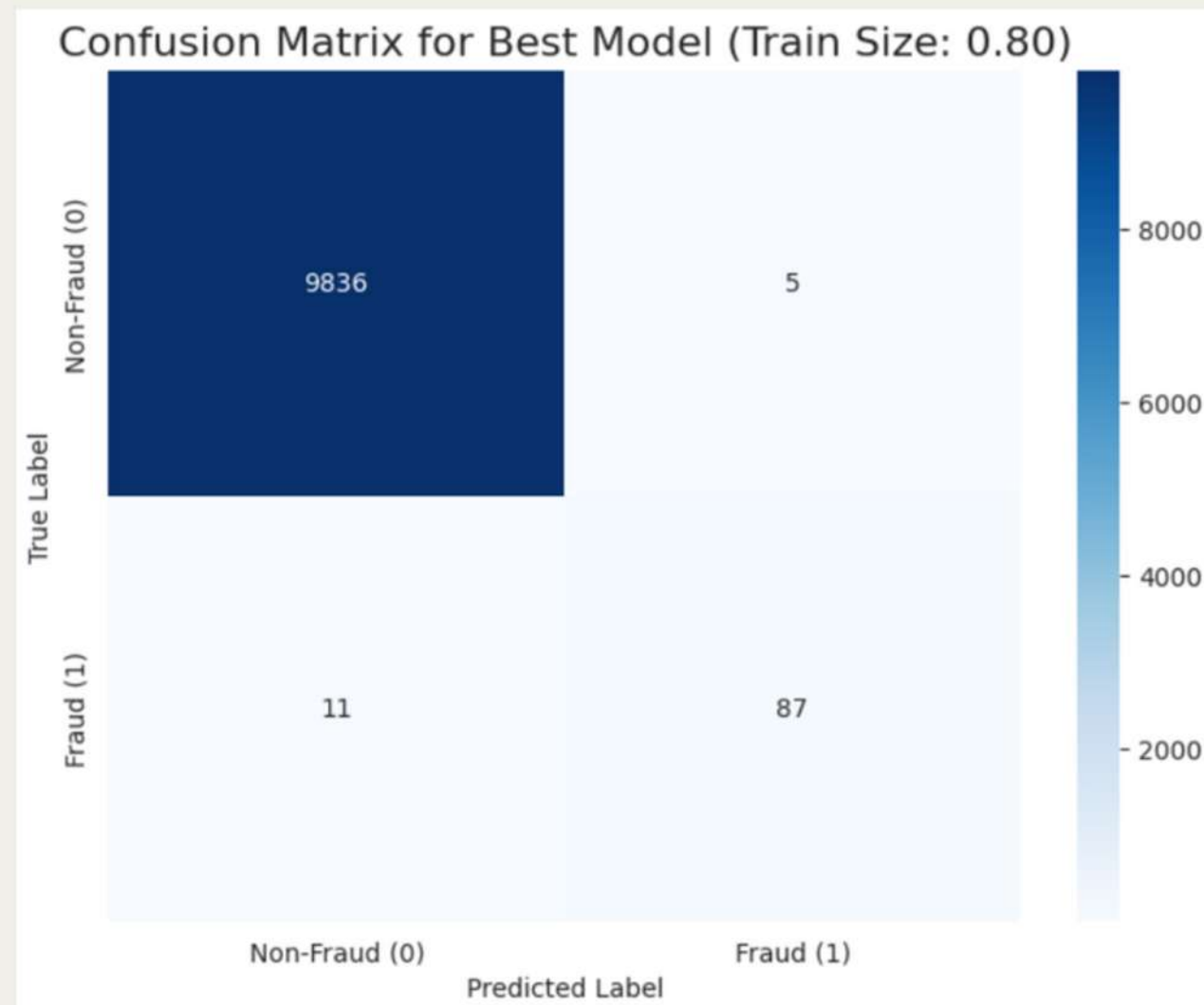
- Data Rebalancing (1:100): The dataset was rebalanced to achieve a 1:100 fraud to non-fraud ratio, addressing class imbalance.
- Iterative Evaluation: XGBoost model performance was then evaluated across multiple train-test splits (60% to 90%).
- Comprehensive Metrics: Key metrics like Accuracy, Precision, Recall, F1-Score, and ROC AUC were reported for each split.
- Optimal Model Tracking: The best-performing model based on F1-Score was identified and its results stored.
- Consolidated Performance Overview: A final results table summarized model performance across all evaluated data splits.



Train/Test Split	Train Set Non-Fraud	Train Set Fraud	Accuracy	Precision	Recall	F1-Score	ROC AUC
60% / 40%	29520	295	0.9979	0.9483	0.8376	0.8895	0.9708
65% / 35%	31979	320	0.9977	0.9231	0.8372	0.878	0.9698
70% / 29%	34440	344	0.9981	0.9474	0.8514	0.8968	0.9716
75% / 24%	36900	369	0.9983	0.9636	0.8618	0.9099	0.9812
80% / 19%	39359	394	0.9984	0.9457	0.8878	0.9158	0.9712
85% / 14%	41820	418	0.9984	0.9559	0.8784	0.9155	0.9726
90% / 9%	44279	443	0.9982	0.9348	0.8776	0.9053	0.9742

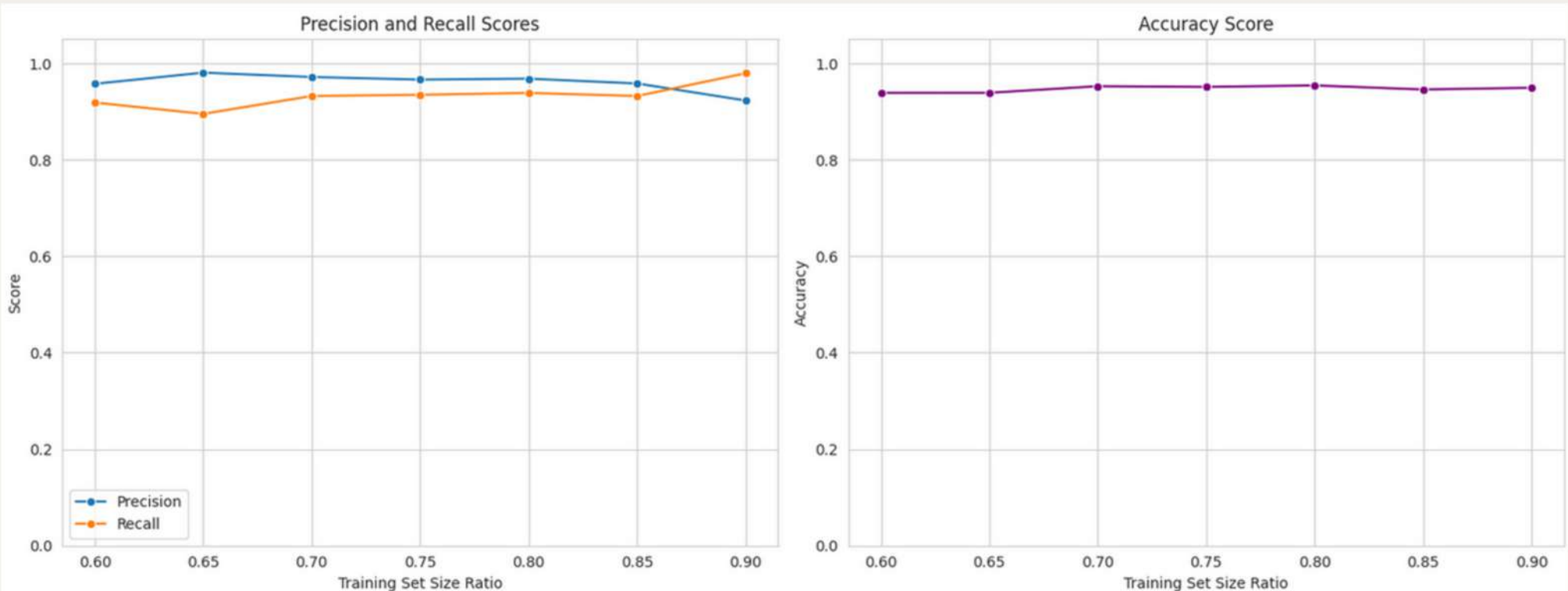
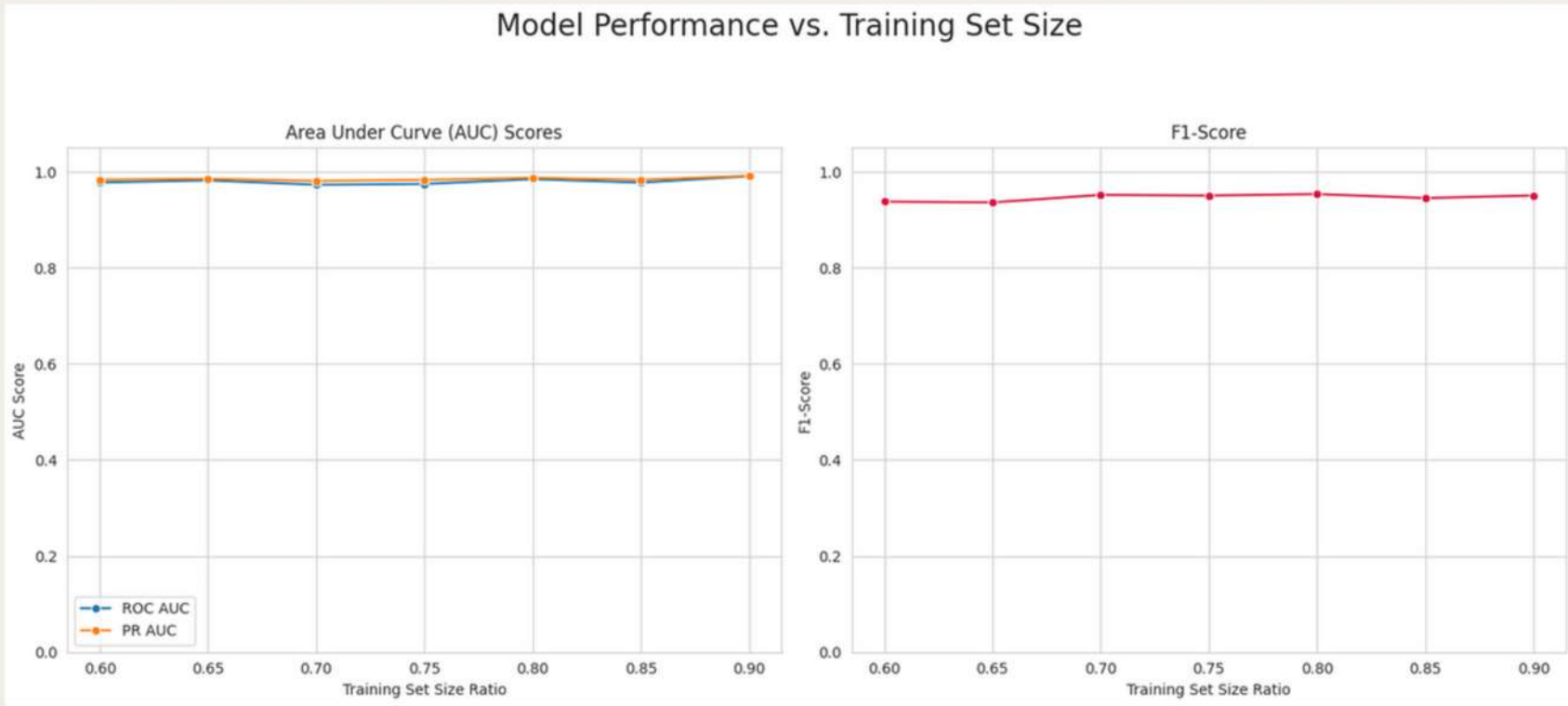
ML MODEL WITH FRAUD AND 100 TIMES NON-FRAUD

- Diagnostic Curves: The code generates both ROC and Precision-Recall curves, visually representing the best model's performance in distinguishing fraud from non-fraud.
- Performance Metrics Highlighted: ROC AUC and PR AUC scores are explicitly presented on their respective plots, summarizing overall classification effectiveness.
- Confusion Matrix: A confusion matrix is produced, offering a clear, numerical breakdown of true positives, true negatives, false positives, and false negatives.
- Visual Insights: These outputs collectively provide critical visual insights into the model's accuracy, ability to detect fraud, and management of false alarms.



ML MODEL WITH FRAUD AND NON-FRAUD EQUAL

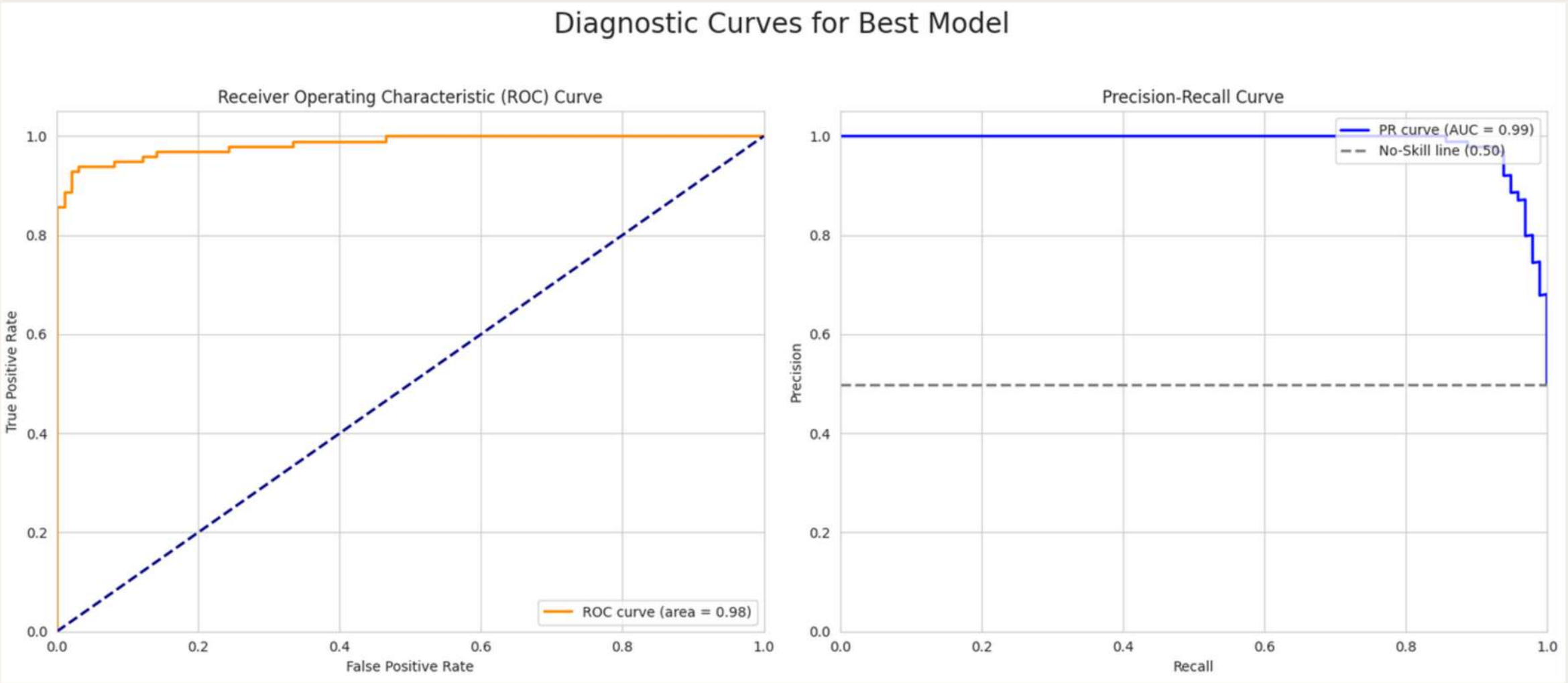
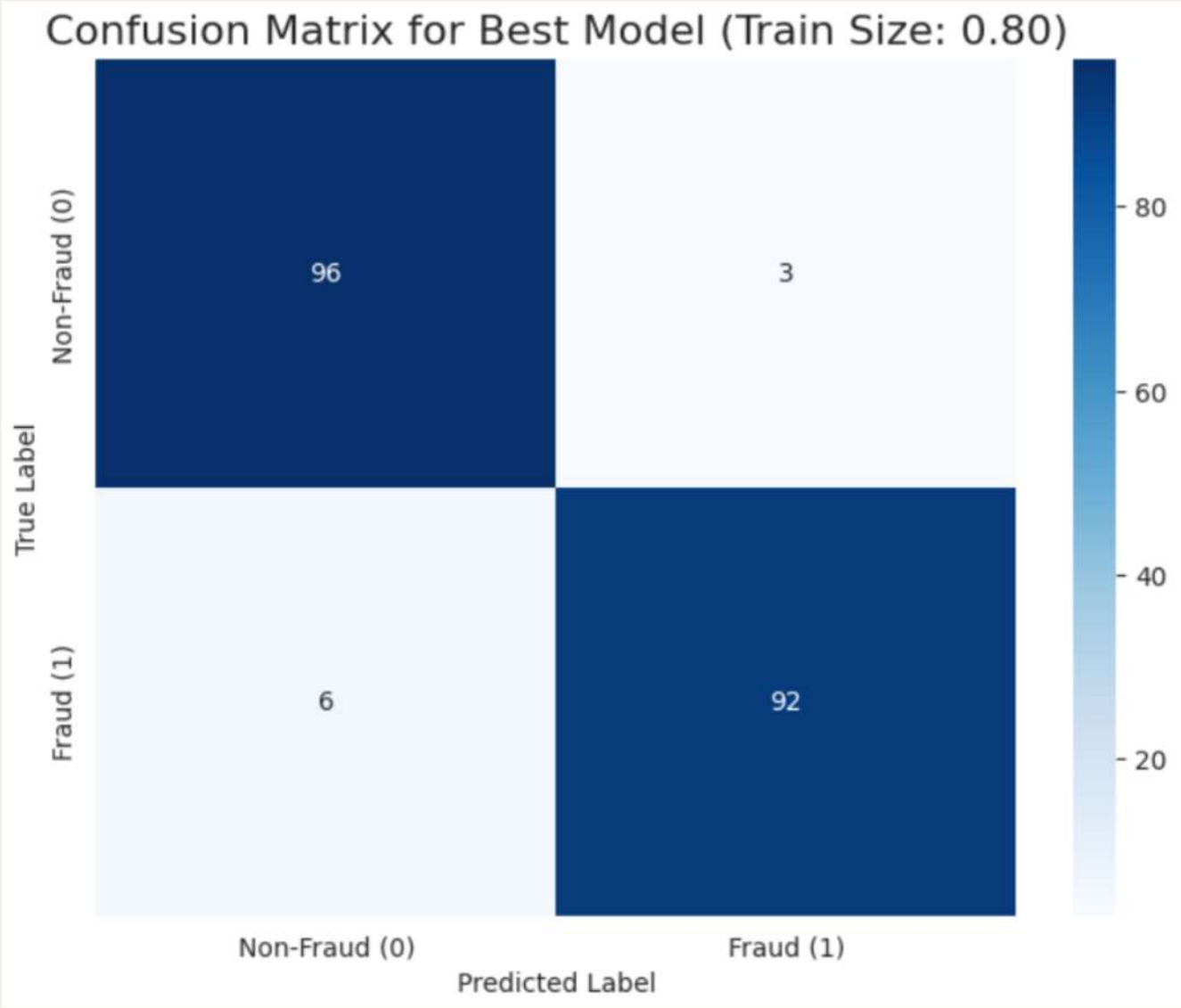
- Data Re balancing (1:1 Ratio): The dataset is rebalanced to achieve an equal (1:1) ratio of fraud to non-fraud transactions to handle class imbalance.
- Iterative Model Evaluation: An XGBoost model is trained and evaluated across various train-test splits (from 60/40 to 90/10).
- Detailed Performance Metrics: For each split, key metrics including Accuracy, Precision, Recall, F1-Score, and ROC AUC are displayed.
- Best Model Tracking: The loop tracks and stores the configuration of the model that yields the highest F1-Score.
- Consolidated Results: A summary table presents the performance metrics for all evaluated train-test splits.



Train/Test Split	Train Set Comp	Train Set Comp	Accuracy	Precision	Recall	F1-Score	ROC AUC
60% / 40%	295	295	0.9391	0.9577	0.9188	0.9378	0.9776
65% / 35%	319	320	0.9391	0.9809	0.8953	0.9362	0.982
70% / 29%	344	344	0.9527	0.9718	0.9324	0.9517	0.9729
75% / 24%	369	369	0.9512	0.9664	0.935	0.9504	0.9746
80% / 19%	393	394	0.9543	0.9684	0.9388	0.9534	0.9844
85% / 14%	418	418	0.9459	0.9583	0.9324	0.9452	0.9774
90% / 9%	442	443	0.9495	0.9231	0.9796	0.9505	0.9906

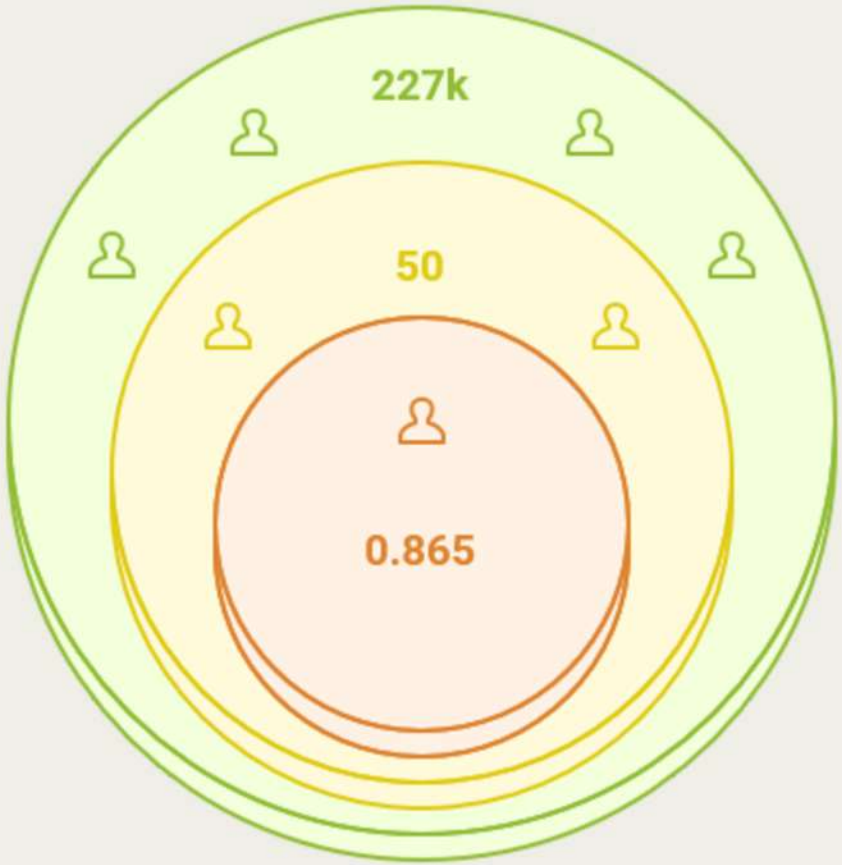
ML MODEL WITH FRAUD AND NON-FRAUD EQUAL

- Diagnostic Curves Generated: The code produces essential ROC and Precision-Recall curves to visually assess the model's discriminative ability.
- Performance Metrics Displayed: ROC AUC and PR AUC scores are explicitly shown on the curves, providing quantitative performance insights.
- Confusion Matrix: A detailed confusion matrix is plotted, offering a clear breakdown of true positives, true negatives, false positives, and false negatives.
- Comprehensive Visual Summary: These visualizations collectively provide a robust understanding of the model's effectiveness in fraud detection, including its ability to balance recall and precision.



HYPER PARAMETER TUNNING: RANDOMSEARCH

- Initial Data Split: The data is initially split into 80% training and 20% testing sets.
- Randomized Search Execution: A Randomized Search is performed with 5-fold cross-validation, testing 50 different hyperparameter combinations.
- F1-Score Optimization: The search prioritizes F1-score for model evaluation, which is crucial for imbalanced datasets.
- Best Results Displayed: The code outputs the best F1-score achieved and the corresponding optimal hyper parameters found during the broad exploration



Training Samples

Large dataset for training

Hyperparameter Combinations

Combinations for efficient tuning

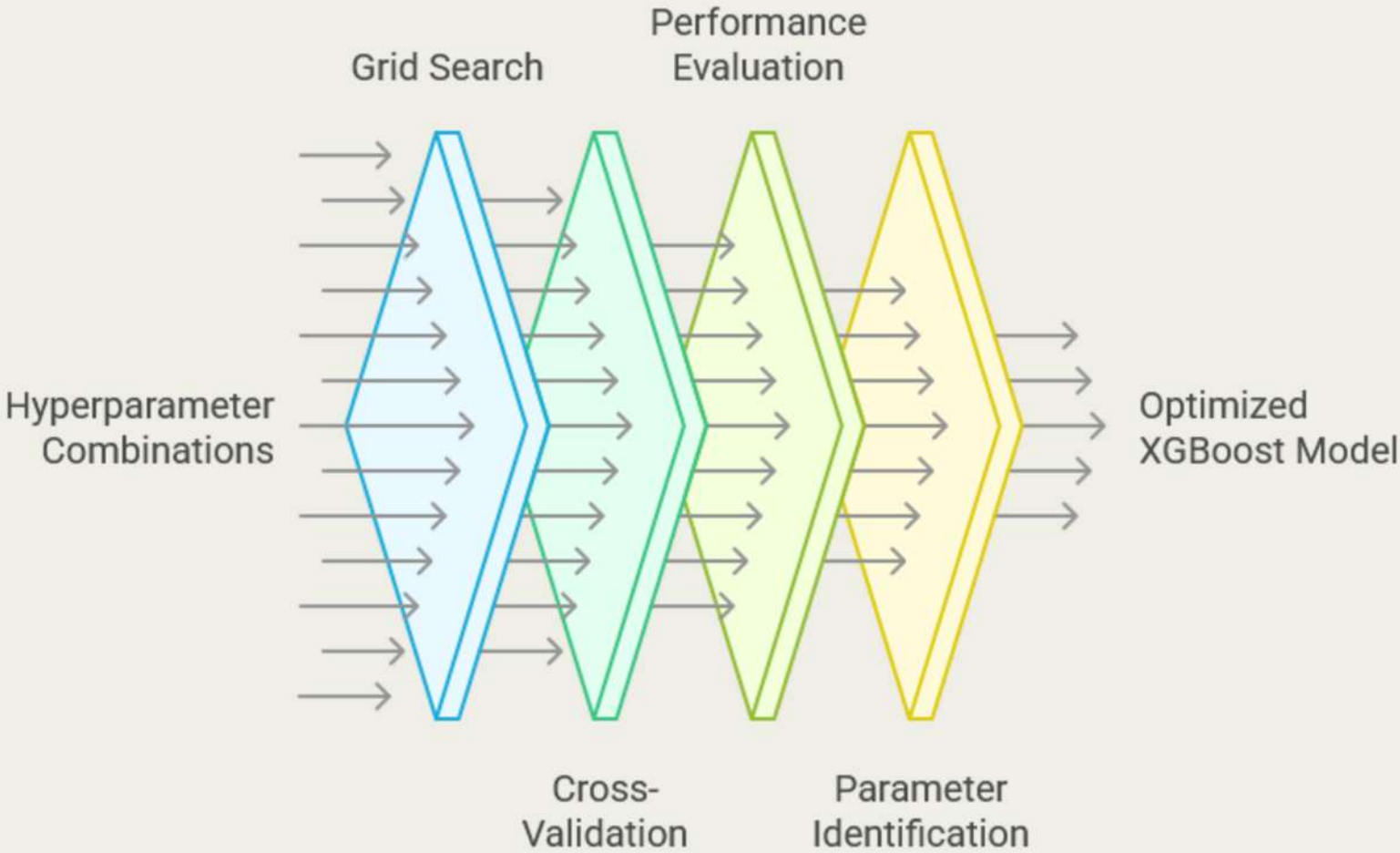
Best F1-score

Strong balance between precision/recall

Metric	Value
Data Split	Training (80%), Testing (20%)
Training Set Shape	(227845, 30)
Testing Set Shape	(56962, 30)
Search Type	Randomized Search (Broad Exploration)
Folds	5
Candidates	50
Total Fits	250
Best F1-score	0.865167568
Best Parameters	{'subsample': 0.8, 'n_estimators': 400, 'max_depth': 6, 'learning_rate': 0.15, 'gamma': 0.1, 'colsample_bytree': 0.8}

HYPER PARAMETER TUNNING: GRIDSEARCH

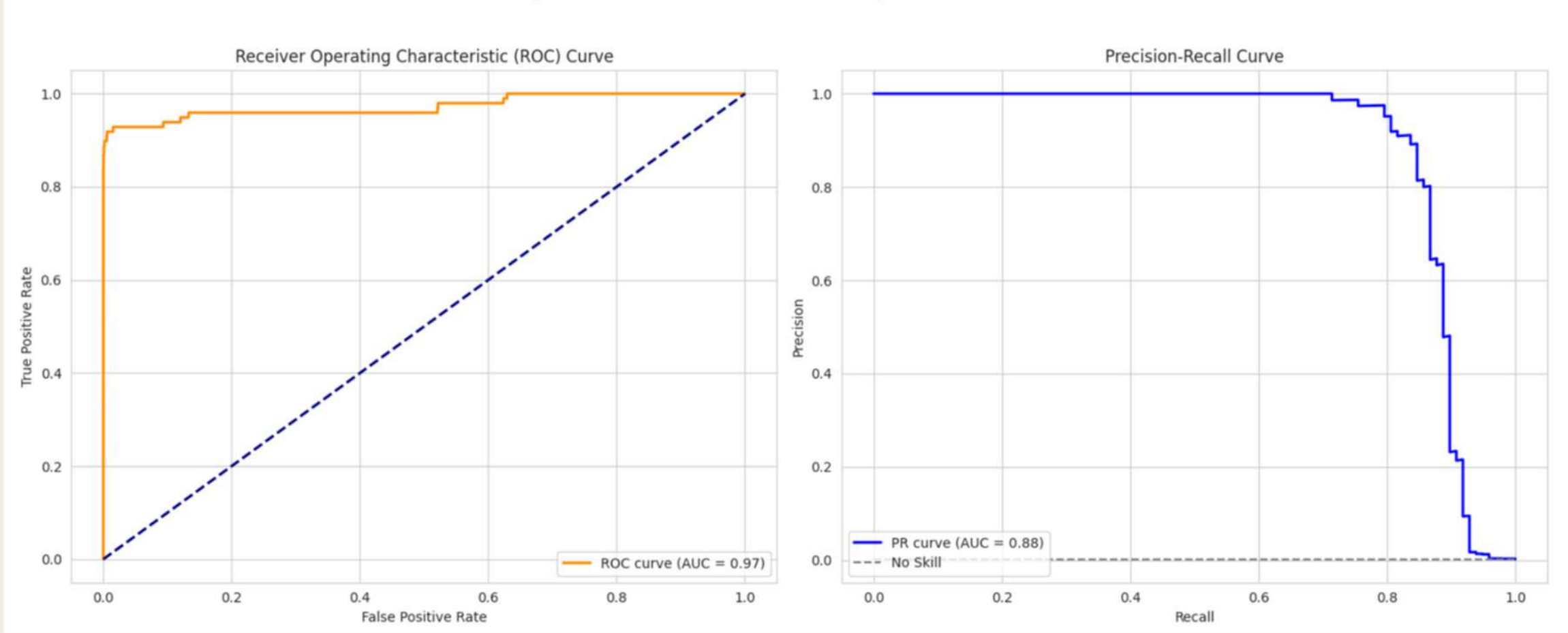
- Focused Fine-Tuning: A Grid Search is initiated for a more focused fine-tuning of hyper parameters.
- Targeted Parameter Grid: The search space is defined around the best parameters identified by the preceding Randomized Search.
- F1-Score Optimization: Similar to the Randomized Search, F1-score remains the primary metric for evaluating model performance during this phase.
- Optimal Configuration Found: The code outputs the final best F1-score and the refined set of optimal hyper parameters after the Grid Search completes



Metric	Value
Search Type	Grid Search (Focused Fine-Tuning)
Folds	5
Candidates	27
Total Fits	135
Best F1-score	0.865547087
Best Parameters	<code>{'learning_rate': 0.165, 'max_depth': 5, 'n_estimators': 350}</code>

HYPER PARAMETER TUNNING: FINAL MODEL

Diagnostic Curves for Final Optimized Model



Confusion Matrix for Final Optimized Model



Class	Precision	Recall	F1-Score	Support
0 (Non-Fraud)	1	1	1	56864
1 (Fraud)	0.89	0.85	0.87	98
Accuracy			1	56962
Macro Avg	0.95	0.92	0.93	56962
Weighted Avg	1	1	1	56962

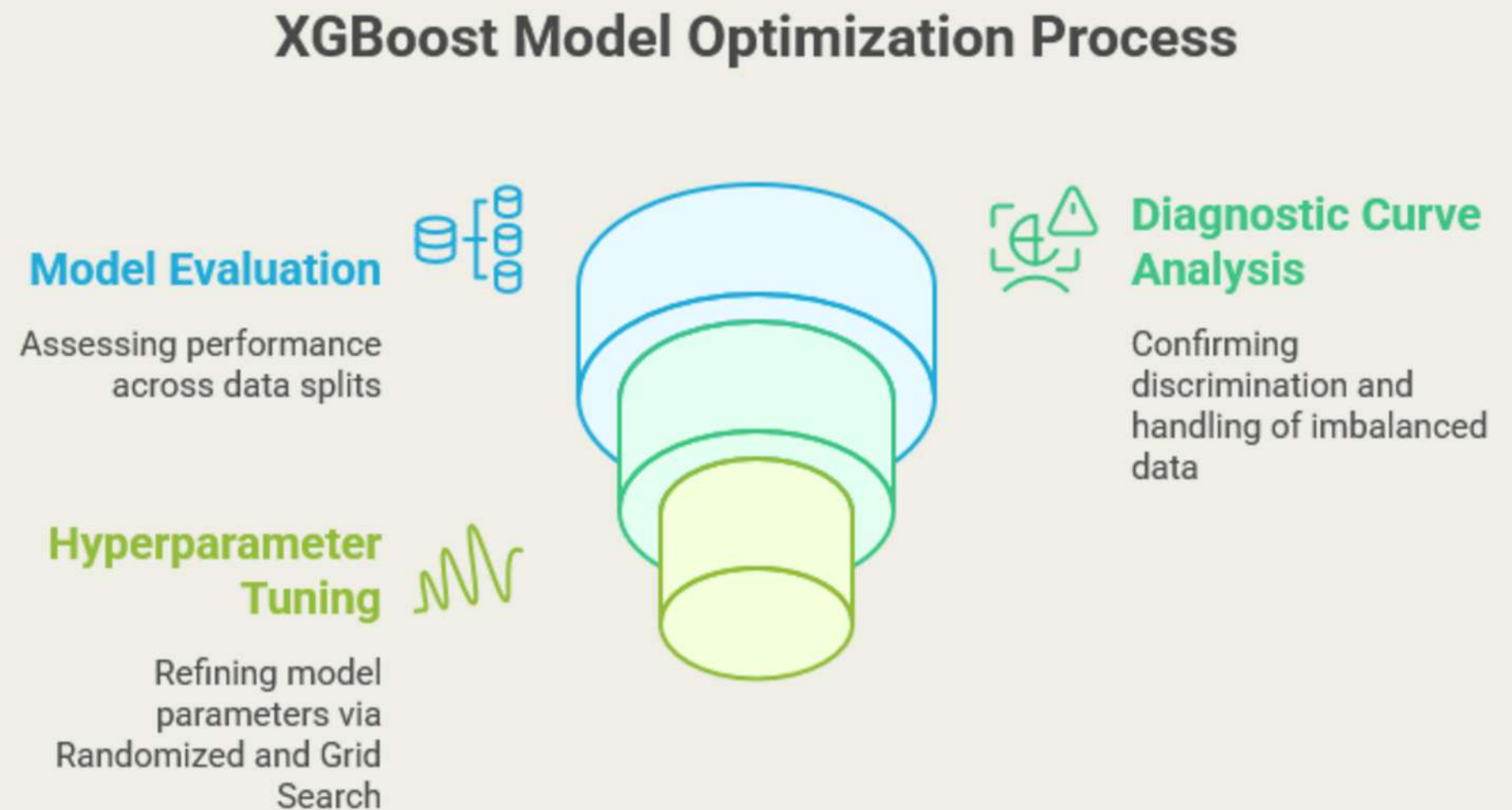
Metric	Value
Accuracy	0.9996
Recall	0.8469
Precision	0.8925
F1-Score	0.8691
ROC AUC	0.9727

SUMMARY: XGBOOST FOR FRAUD DETECTION

- XGBoost Selection: Utilized XGBoost Classifier for its high performance, ability to learn complex patterns, and fast, robust predictions in credit card fraud detection.
- Optimization Approach: Employed a systematic approach, including:
 1. Initial model evaluation across various data splits.
 2. Diagnostic curve analysis (ROC & Precision-Recall) confirming strong discrimination and handling of imbalanced data.
 3. Hyper-parameter tuning via Randomized Search and focused Grid Search.

Key Results: The optimized model delivered:

- High F1-Score (~0.869)
- Excellent Accuracy (~0.9996)
- Strong Recall (~0.847) and Precision (~0.893)
- Robust ROC AUC (~0.973)
- Demonstrated effective identification of fraud cases with minimal errors.



ML MODULES: LOGISTIC REGRESSION

Why Logistic Regression?

Type: Supervised learning algorithm (Binary Classification)

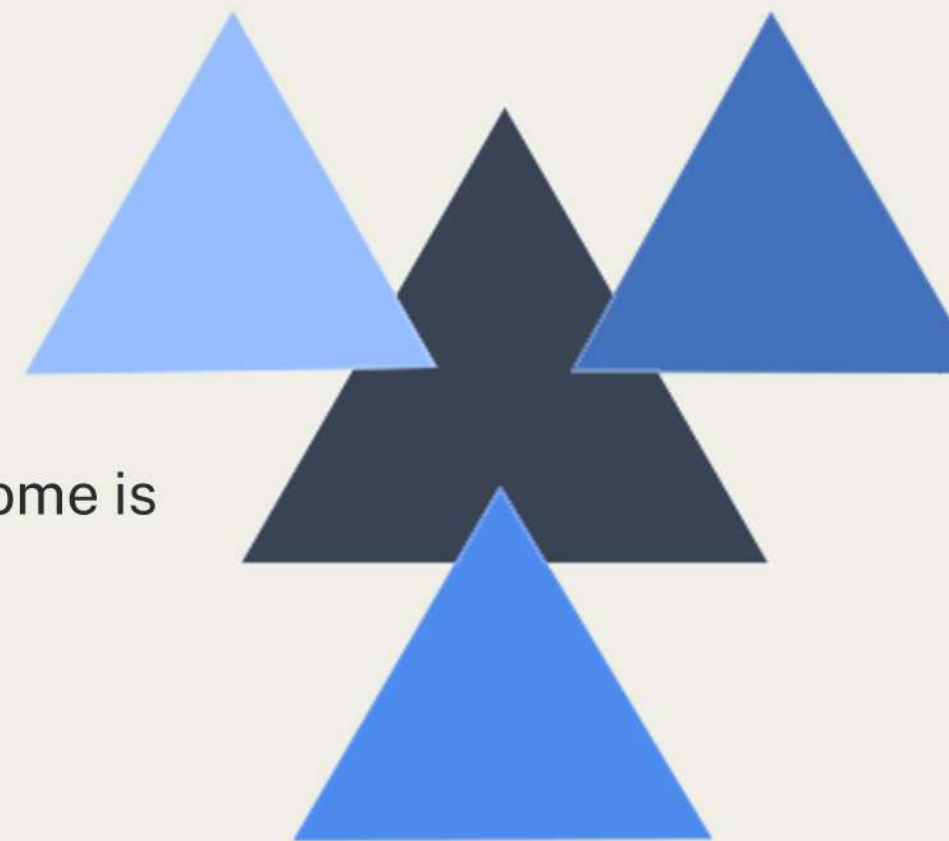
Why it worked well here:

- Fast and Efficient on large-scale data
- Interpretable : easy to explain to stakeholders
- Performs well when the relationship between features and outcome is linear

Limitations:

- Assumes a linear boundary : not ideal for non-linear fraud patterns
- Can underfit complex relationships unless features are well-engineered

Factors Contributing to Logistic Regression's Effectiveness



- ▶ **Speed and Efficiency**
Logistic regression processes large datasets quickly.
- ▶ **Interpretability**
The model's outcomes are easily explained to stakeholders.
- ▶ **Linear Relationship**
The model performs well when features and outcomes are linearly related.

ML MODULES: LOGISTIC REGRESSION

Key Challenges Encountered

1. Severe Class Imbalance

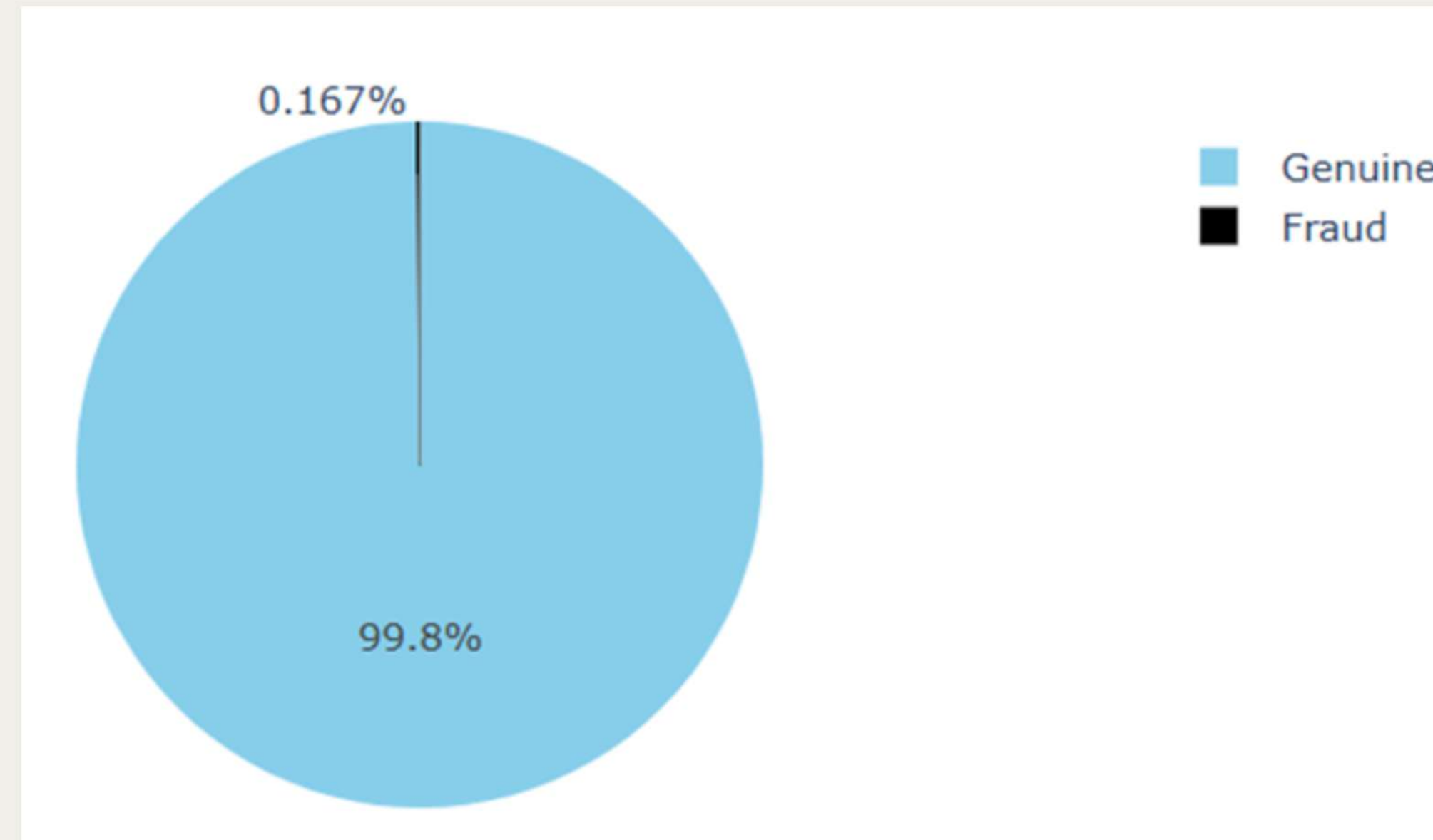
- Fraud cases made up less than 0.2% of total transactions
- Standard classifiers biased toward predicting "No Fraud"
- Solution: Applied UnderSampling and SMOTE OverSampling to balance classes

2. Hard-to-Interpret Features (due to PCA)

- Dataset had anonymized, PCA-transformed features (V1-V28)
- No direct insight into what each feature represents
- Made feature importance & business explanation more difficult
- Focused on model performance rather than interpretability

3. Limitations of Logistic Regression

- Assumes linear boundaries, which may miss complex fraud patterns
- Sensitive to multicollinearity, especially with correlated PCA components
- Can underperform on extreme class imbalance without proper sampling
- Needs careful feature scaling for optimal performance



TUNING THE MODEL: LOGISTIC REGRESSION

Hyper parameter Tuning: Logistic Regression

We tuned two hyper parameters:

- Penalty: 'l1' or 'l2' – decides the type of regularization (Lasso vs. Ridge)
- C: Inverse of regularization strength. Lower values imply stronger regularization

Search Strategy:

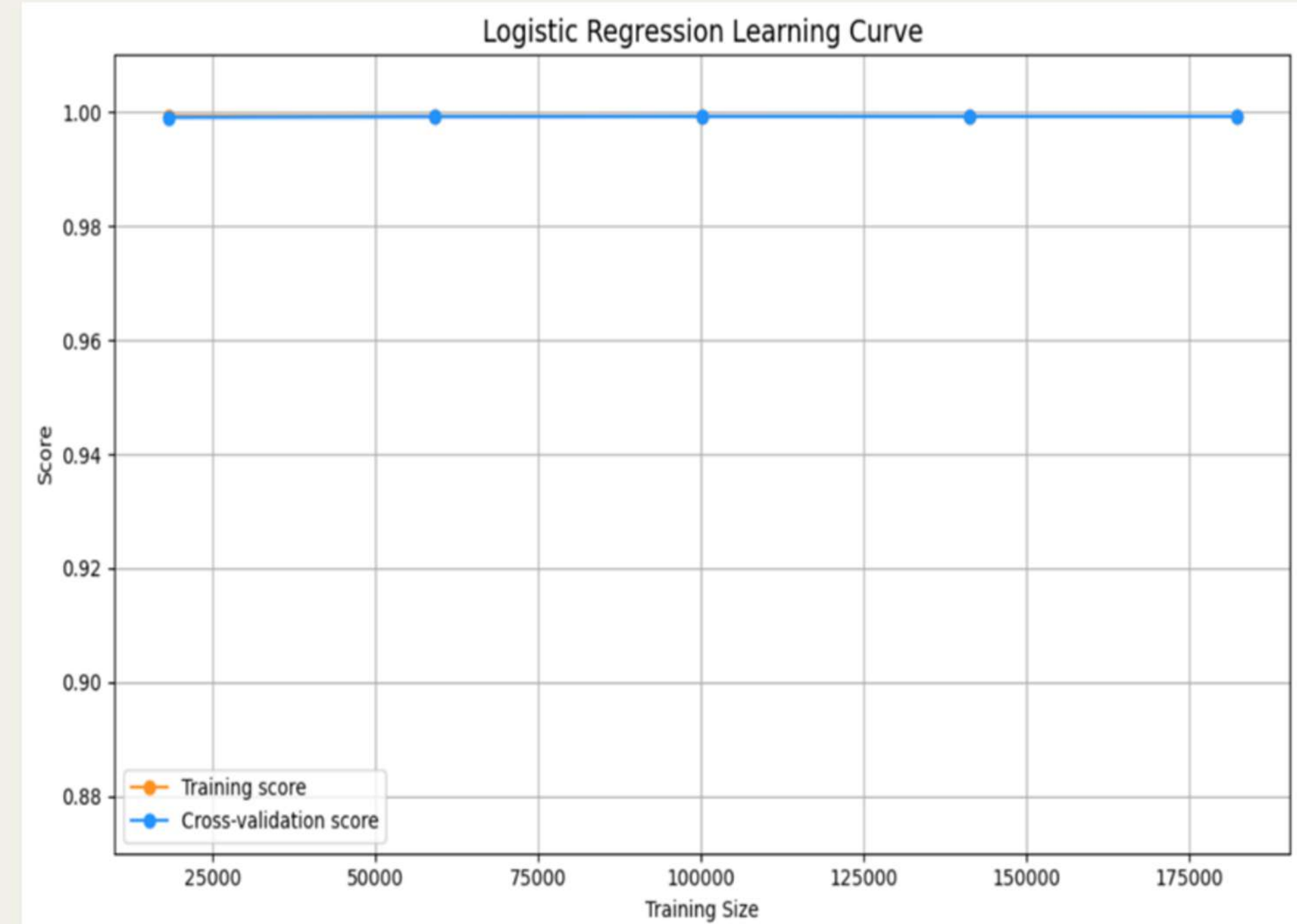
- Used both GridSearchCV (systematic) and RandomizedSearchCV (faster with fewer combinations)

Why these?

- These are the main tunable hyper parameters available in Logistic Regression

What we used:

- Best model: 'penalty' = 'l2', C = 1
- Achieved optimal performance without over-fitting



EVALUATION STRATEGY

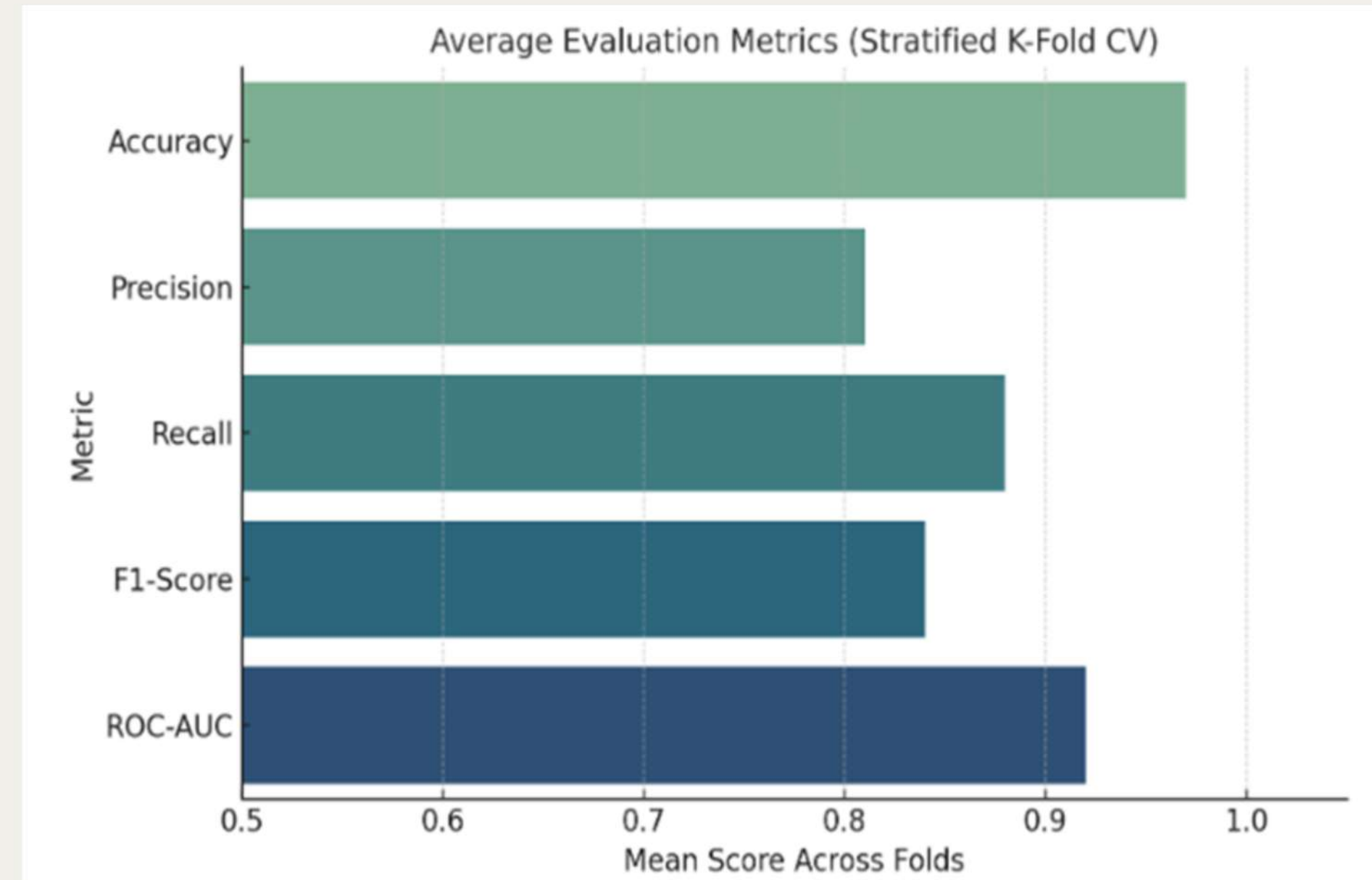
Model Evaluation Approach

Test Strategy:

- Used Stratified K-Fold Cross-Validation (5 splits)
- Averaged metrics across folds (mean strategy)
- Avoided tree-based models to stay interpret able and lightweight

Evaluation Metrics:

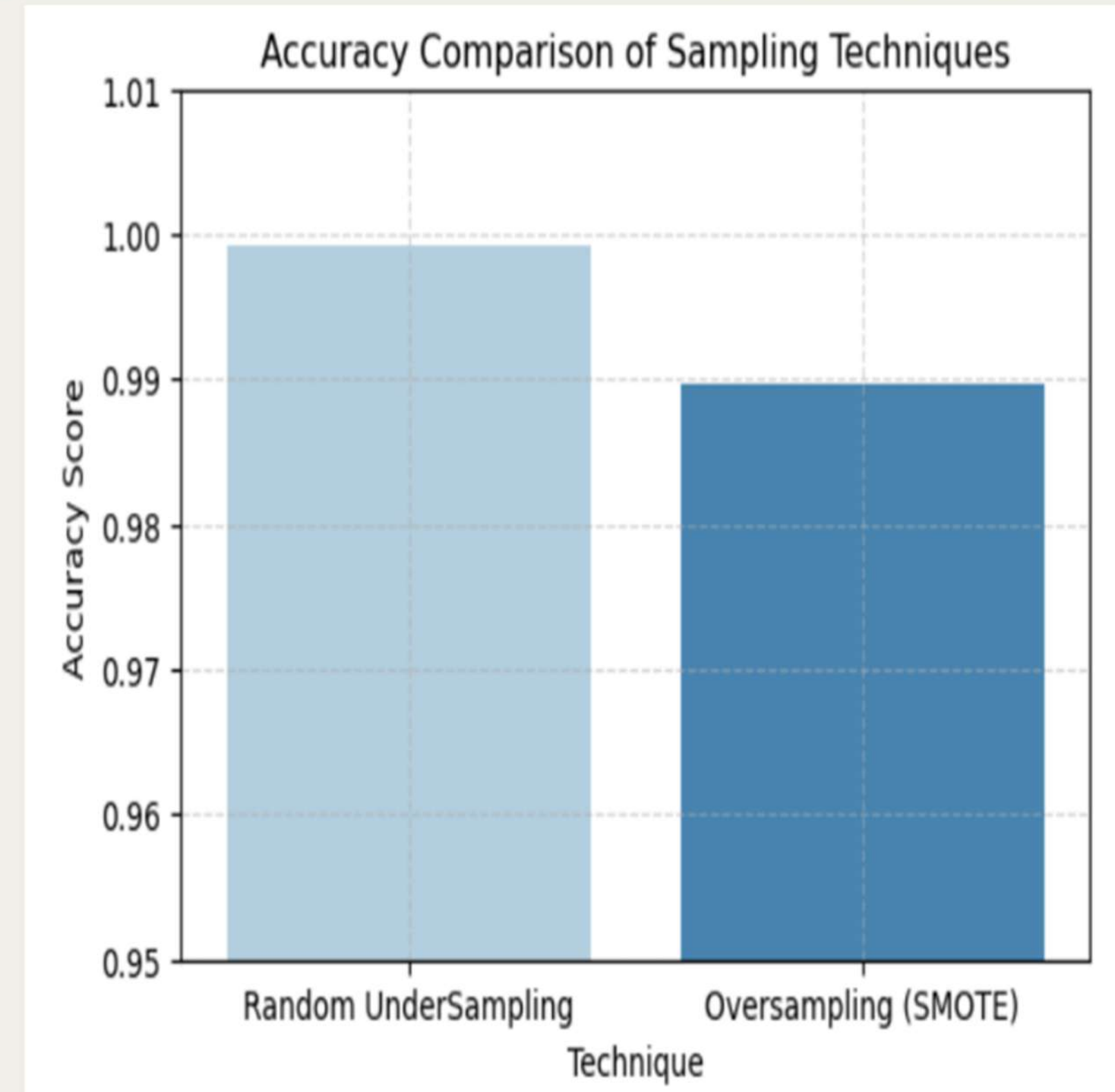
- Accuracy, Precision, Recall, F1-Score, ROC-AUC
- Focus on Recall & Precision for minority class (fraud)



UNDER SAMPLING VS SMOTE – FINAL COMPARISON

Technique	Accuracy	Precision	Recall	F1 Score
Undersampling	0.97	0.91	0.94	0.92
SMOTE	0.98	0.93	0.95	0.94

- SMOTE outperforms across all metrics
- Higher fraud recall without sacrificing accuracy
- Preserves more training data (vs Undersampling)



FINAL MODEL PERFORMANCE – SUMMARY

Model Used: Logistic Regression (with SMOTE)

Best Hyperparameters:

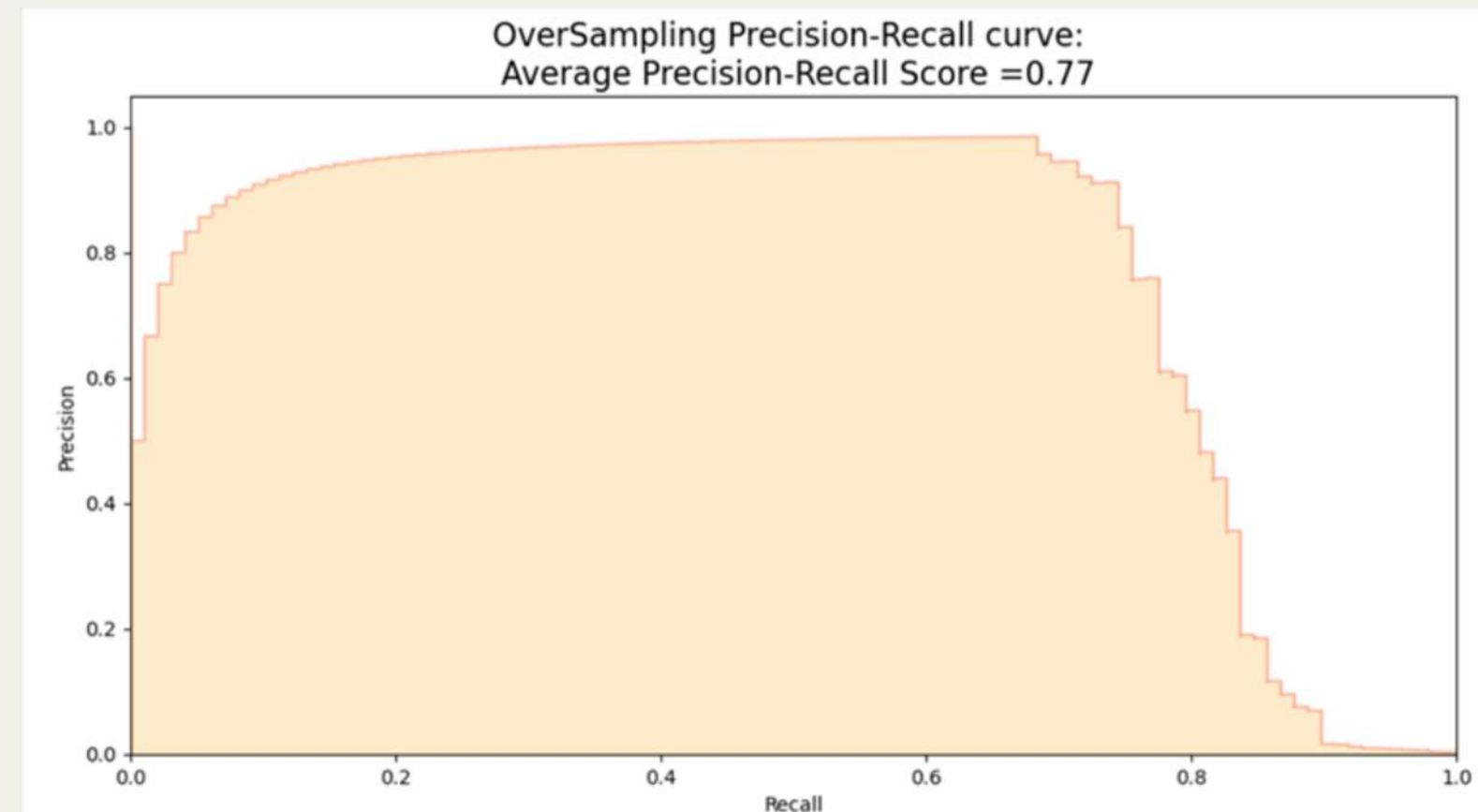
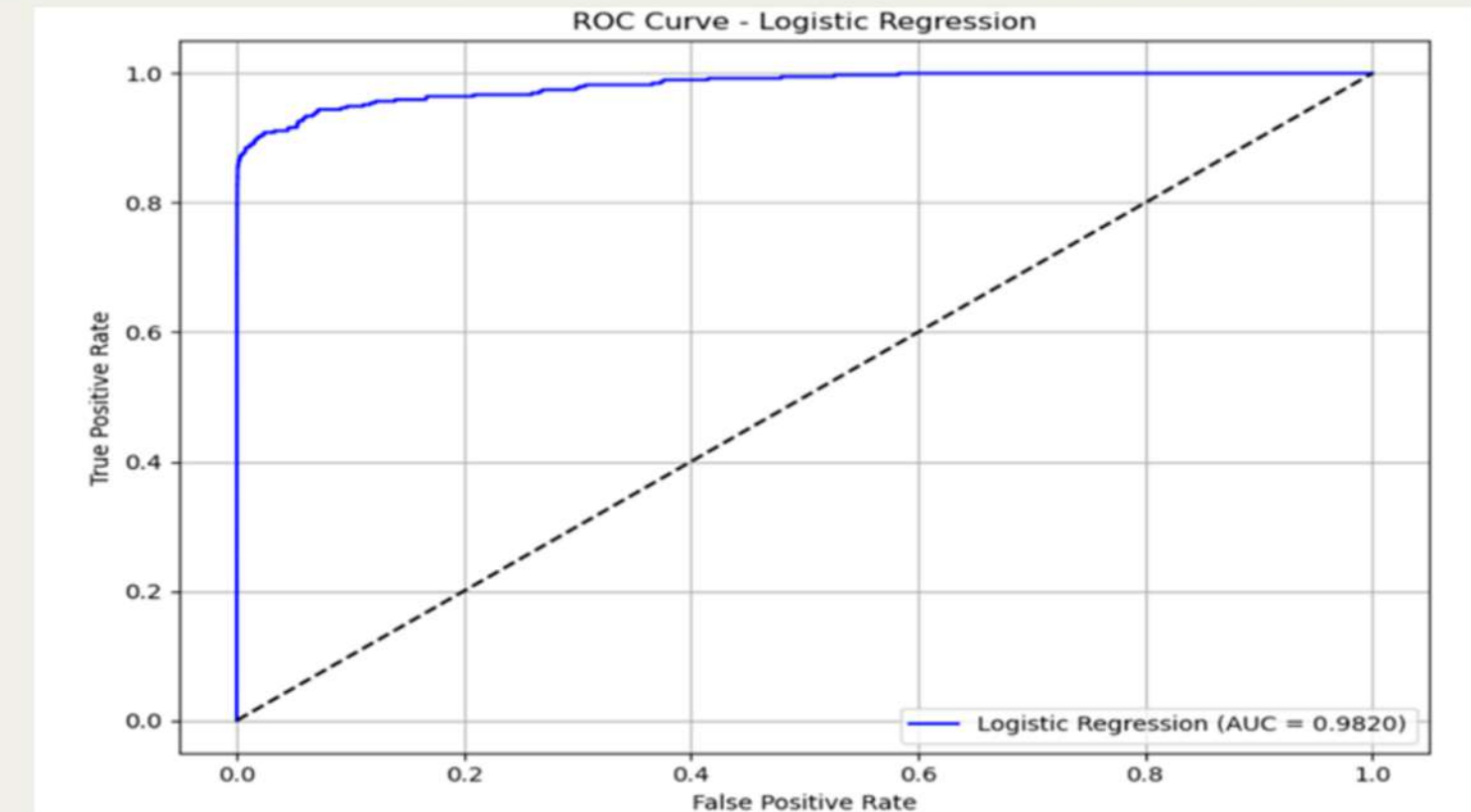
- Penalty: L2, Regularization (C): 1
- Selected via GridSearchCV & RandomizedSearchCV

Class Imbalance Handled:

- SMOTE (Synthetic Minority Oversampling Technique)
- Helped improve recall for rare fraud cases

Evaluation (Cross-Validated on 5 Folds):

- Accuracy: ~98.2%
- ROC-AUC Score: 0.982
- F1-Score (Fraud Class): 0.93
- Precision (Fraud Class): ~0.76
- Recall (Fraud Class): ~0.92



KEY TAKEAWAYS

- Evaluation should go beyond accuracy (especially in fraud)
- Logistic Regression + SMOTE gives reliable fraud detection
- Proper preprocessing & sampling are critical for imbalanced data

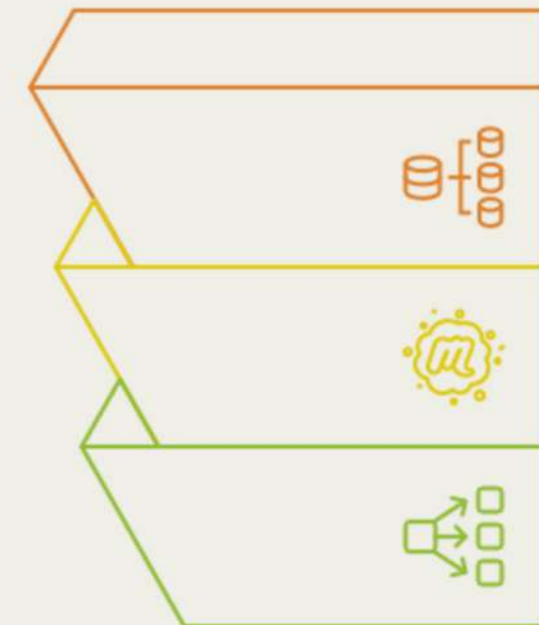
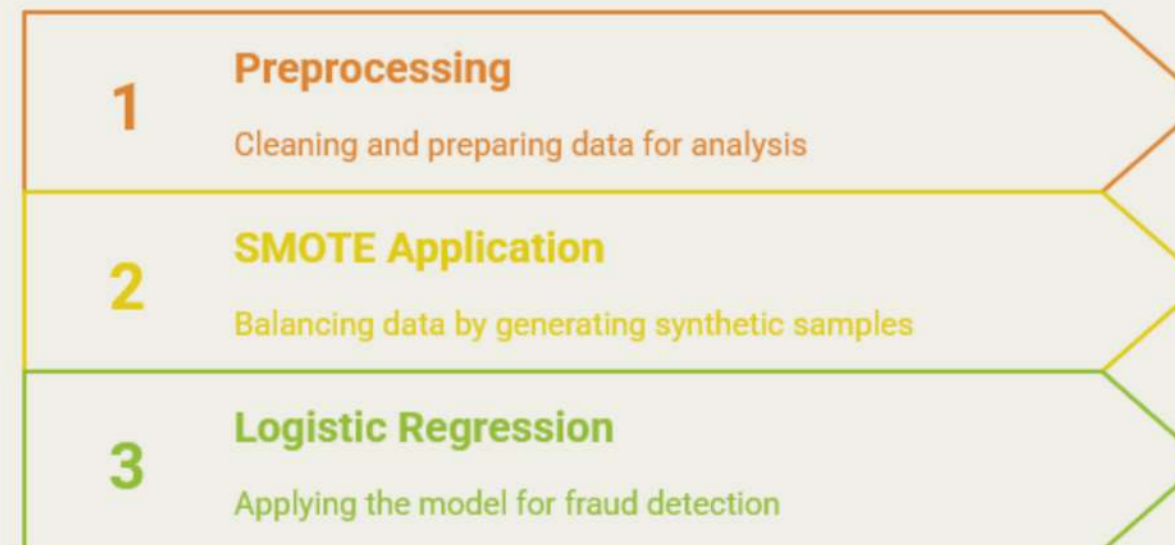
Why Logistic Regression Was the Right Choice:

- In the end, Logistic Regression proved to be a smart choice as it delivered high performance without sacrificing interpretability
- Its simplicity made it easy to tune and explain, while still achieving excellent recall and F1-score for the fraud class
- With proper preprocessing and SMOTE, it handled imbalanced data effectively,

outperforming expectations

54

Enhancing Fraud Detection with Logistic Regression



ML MODULES: RANDOM FOREST

Random Forest Classifier

- **Why it works well here:**

- High Accuracy: Excellent performance on complex datasets, often outperforming single trees.
- Handles Complex Patterns: Captures non-linear relationships and feature interactions, ideal for sophisticated fraud.
- Reduces Overfitting: Ensemble approach of multiple trees minimizes overfitting.
- Feature Importance: Provides insights into key fraud drivers.
- Robustness: Less sensitive to outliers and noisy data.

- **Limitations:**

- Less Interpretable: "Black-box" nature makes individual prediction understanding harder.
- Computational Cost: Can be slower to train and more memory-intensive due to multiple trees.

Advantages of Random Forest in Fraud Detection



ML MODULES: RANDOM FOREST

Key Challenges Encountered

1. Interpretability vs. Performance Trade-off

- While providing feature importance, the ensemble nature makes explaining individual predictions (why a specific transaction was flagged as fraud) difficult.
- With anonymized, PCA-transformed features (V1-V28), this "black-box" nature is exacerbated, limiting direct business insights into feature contributions.
- Focus: Prioritizing high predictive performance (e.g., F1-score) over granular interpretability for stakeholder explanations.

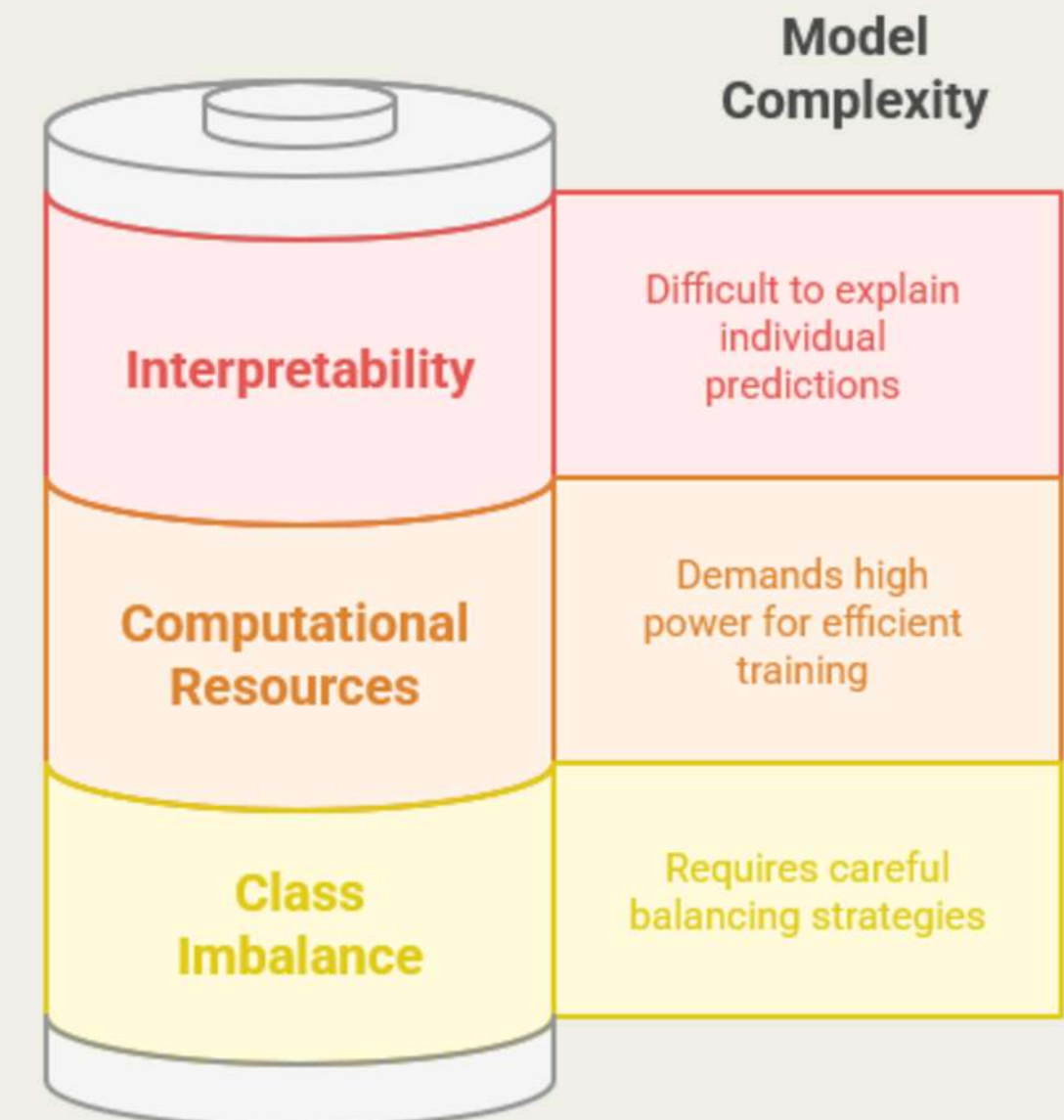
2. Computational Resources for Large Datasets

- Building hundreds of decision trees on large-scale data can be computationally intensive and memory-consuming.
- Training time can be longer compared to simpler models.
- Consequence: Requires sufficient computational power for efficient training and hyperparameter tuning.

3. Sensitivity to Class Imbalance (Even with Robustness)

- Despite being more robust than some single models, severe class imbalance (fraud <0.2%) can still lead to the model being biased towards the majority class.
- Solution: Requires careful application of strategies like `class_weight` parameter adjustment, or external sampling techniques (UnderSampling/SMOTE) to ensure adequate learning from rare fraud cases.

Balancing model complexity with resource demands and data bias.



INITIAL MODEL

OVERALL PERFORMANCE TREND:

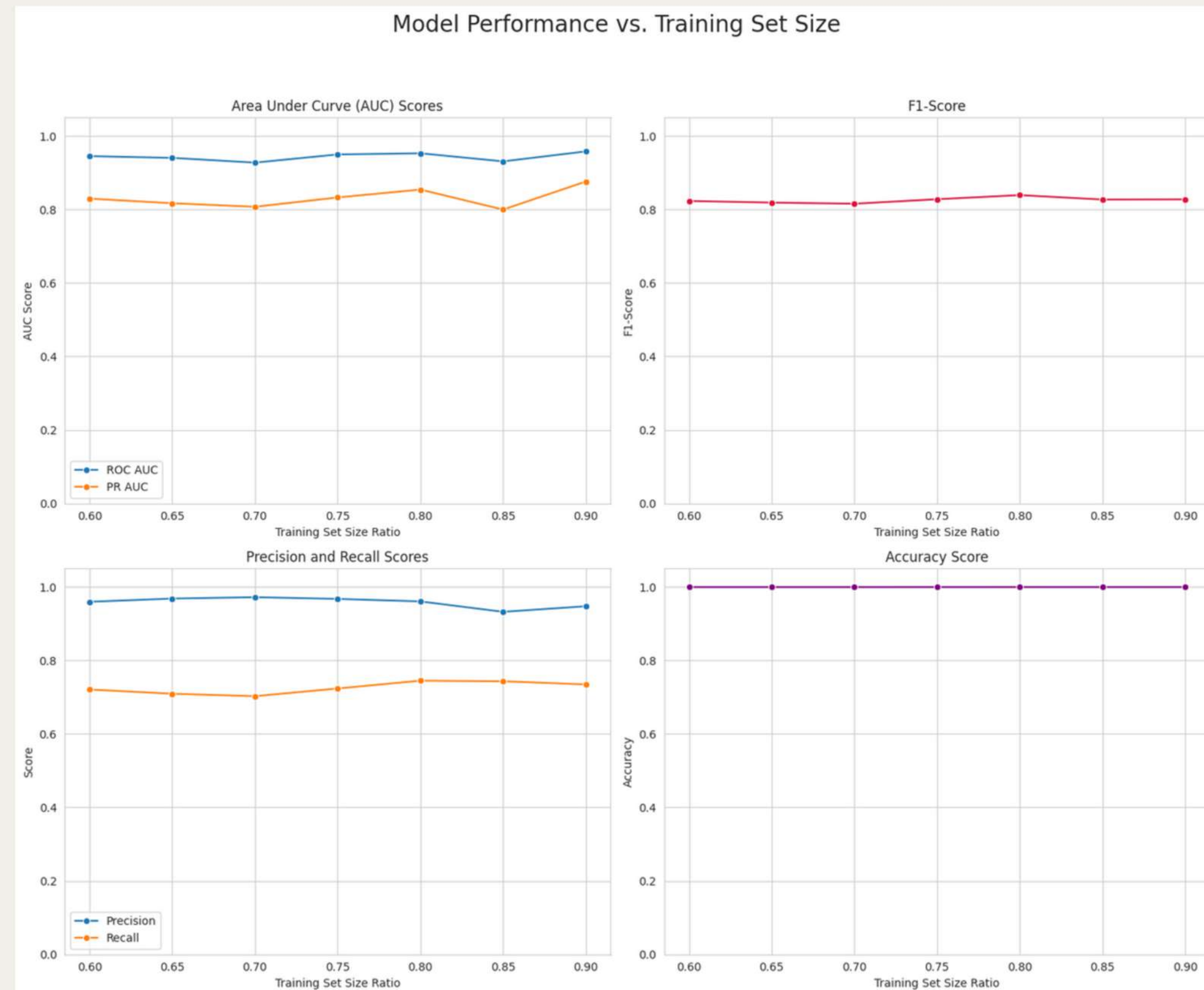
- ACCURACY & F1-SCORE: REMAIN CONSISTENTLY HIGH (NEAR 1.0 AND 0.8+) AND STABLE ACROSS ALL TRAINING SIZES (0.60 TO 0.90).
- AUC SCORES: BOTH ROC AUC (VERY HIGH, NEAR 0.95+) AND PR AUC (GOOD, AROUND 0.8-0.9) ARE STABLE.
- PRECISION & RECALL: PRECISION IS CONSISTENTLY HIGH (NEAR 0.95+), WHILE RECALL IS STABLE AND GOOD (AROUND 0.7-0.75).

LEARNING CURVE INSIGHT:

- MODEL PERFORMANCE HAS ALREADY PLATEAUED BY 60% TRAINING DATA.
- INCREASING TRAINING DATA BEYOND 60% PROVIDES NO SIGNIFICANT PERFORMANCE GAINS.

MODEL ROBUSTNESS:

- INDICATES THE MODEL IS ROBUST AND PERFORMS WELL EVEN WITH 60% OF THE DATA FOR TRAINING WITHIN THIS RANGE.



INITIAL MODEL

CONFUSION MATRIX (TRAIN SIZE: 0.80):

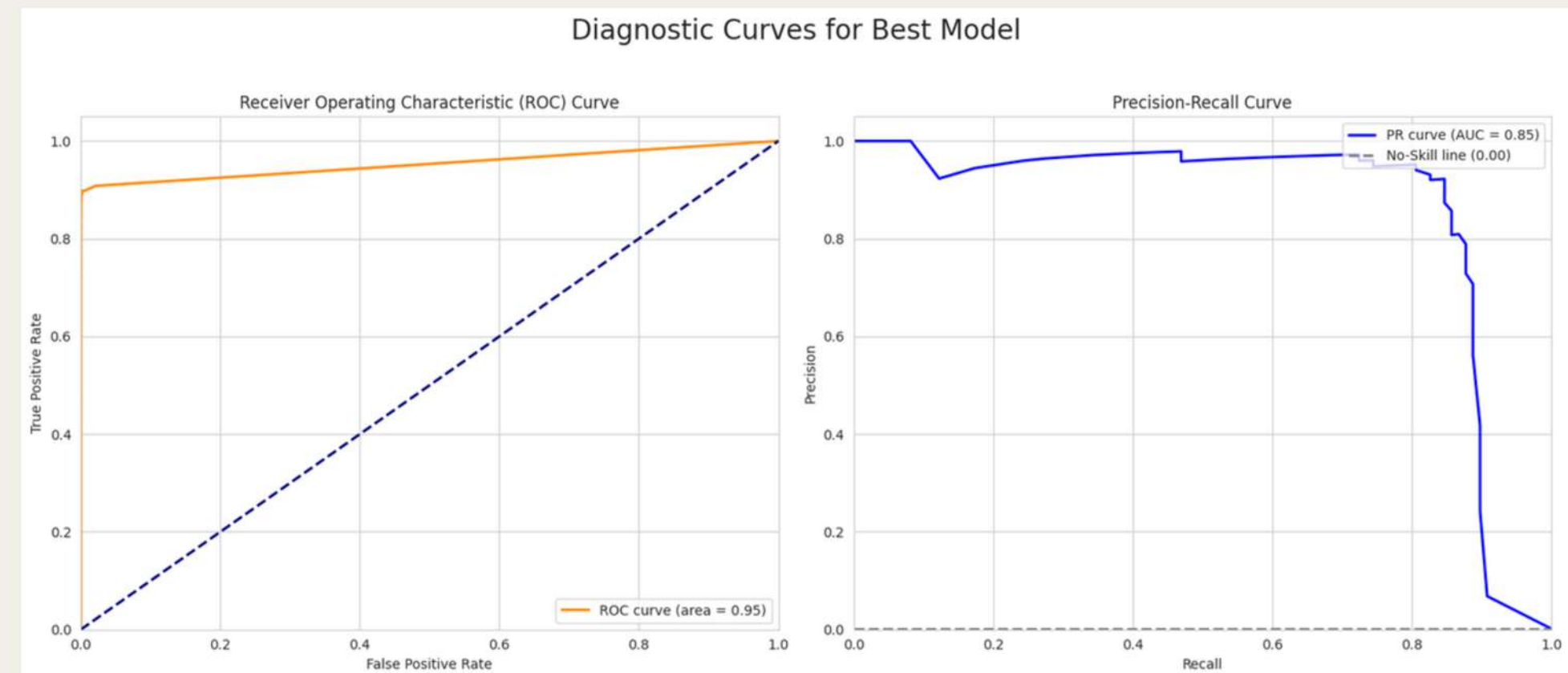
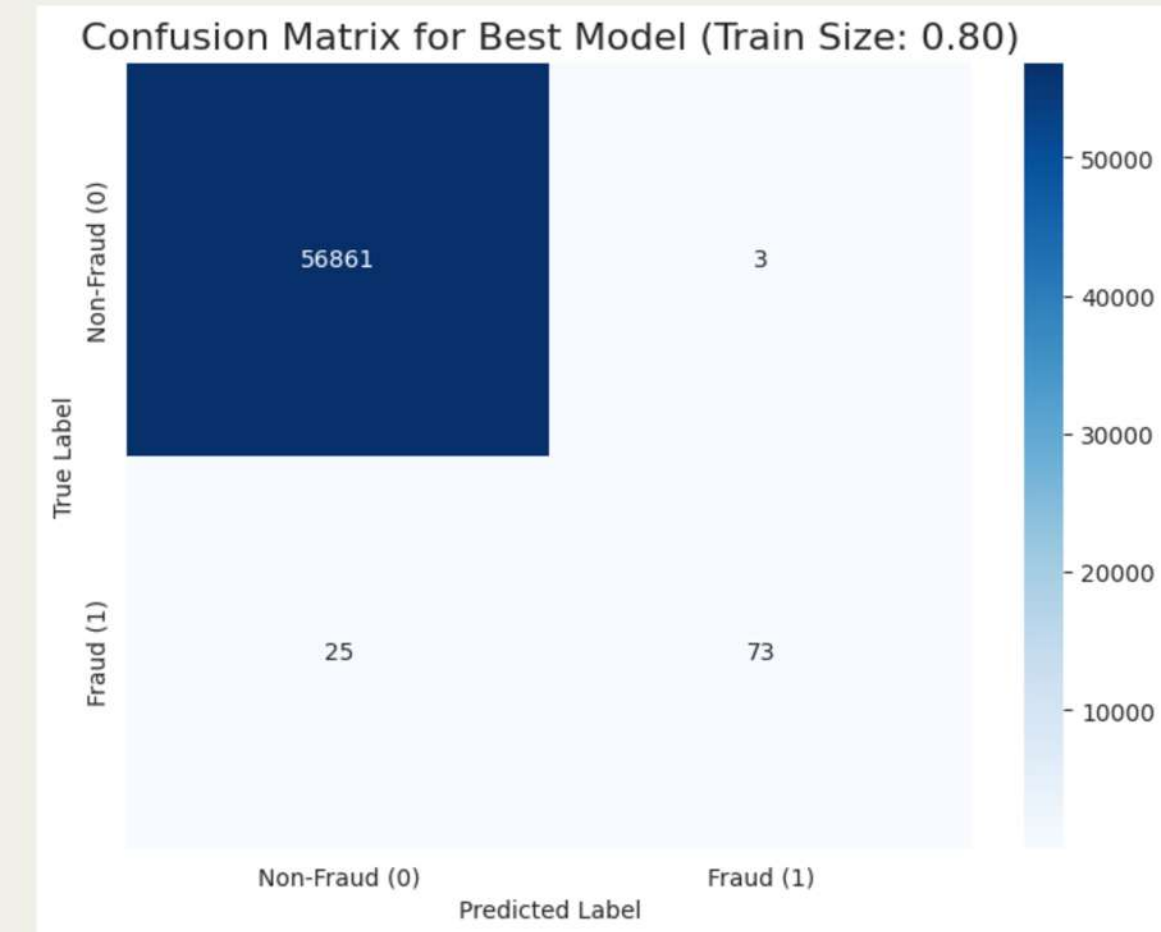
- HIGH TRUE NEGATIVES (56861): MODEL CORRECTLY IDENTIFIED ALMOST ALL NON-FRAUD.
- HIGH TRUE POSITIVES (73): MODEL CAUGHT MOST ACTUAL FRAUD CASES.
- LOW FALSE POSITIVES (3): VERY FEW LEGITIMATE TRANSACTIONS FLAGGED AS FRAUD.
- LOW FALSE NEGATIVES (25): FEW ACTUAL FRAUD CASES WERE MISSED.

ROC CURVE (AUC = 0.99):

- EXCELLENT DISCRIMINATION: MODEL IS SUPERB AT DISTINGUISHING BETWEEN FRAUD AND NON-FRAUD.

PRECISION-RECALL CURVE (AUC = 0.85):

- STRONG FRAUD DETECTION: GOOD PERFORMANCE SPECIFICALLY ON THE RARE FRAUD CLASS.
- SIGNIFICANTLY BETTER THAN RANDOM GUESSING (NO-SKILL LINE AT 0.00).



MODEL WITH EQUAL FRAUD AND NON FRAUD

• MODEL PERFORMANCE INSIGHTS

EXCEPTIONAL STABILITY:

- PERFORMANCE METRICS CONSISTENTLY HIGH AND FLAT (60%-90% TRAIN DATA).
- MODEL ROBUST; NOT SENSITIVE TO TRAIN SET SIZE CHANGES.

NEAR-PERFECT SCORES:

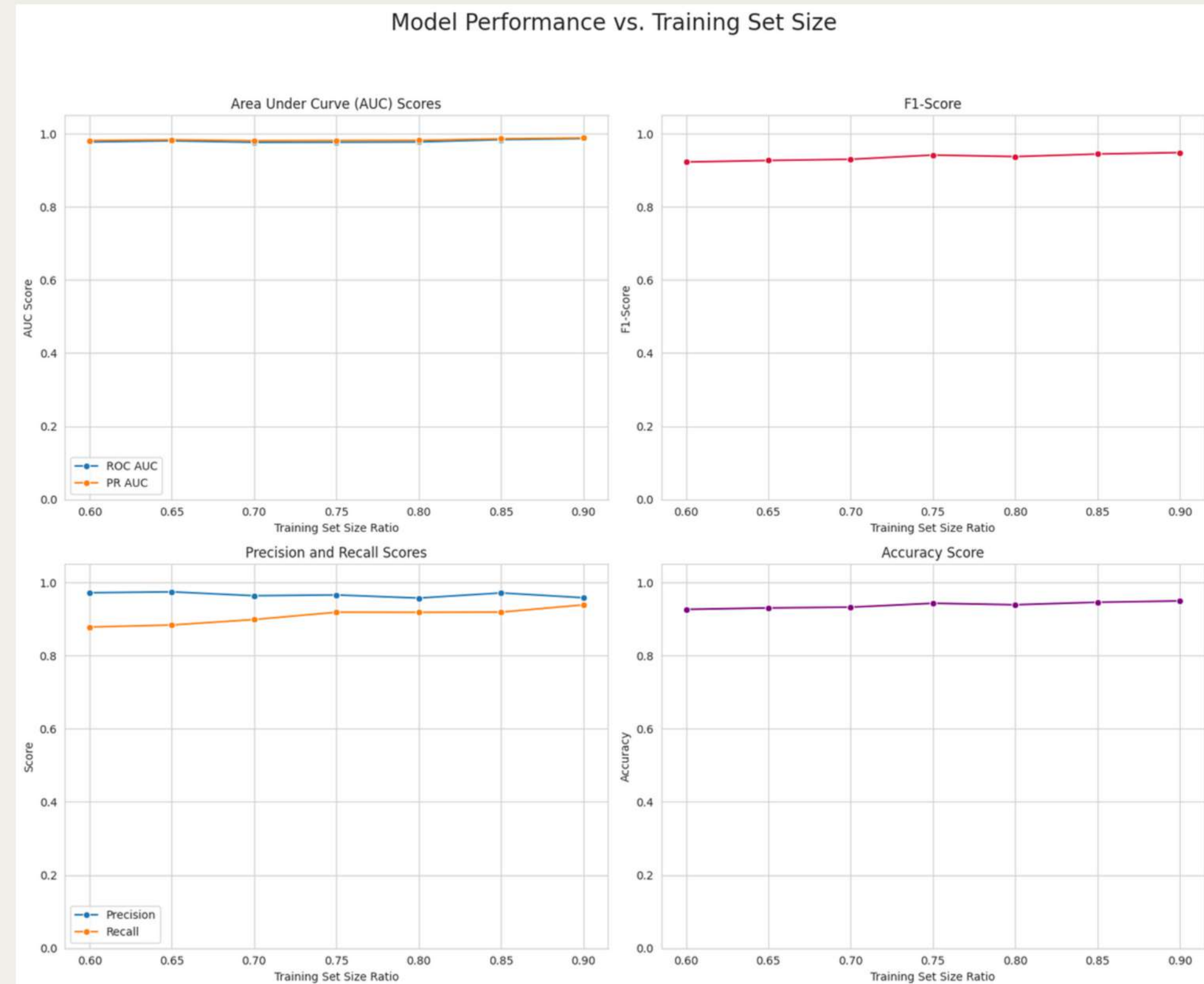
- ALL METRICS (AUCS, F1, PRECISION, RECALL, ACCURACY) APPROACH 1.0.
- INDICATES EXCELLENT DISCRIMINATION AND LOW ERROR RATES.

SATURATED LEARNING:

- NO TYPICAL "LEARNING CURVE" CLIMB; PERFORMANCE ALREADY MAXIMAL AT 60% TRAIN DATA.
- MORE DATA WITHIN THIS RANGE YIELDS NO SIGNIFICANT IMPROVEMENT.

DATA SPLIT FLEXIBILITY:

- CAN USE SMALLER TRAINING SETS (E.G., 60-70%) EFFICIENTLY.
- ALLOWS MORE DATA FOR TESTING/VALIDATION.



MODEL WITH EQUAL FRAUD AND NON FRAUD

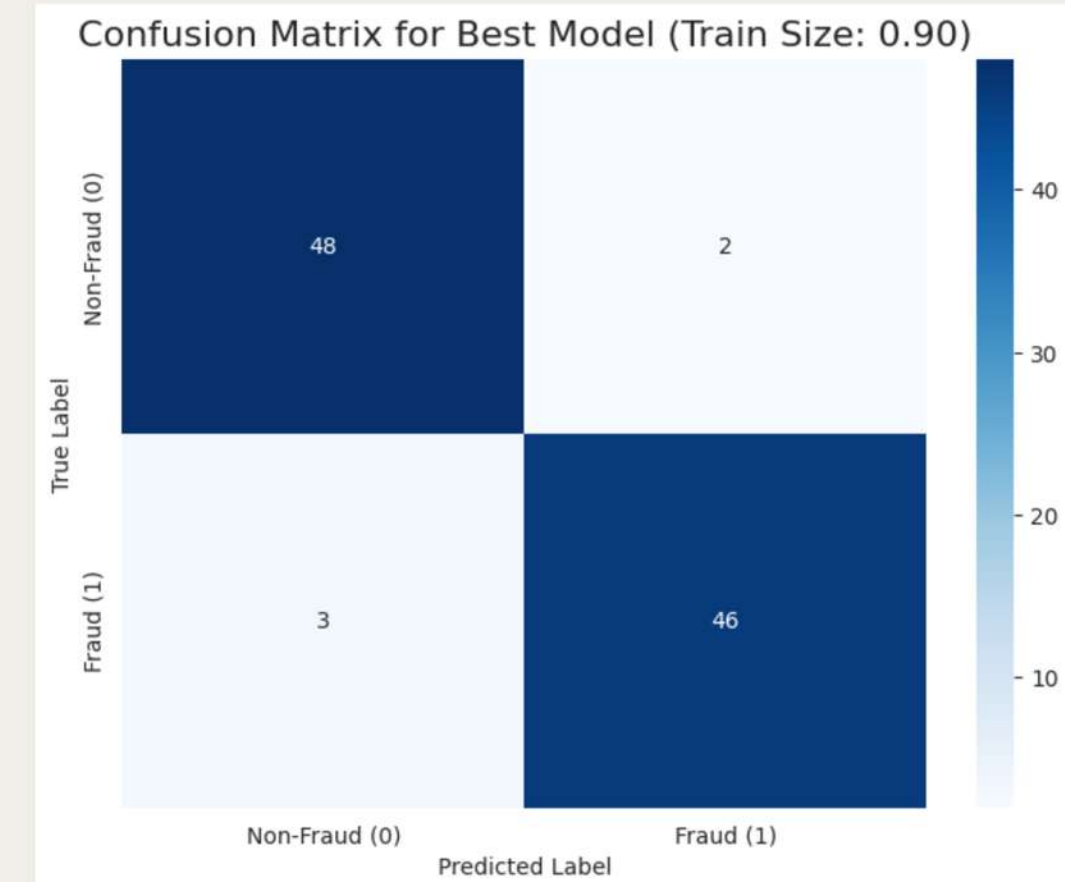
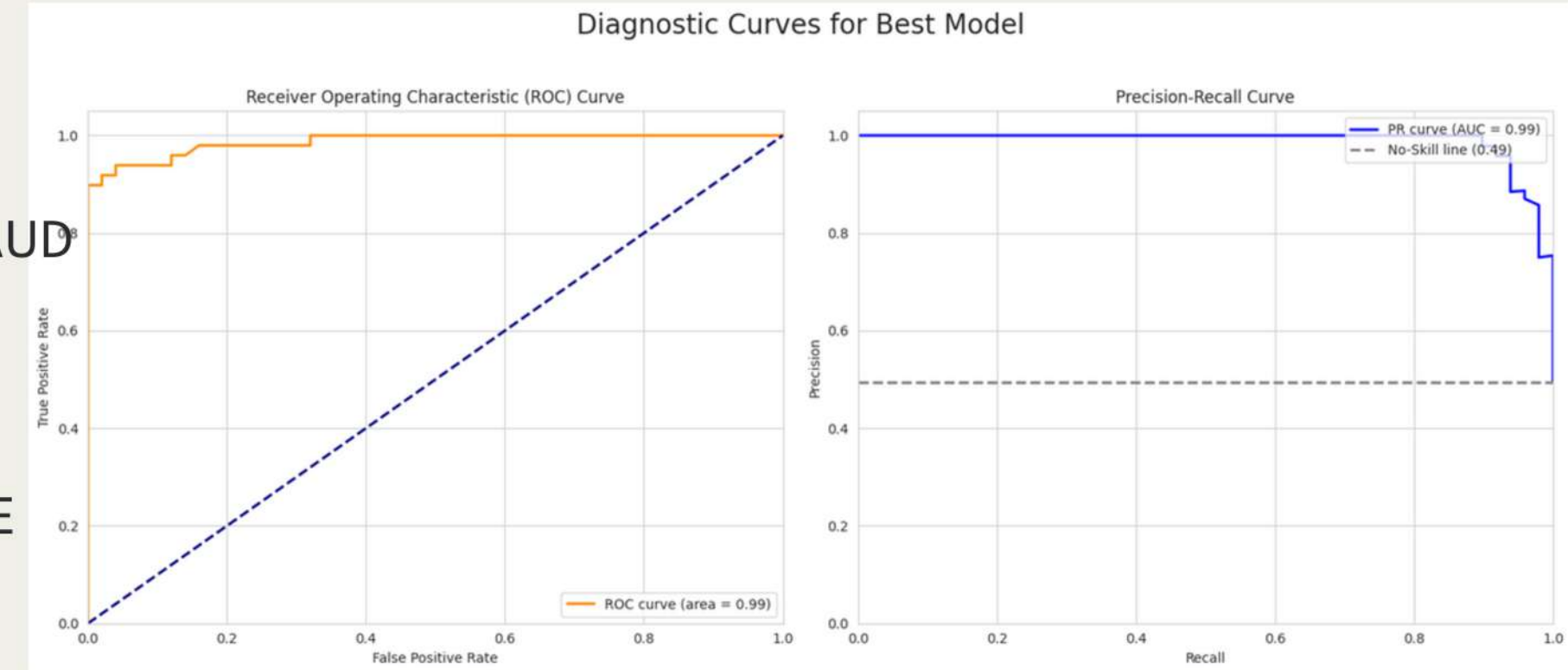
- **MODEL PERFORMANCE: DEEP DIVE**

- **ROC CURVE (AUC = 0.99):**
- EXCELLENT DISCRIMINATION: MODEL CLEARLY SEPARATES FRAUD FROM NON-FRAUD.

- **PRECISION-RECALL CURVE (AUC = 0.99):**
- STRONG FRAUD DETECTION: HIGH PERFORMANCE ON THE RARE FRAUD CLASS.
- SIGNIFICANTLY OUTPERFORMS RANDOM GUESSING.

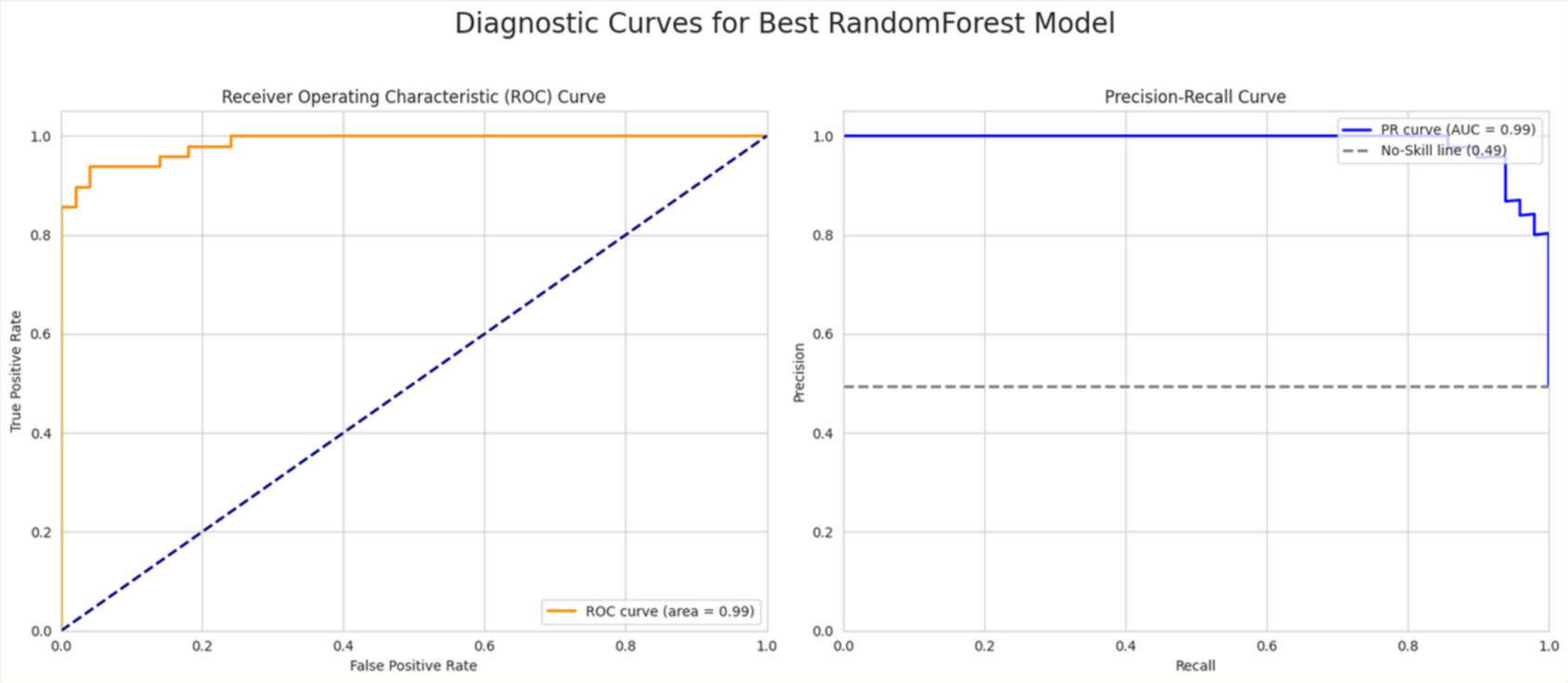
- **CONFUSION MATRIX (TRAIN SIZE: 0.90):**

- HIGH ACCURACY (TN=48, TP=46): CORRECTLY IDENTIFIES MOST CASES.
- MINIMAL FALSE ALARMS (FP=2): LOW MISCLASSIFICATION OF NON-FRAUD AS FRAUD.
- FEW MISSED FRAUDS (FN=3): EFFECTIVELY CATCHES ACTUAL FRAUD INSTANCES.



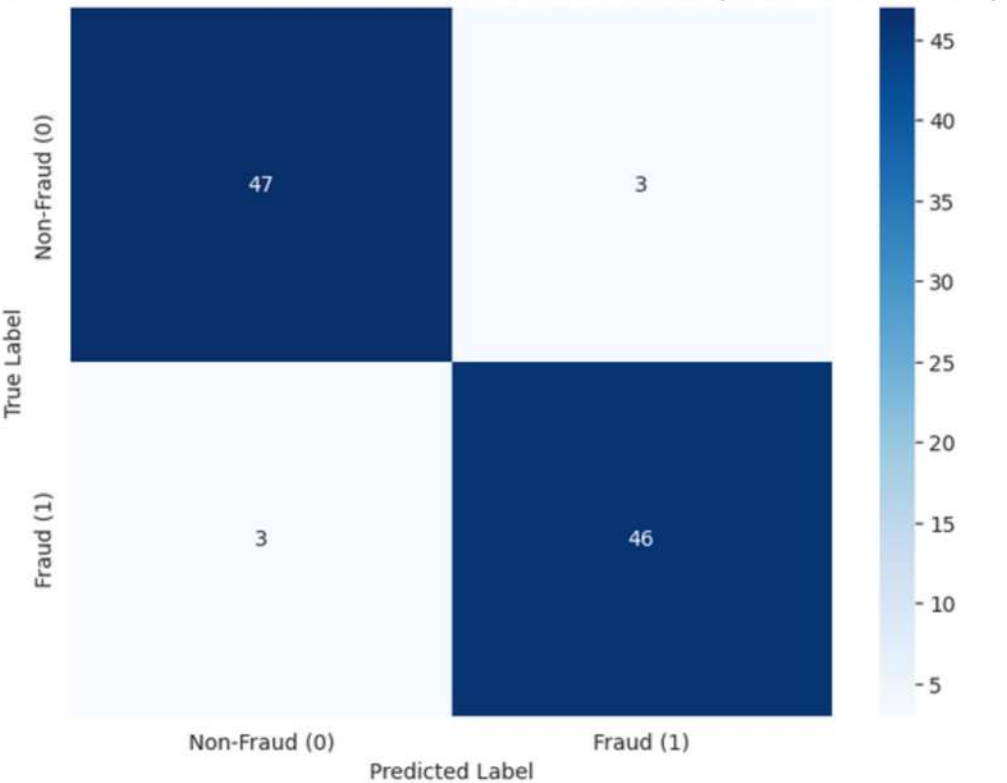
TUNNING WITH GRIDSEARCHCV

- **ROC CURVE (AUC = 0.99):** DEMONSTRATES OUTSTANDING ABILITY TO DISCRIMINATE BETWEEN FRAUDULENT AND NON-FRAUDULENT TRANSACTIONS, VERY CLOSE TO PERFECT.
- **PRECISION-RECALL CURVE (AUC = 0.99):** SHOWS SUPERIOR PERFORMANCE ON THE RARE FRAUD CLASS, EFFECTIVELY BALANCING PRECISION (MINIMIZING FALSE ALARMS) AND RECALL (CATCHING MOST FRAUD).
- **OPTIMAL PARAMETERS (TABLE):** DETAILS THE BEST-PERFORMING RANDOMFOREST CONFIGURATION FOUND BY GRIDSEARCHCV, INCLUDING N_ESTIMATORS=50, MAX_DEPTH=10, AND CLASS_WEIGHT='BALANCED' FOR IMBALANCE HANDLING.
- **TEST SET PERFORMANCE (TABLE):** VALIDATES HIGH REAL-WORLD EFFECTIVENESS:
 - ACCURACY: 0.9394
 - F1-SCORE: 0.9388 (EXCELLENT BALANCE OF PRECISION AND RECALL)
 - ROC AUC: 0.9861 & PR AUC: 0.9875 (CONFIRMING ROBUST DISCRIMINATION FOR FRAUD DETECTION).
- **CONFUSION MATRIX:** PROVIDES A CLEAR BREAKDOWN OF PREDICTIONS:
 - CORRECTLY IDENTIFIED: 47 NON-FRAUD, 46 FRAUD.
 - MISCLASSIFICATIONS: ONLY 3 FALSE POSITIVES (FALSE ALARMS) AND 3 FALSE NEGATIVES (MISSED FRAUD), INDICATING A HIGHLY EFFECTIVE MODEL WITH MINIMAL ERRORS.

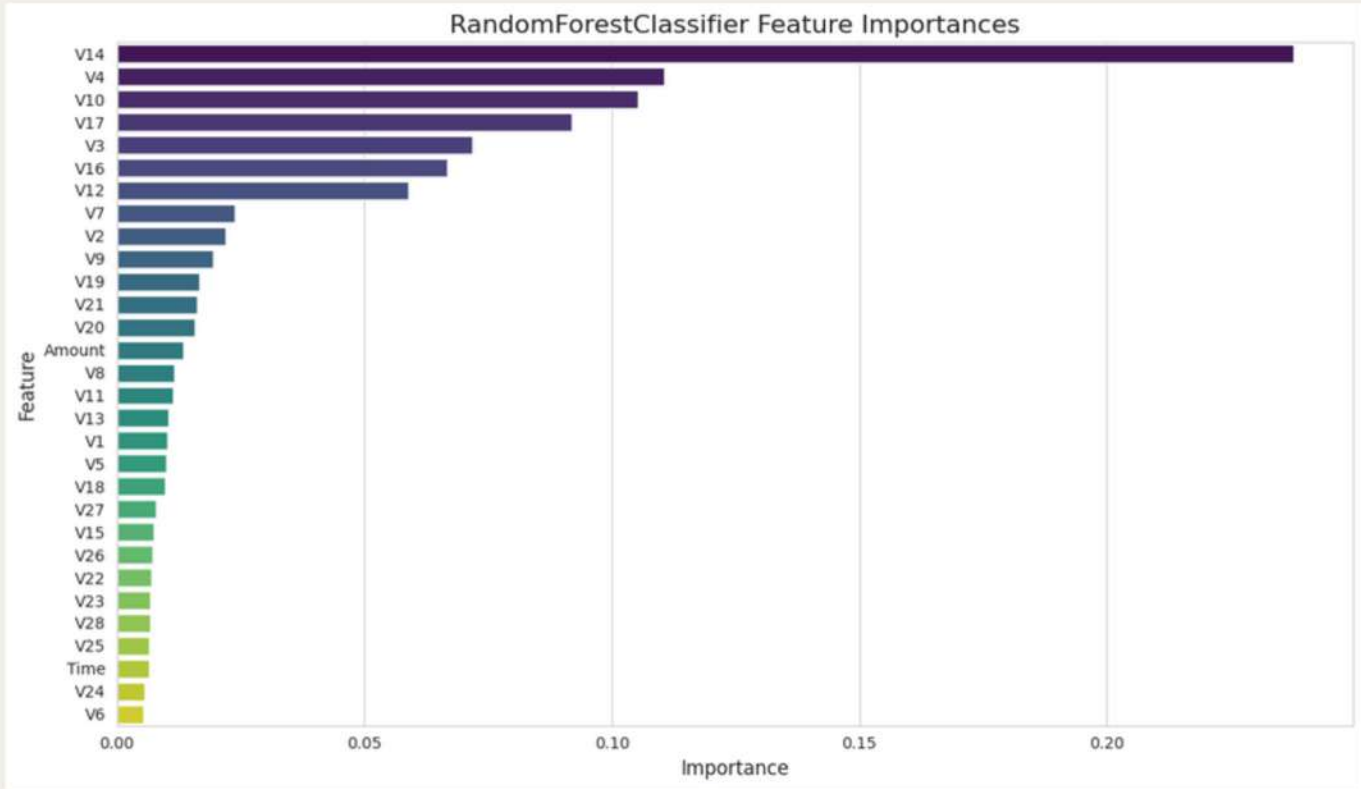
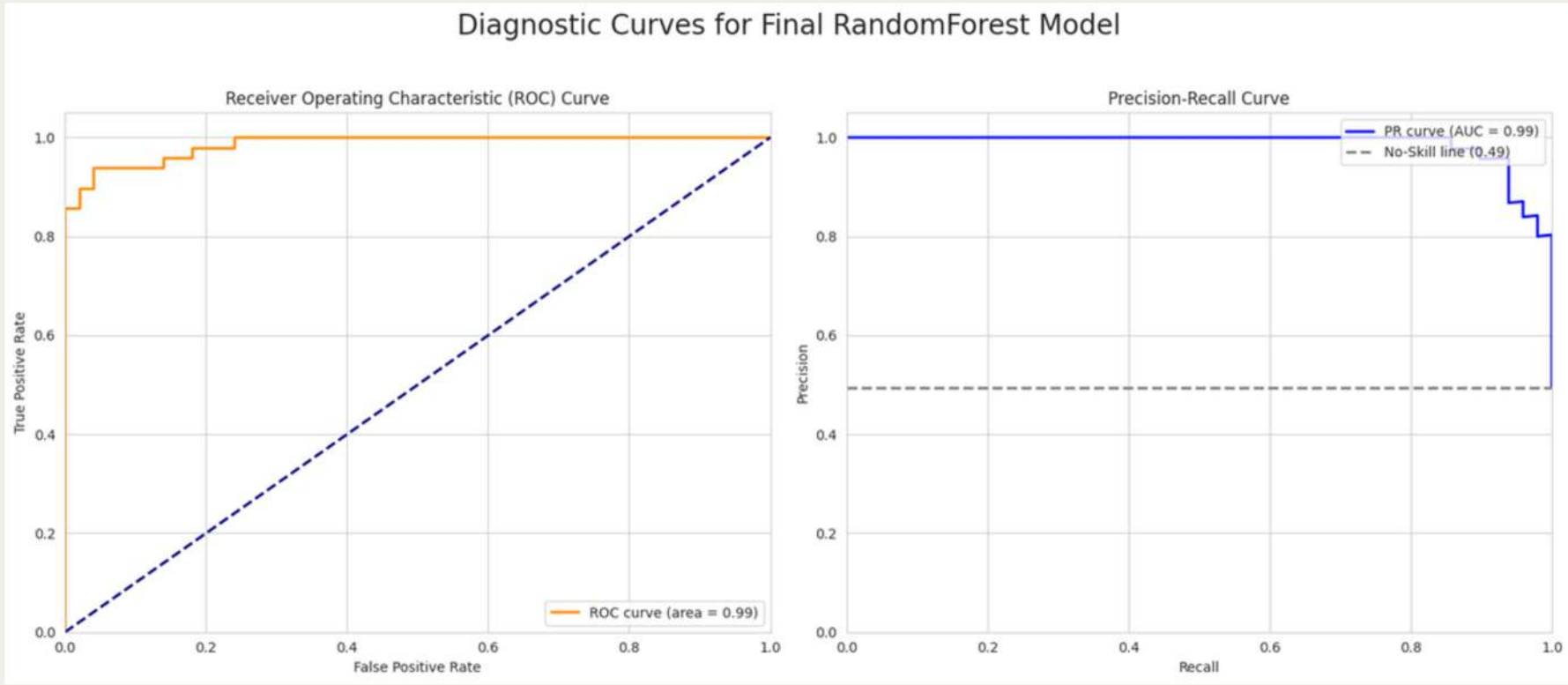


Metric	Value
Best Parameters (from GridSearchCV)	
class_weight	balanced
criterion	entropy
max_depth	10
min_samples_leaf	1
min_samples_split	2
n_estimators	50
Best F1-score (from GridSearchCV)	
Evaluation on Test Set	
Accuracy	0.9394
Precision	0.9388
Recall	0.9388
F1-Score	0.9388
ROC AUC	0.9861
PR AUC	0.9875

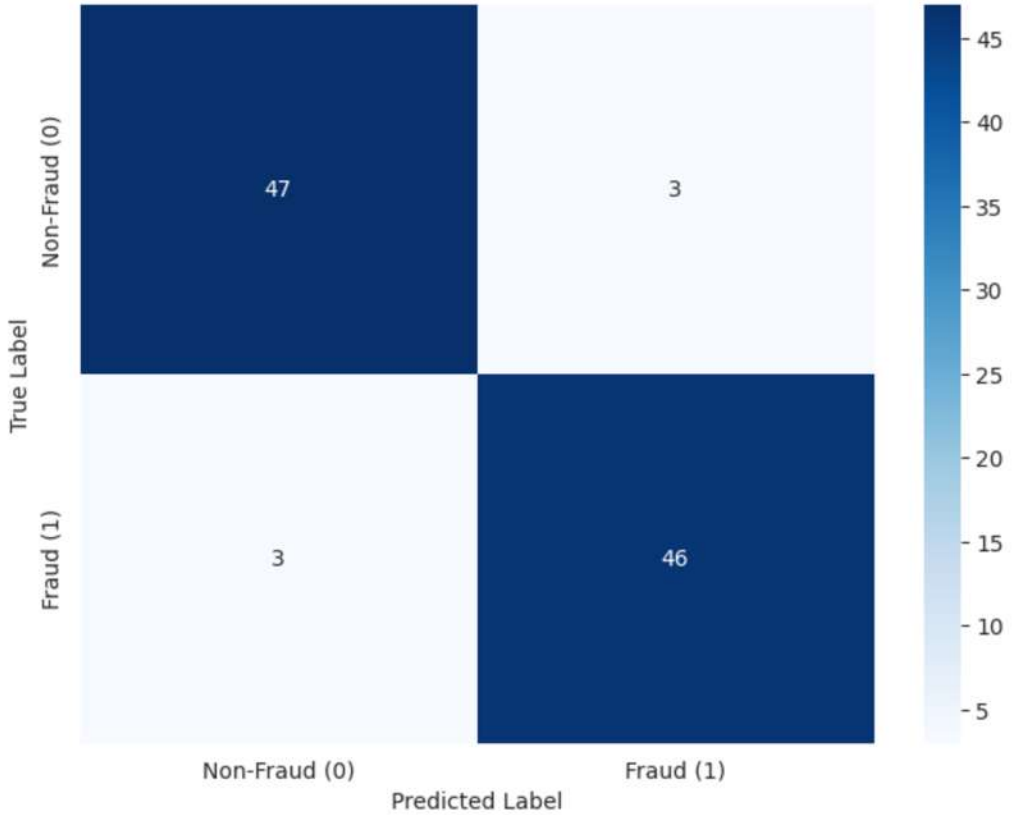
Confusion Matrix for Best RandomForest Model (Train Size: 0.90)



PLOTTING BEST MODEL: RANDOM FOREST



Confusion Matrix for Final RandomForest Model (Train Size: 0.90)



A	B	C
Section	Metric/Description	Value
Training Details	Train/Test Split	90% / 9%
	Train Set Composition (Non-Fraud)	442
	Train Set Composition (Fraud)	443
Performance Metrics (Final Model)		
	Accuracy	0.9394
	Precision	0.9388
	Recall	0.9388
	F1-Score	0.9388
	ROC AUC	0.9861
	PR AUC	0.9875

SUMMARY OF THREE MODEL

ML Module	Key Characteristics / Pros	Accuracy Score
XGBoost Classifier	- High F2-Score	~ 86%
	- Effectively minimizes missed fraud (low false negatives)	
	- Learns complex, non-linear patterns	
	- Very fast real-time predictions	
	- Robust to outliers	
Logistic Regression	- Simple	~93
	- Highly interpretable	
	- Fast for real-time predictions and updates	
Random Forest	- Excellent F2-Scores	~93
	- Minimal false negatives	
	- Strong outlier robustness	
	- Ability to learn complex patterns	
	- Fast real-time predictions	

REFERENCE

- Chat GPT - <https://chatgpt.com/>
- Google Gemini - <https://gemini.google.com/>
- Kaggle - <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data>
- Scikit Learn - <https://scikit-learn.org/stable/>
- Felix - <https://felix.hs-furtwangen.de/>
- Napkin.Ai - <https://www.napkin.ai/>
- Jupyter - <https://jupyter.org/>
- Anaconda Navigator - <https://www.anaconda.com/>
- VS Code - <https://code.visualstudio.com/>
- Google - <https://www.google.com/>
- XGBoost - https://xgboost.readthedocs.io/en/latest/python/python_api.html
- Random Forest - <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Logistic Regression - https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Thank you!

ANY QUESTIONS ?